BOOKVERSE

Technical Study Guide

Complete architecture, feature & interview reference

A ground-up walkthrough of the Bookverse library platform — a MERN application that runs **MongoDB and Cloud Firestore side by side**, authenticates with Firebase, and takes real payments through Stripe Checkout. Every explanation in this document is derived from the actual source code in the repository, not from a design plan.

React 19 · Vite 8 · Tailwind CSS 3 · Framer Motion · React Router 7 · Axios · Node.js · Express 4 · MongoDB + Mongoose 8 · Cloud Firestore · Firebase Auth · Firebase Admin SDK · Stripe Checkout · Vercel

PREPARED FOR
Technical interview preparation

SOURCE OF TRUTH
D:\Library Project · branch `main`

SECTIONS
28 + glossary

Contents

Every section below describes the codebase as it exists today. Where a capability was planned but never built, it is labelled explicitly rather than described as if it shipped.

HOW THIS DOCUMENT WAS PRODUCED

Every file in `backend/` and `frontend/src/` was read in full, along with `firestore.rules`, both `vercel.json` files, the seed scripts and the complete git history. Code excerpts are copied verbatim from the repository. Section 28 reconstructs the project's real bug history from commit messages and diffs, and labels anything that cannot be established from evidence.

SECTION ONE

Complete Project Overview

What Bookverse is, what it does, which technologies actually appear in the repository, and how every moving part talks to every other moving part.

1.1 What Bookverse is

Bookverse is a **full-stack digital library platform**. It is not a single-purpose CRUD app — it combines three distinct "shelves", each with its own economics and its own storage engine:

SHELF	WHAT IT IS	HOW A BOOK LEAVES IT	WHERE ITS DATA LIVES
Official Collection	The library's own holdings, curated by an administrator	Borrowed (comes back), or bought permanently with contribution points	MONGODB `books` + `borrows` collections
Community Shelf	Second-hand books that ordinary members share with each other	Borrowed peer-to-peer, with the owner approving each request	FIRESTORE `usedBooks` + `borrowRequests`
Store	Brand-new copies for sale in Bangladeshi Taka	Bought with real money through Stripe Checkout	FIRESTORE `storeBooks`, `carts`, `orders`

Sitting across all three is a **contribution-point economy**: sharing a used book on the community shelf earns 50 points, and 200 points buys an official Collection book outright. That single mechanic is what ties the three shelves into one product rather than three separate apps bolted together — generosity on the community shelf is what pays for ownership on the official shelf.

1.2 What a user can actually do

As a member

- Register / sign in with email + password or Google
- Browse, search, filter and sort the official Collection
- Borrow an official book (delivery details required)
- Return a borrowed book
- Buy an official book permanently with 200 points
- Share a used book and earn 50 points
- Request to borrow someone else's used book
- Approve / decline requests for their own books
- Receive live notifications when either happens
- Write one review per book, on either shelf
- Wishlist books from both shelves
- Shop the Store, manage a cart, pay via Stripe
- View order history and purchase history

As an administrator

- Self-promote once using a server-held admin secret
- Create, edit and delete official Collection books
- See Collection statistics (total / available / borrowed / sold / categories)
- Create, edit and delete Store books and prices
- See every registered member, their role and join date
- See exactly which books each member currently has on loan, and their full borrow history

> **NOTE**
>
> An admin has **no** power over the community shelf. Used books belong to the members who shared them; only the owner can withdraw one. That is a deliberate ownership boundary, not a missing feature.

1.3 The technologies actually used

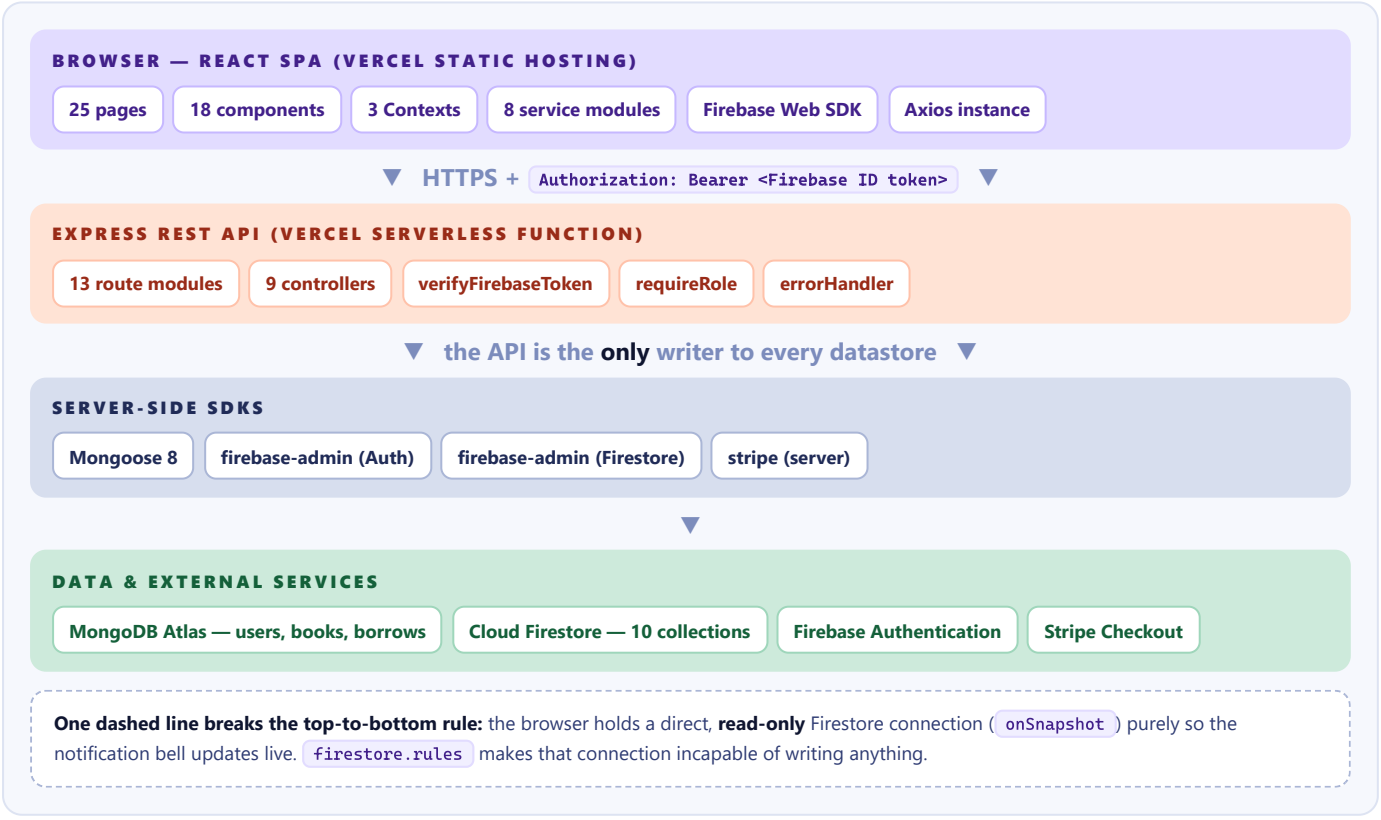
Taken from `frontend/package.json` and `backend/package.json` — these are the real dependency lists, not an aspirational stack.

TECHNOLOGY	VERSION	RESPONSIBILITY IN BOOKVERSE
React	19.2.8	The entire user interface. Function components + hooks only; no class components anywhere.
Vite	8.2.0	Dev server and production bundler. Supplies `import.meta.env` for configuration.
React Router DOM	7.18.2	Client-side routing, route parameters, and the query-string-as-state pattern used by every listing page.
Tailwind CSS	3.4.19	All styling, via a custom navy/coral/indigo/cream palette defined in `tailwind.config.js`.
Framer Motion	13.1.0	Page transitions, list entry animations, modal enter/exit, the animated admin tab underline.
Axios	1.19.0	Every HTTP call to the backend. Its request interceptor is where the Firebase token is attached.
Firebase (web SDK)	12.18.0	Client-side authentication, and a *read-only* Firestore connection used solely for the live notification listener.
react-icons	5.7.0	Feather (`Fi*`) and Font Awesome (`Fa*`) icon sets.
Node.js + Express	4.19.2	The REST API. CommonJS modules throughout the backend.
Mongoose	8.5.0	Schemas, models and queries for the three MongoDB collections.
firebase-admin	12.7.0	Server-side ID-token verification *and* all privileged Firestore writes.
stripe (server SDK)	22.5.0	Creating Checkout Sessions and verifying completed payments.
cors, **dotenv**	2.8.5 / 16.4.5	Cross-origin access for the separately-deployed frontend; environment configuration.
nodemon, **oxlint**	3.1.4 / 1.75.0	Dev-only: backend auto-restart, frontend linting.

ACCURACY CHECK — WHAT IS NOT IN THIS STACK

There is **no Firebase Cloud Functions** directory, no `firebase.json` and no `firestore.indexes.json` anywhere in the repository. There is no Stripe browser SDK (`@stripe/stripe-js` was deliberately removed — see §28.11), no Stripe webhook endpoint, no Redis, no Firebase Storage upload flow, and no test framework. Every "server-side" behaviour in Bookverse happens inside the Express process.

1.4 The overall architecture



One dashed line breaks the top-to-bottom rule: the browser holds a direct, **read-only** Firestore connection (`onSnapshot`) purely so the notification bell updates live. `firestore.rules` makes that connection incapable of writing anything.

1.5 How each part communicates with the others

Browser → Express

Every call goes through one shared Axios instance (`frontend/src/services/api.js`). Its request interceptor waits for Firebase to restore the session, then attaches the current ID token as a bearer header. No page ever constructs a token or a header by hand.

Browser → Firebase Authentication (direct)

Sign-up, sign-in, Google OAuth and sign-out are handled by the Firebase Web SDK talking straight to Google's servers. **The Express API never sees a password.** Bookverse's backend only ever receives an already-issued ID token, which it verifies cryptographically.

Browser → Firestore (direct, read-only)

`NotificationBell.jsx` opens an `onSnapshot` listener on the `notifications` collection filtered to the signed-in user's UID. This is the only direct database connection the browser holds, and the security rules permit reads only.

Express → MongoDB

Through Mongoose, connected once at boot by `config/db.js` . Holds the three entities that behave like classic relational records: `User` , `Book` , `Borrow` .

Express → Firestore

Through the Firebase Admin SDK, which **bypasses security rules entirely** because it authenticates as a service account. This is precisely why the rules can deny all client writes without breaking the app.

Express → Firebase Auth (verification)

`admin.auth().verifyIdToken(token)` checks the token's signature against Google's public keys, its expiry, its issuer and its audience. A forged or expired token cannot survive this check.

Express ↔ Stripe

The server creates a Checkout Session with prices it computed itself, hands the browser a Stripe-hosted URL, and later re-fetches that session from Stripe to confirm the payment really completed. **Bookverse never touches a card number.**

1.6 Responsibility of each major technology — one line each

TECHNOLOGY	OWNS	EXPLICITLY DOES NOT OWN
Firebase Authentication	Identity: who this person is, password storage, Google OAuth, session persistence	Permissions — it has no idea what an "admin" is
MongoDB / Mongoose	The user's role, the official book catalogue, loan records	Anything added after the Firestore convention was adopted
Cloud Firestore	Points, community books, requests, notifications, reviews, wishlist, purchases, store, carts, orders	Authorisation decisions — it only enforces read scoping
Firebase Admin SDK	Verifying tokens; performing every privileged Firestore write	Ever running in the browser
Express	All business rules, all authorisation, all pricing	Rendering — it serves JSON only
Stripe	Card capture, PCI compliance, the payment page itself	Deciding what anything costs
React + Router	Presentation, local UI state, navigation	Security — its guards are convenience only

> **THE ONE SENTENCE TO REMEMBER**
>
> **Firebase says who you are; MongoDB says what role you have; Express decides what you may do; Firestore stores the result; and the browser is never trusted with any of those decisions.**

SECTION TWO

Complete Project Structure

Why each folder and file exists, what it does, who uses it, and how it connects to the rest of the system.

2.1 Repository root

```
D:\Library Project\
├── backend\            Express REST API (CommonJS)
├── frontend\           React SPA (ES modules)
├── firestore.rules     Firestore access control — deployed to Firebase, not to a server
└── .claude\launch.json Local tooling config; not application code
```

`firestore.rules` sitting at the *root* rather than inside either app is meaningful: it belongs to neither the frontend nor the backend. It is deployed to Google's infrastructure and enforced by Firestore itself, so it protects data even if both applications were switched off. Section 20 covers it line by line.

2.2 Backend structure

```
backend\
├── server.js                local entry point — starts the HTTP listener
├── app.js                   builds the Express app: middleware + route mounting
├── api\index.js             Vercel serverless entry point — exports the same app
├── vercel.json              routes every incoming path into api/index.js
├── config\db.js             the single MongoDB connection
├── lib\
│   ├── firebaseAdmin.js     initialises the Admin SDK from env credentials
│   ├── firestore.js         Firestore handle + collection names + points constants
│   └── stripe.js            Stripe client + BDT currency helpers
├── middleware\
│   ├── auth.js              verifyFirebaseToken + requireRole
│   └── errorHandler.js      terminal error middleware
├── models\                  Mongoose schemas — MongoDB only
│   ├── User.js  ├── Book.js  └── Borrow.js
├── routes\                  13 routers: URL shape + which middleware guards it
│   ├── auth.js      ├── books.js     ├── borrow.js    ├── stats.js
│   ├── community.js ├── points.js    ├── reviews.js   ├── notifications.js
│   ├── wishlist.js  ├── store.js     ├── cart.js      ├── checkout.js
│   └── admin.js
├── controllers\             9 controllers: the business logic
│   ├── authController.js      ├── communityController.js ├── pointsController.js
│   ├── reviewController.js    ├── wishlistController.js  ├── storeController.js
│   ├── cartController.js      ├── checkoutController.js  └── adminController.js
└── seed\
    ├── seed.js              wipes + repopulates MongoDB books
    └── seedStore.js         wipes + repopulates Firestore storeBooks
```

The three entry points, and why there are three

`app.js` **builds** the application but never starts a server. `server.js` requires it and calls `listen()` — that is local development. `api/index.js` requires it and simply exports it — that is Vercel, where the platform owns the listener and invokes the exported handler per request.

```
// backend/server.js — local only
const app = require("./app");
const PORT = process.env.PORT || 5050;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

```
// backend/api/index.js — serverless only
const app = require("../app");
module.exports = app;
```

WHY THIS SPLIT MATTERS

A serverless function must not call `listen()` — there is no long-lived port to bind. Separating "build the app" from "start the app" lets one codebase serve both models. This split was made deliberately in commit `2313592` (see §28.9).

Folder-by-folder responsibilities

FOLDER	WHY IT EXISTS	HOW IT CONNECTS
`config/`	Isolate infrastructure connection code from business code	`app.js` calls `connectDB()` once at boot; every model then uses the shared Mongoose connection implicitly
`lib/`	Third-party clients created **once** and shared	`firebaseAdmin` → used by `middleware/auth.js` and every Firestore controller. `firestore.js` re-exports the db handle plus `COLLECTIONS` and the points constants. `stripe.js` is used only by `checkoutController`.
`middleware/`	Cross-cutting concerns that apply to many routes	Routers import and attach them; controllers then read `req.user` as a guaranteed value
`models/`	Shape + validation of MongoDB documents	Imported by `routes/books.js`, `routes/borrow.js`, `routes/stats.js`, and the auth/points/review/admin controllers
`routes/`	Map URLs to handlers and declare the guard for each	Mounted onto path prefixes in `app.js`; delegate to controllers
`controllers/`	The actual rules — validation, ownership, transactions	Talk to Mongoose and/or Firestore; never know their own URL
`seed/`	Reproducible demo data	Standalone scripts run via `npm run seed` / `npm run seed:store`; never imported by the app

AN HONEST INCONSISTENCY WORTH BEING ABLE TO EXPLAIN

Four routers keep their logic *inline* instead of delegating to a controller: `books.js`, `borrow.js`, `stats.js` and `notifications.js`. These are the **oldest** files in the project — they predate the controllers folder, which arrived with the Firebase migration. Everything written afterwards follows router → controller. In an interview, name this as a known inconsistency and say the fix is to extract `bookController.js` and `borrowController.js`. That reads far better than pretending the codebase is uniform.

2.3 Frontend structure

```
frontend\
├── index.html              the single HTML shell; loads Fraunces + Manrope fonts
├── vite.config.js          Vite + React plugin
├── tailwind.config.js      custom palette, fonts, shadows
├── vercel.json             SPA rewrite + the Firebase auth-handler proxy (critical, see §28.1)
└── src\
    ├── main.jsx            React root; nests Router → Auth → Wishlist → Cart providers
    ├── App.jsx             route table, Navbar/Footer shell, AnimatePresence
    ├── firebase.js         Firebase Web SDK init; exports auth + read-only firestore
    ├── context\            AuthContext · CartContext · WishlistContext
    ├── services\           api.js (Axios) + 7 typed API wrappers
    ├── pages\              25 route components
    ├── components\         18 shared components, incl. admin\ (3 tabs)
    ├── utils\              authError.js · currency.js
    └── data\genres.js      static genre cards for the marketing pages
```

The files that hold the architecture together

FILE	WHAT IT DOES	WHO DEPENDS ON IT
`main.jsx`	Mounts React and establishes provider nesting order	Nothing — it is the root. Its *order* matters: Cart and Wishlist both call `useAuth()`, so `AuthProvider` must wrap them.
`App.jsx`	Declares all 25 routes and which are protected	Every page reaches the user through here
`firebase.js`	Initialises the SDK; exports `auth` and `firestore`	`api.js`, `AuthContext`, `NotificationBell`
`services/api.js`	The Axios instance + token interceptor	**Every** other service, plus pages that call the API directly
`context/AuthContext.jsx`	Auth state, profile, points, and all auth actions	Navbar, ProtectedRoute, both Cart/Wishlist contexts, and ~12 pages
`components/ProtectedRoute.jsx`	Client-side route gating by session and role	`App.jsx` wraps 6 routes with it
`utils/currency.js`	One `taka()` formatter	Store card, store details, cart, orders — so the ৳ symbol can never drift

The service layer — why it exists at all

Seven thin modules (`storeService`, `cartService`, `communityService`, `reviewService`, `wishlistService`, `pointsService`, `adminService`) wrap the raw endpoints. Each function does one thing: call the endpoint and unwrap `response.data`.

```
// frontend/src/services/storeService.js — the whole pattern in three lines
export async function fetchStoreBooks(params) {
  const { data } = await api.get("/store/books", { params });
  return data;
}
```

The value is that **a component never types a URL string**. If `/store/books` were renamed, exactly one file changes. It also means components deal in domain vocabulary (`fetchStoreBooks`) rather than transport vocabulary (`axios.get`).

> **WHERE THE PATTERN IS NOT FOLLOWED**
>
> `Home.jsx`, `Collection.jsx`, `BookDetails.jsx`, `MyLibrary.jsx` and `AdminCollectionTab.jsx` import `api` directly for the `/books` and `/borrowed` endpoints. Same historical reason as the inline routers: those pages predate the service layer. There is no `bookService.js` in the project.

2.4 Generated and non-source folders

PATH	WHAT IT IS	WHY IT IS NOT PART OF THE ARCHITECTURE
`node_modules/`	Installed dependencies	Reproducible from `package.json` + the lockfile. Thousands of third-party files; nothing here was written for Bookverse.
`package-lock.json`	Exact resolved versions	Machine-generated. Committed so every install is identical, but never hand-edited.
`.git/`	Version history	Tooling metadata. Valuable as evidence (§28 is built from it) but not runtime code.
`backend/.vercel/`	Deployment link	Generated by the Vercel CLI; ties the folder to a hosted project.
`dist/` (when built)	Vite build output	Compiled artefact of `src/`. Regenerated by `npm run build`.
`backend/server.log`	Captured dev output	A runtime by-product, not source.

2.5 Environment variables — the security boundary in the file layout

The split between the two `.env.example` files *is* a security design, and it is a very common interview question.

backend/.env.example — secret

```
PORT=5050
MONGO_URI=...
CLIENT_URL=http://localhost:5173
FIREBASE_PROJECT_ID=...
FIREBASE_CLIENT_EMAIL=...
FIREBASE_PRIVATE_KEY="-----BEGIN..."
ADMIN_SECRET=...
STRIPE_SECRET_KEY=sk_test_...
```

Never leaves the server. The private key and the Stripe secret key grant total control over the Firebase project and the Stripe account respectively.

frontend/.env.example — public

```
VITE_API_URL=http://localhost:5050/api
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
```

Vite inlines every `VITE_*` value into the bundle, so all of these are readable by anyone. The repo's own comment says it plainly: "*These are public by design.*"

INTERVIEW ANSWER: "ISN'T EXPOSING THE FIREBASE API KEY A VULNERABILITY?"

No. A Firebase web API key is an **identifier**, not a credential — it tells Google which project a request belongs to. It grants no data access on its own. Access is decided by Firebase Authentication and by `firestore.rules`. The genuinely dangerous credential is `FIREBASE_PRIVATE_KEY`, which is why it lives only in the backend environment and is deliberately *not* prefixed `VITE_`.

SECTION THREE

Frontend Architecture

How the React application is assembled, why each pattern was chosen, and the exact path a piece of data travels from a click to the screen.

3.1 The entry point and provider nesting

`frontend/src/main.jsx`

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <WishlistProvider>
          <CartProvider>
            <App />
          </CartProvider>
        </WishlistProvider>
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
);
```

Read that nesting outside-in — the order is not decorative:

- `StrictMode` — development-only double-invocation of effects to surface impure code. It is why several effects in this project are written to be safely re-runnable.
- `BrowserRouter` — must be outermost of the app providers because `ProtectedRoute` and several components call `useNavigate()` and `useLocation()`.
- `AuthProvider` — must wrap the next two, because `CartProvider` and `WishlistProvider` both call `useAuth()` to know whose cart/wishlist to load. Invert this order and the app crashes with "*useAuth must be used within AuthProvider*".
- `WishlistProvider` / `CartProvider` — independent of each other; their relative order is arbitrary.

> **INTERVIEW-READY**
>
> "Provider nesting order encodes the dependency graph between contexts. Any context that consumes another must be nested inside it."

3.2 Routing

`frontend/src/App.jsx`

```
function App() {
  const location = useLocation();
  return (
    <div className="flex min-h-screen flex-col bg-cream-50">
      <Navbar />
      <main className="flex-1">
        <AnimatePresence mode="wait">
          <Routes location={location} key={location.pathname}>
            <Route path="/" element={<Home />} />
            ...
          </Routes>
        </AnimatePresence>
      </main>
      <Footer />
    </div>
```

```
  );
}
```

`Navbar` and `Footer` sit *outside* `<Routes>`, so they persist across navigation and never remount — which is what keeps the notification listener alive and the cart badge stable while you move around the site.

The `key={location.pathname}` on `<Routes>` is the trick that makes exit animations work. Without a changing key, React would reconcile the old page into the new one and `AnimatePresence` would never observe an unmount. With it, each path is a distinct element, so the outgoing page can animate out (`mode="wait"` means it fully finishes before the next animates in).

The complete route table

PATH	COMPONENT	GUARD	PURPOSE
`/`	Home	PUBLIC	Landing page; loads 8 top-rated books
`/collection`	Collection	PUBLIC	Official catalogue with search/filter/sort
`/books/:id`	BookDetails	PUBLIC	Official book; borrow / buy / wishlist / reviews
`/community`	Community	PUBLIC	Community shelf browse
`/community/:id`	CommunityBookDetails	PUBLIC	Used book; request to borrow
`/store`	Store	PUBLIC	Shop with sidebar filters
`/store/:id`	StoreBookDetails	PUBLIC	Product page; add to cart
`/genres`, `/about`	Genres, About	PUBLIC	Static marketing pages
`/login`, `/register`	Login, Register	PUBLIC	Auth forms; redirect away if already signed in
`/my-library`	MyLibrary	SESSION	4 tabs: borrowed, contributions, requests, purchased
`/account`	AccountInfo	SESSION	Profile summary
`/account/reviews`	MyReviews	SESSION	Every review this user wrote
`/wishlist`	Wishlist	SESSION	Saved books from both shelves
`/cart`	Cart	SESSION	Cart **and** the post-payment receipt
`/orders`	Orders	SESSION	Stripe order history
`/admin`	Admin	ROLE: ADMIN	3-tab dashboard
`/admin/setup`	AdminSetup	SESSION*	One-time promotion; guards itself internally
`*`	NotFound	PUBLIC	404

* `/admin/setup` is deliberately **not** wrapped in `ProtectedRoute roles={["admin"]}` — a non-admin must be able to reach it in order to become one. It performs its own session check and renders `<Navigate to="/login">` if signed out.

3.3 ProtectedRoute — and its two subtle behaviours

frontend/src/components/ProtectedRoute.jsx

```
const ProtectedRoute = ({ children, roles }) => {
  const { firebaseUser, profile, loading } = useAuth();
  const location = useLocation();

  if (loading) return <Spinner />;                    ① never judge before auth resolves

  if (!firebaseUser) {
    const from = `${location.pathname}${location.search}`; ② keep the query string
```

```
    return <Navigate to="/login" state={{ from }} replace />;
  }

  if (roles && !profile) return <Spinner />;              ③ role needs the Mongo profile

  if (roles && !roles.includes(profile.role)) return <Navigate to="/" replace />;

  return children;
};
```

1. **The `loading` gate.** On a hard refresh, Firebase restores the session asynchronously. For the first few hundred milliseconds `firebaseUser` is `null` even for a signed-in user. Without this gate, refreshing `/my-library` would bounce a logged-in user to `/login` every single time.
2. **Preserving `location.search`.** This line is the fix for a real payment bug. Stripe returns the shopper to `/cart?session_id=cs_test_...`. If the guard captured only the pathname, a user whose session had not yet restored would lose the session id and their paid order would never be recorded. Covered fully in §28.5.
3. **Waiting for `profile` before judging role.** `firebaseUser` arrives from Firebase; `profile` (which carries `role`) arrives from a separate round-trip to `/auth/sync`. There is a window where the user exists but the profile does not — checking `profile.role` then would throw, and defaulting to "not admin" would eject a genuine admin from their own dashboard.

SAY THIS BEFORE YOU ARE ASKED

`ProtectedRoute` is **user experience, not security**. Anyone can edit React state in devtools and render `<Admin />`. It would be an empty shell, because every request it fires is independently checked by `verifyFirebaseToken` + `requireRole("admin")` on the server. See §18.6.

3.4 State management — three tiers, no Redux

TIER	MECHANISM	WHAT LIVES THERE
Local	`useState` in one component	Form fields, modal open/closed, `submitting`, `busyId`, hovered star, active tab
Shared	React Context	Auth session + profile + points; cart contents; wishlist items
URL	`useSearchParams`	Search text, author filter, category, sort, rating filter

Why the URL is used as state — the pattern in three listing pages

`frontend/src/pages/Collection.jsx`

```
const [searchParams, setSearchParams] = useSearchParams();

const search   = searchParams.get("search")   || "";
const author   = searchParams.get("author")   || "";
const category = searchParams.get("category") || "All";
const sort     = searchParams.get("sort")     || "newest";

const updateParam = (key, value) => {
  const next = new URLSearchParams(searchParams);
  if (value && value !== "All") next.set(key, value);
  else next.delete(key);
  setSearchParams(next);
};

useEffect(() => {
  const fetchBooks = async () => {
    setLoading(true);
    try {
      const { data } = await api.get("/books", { params: { search, author, category, sort } });
      setBooks(data);
```

```
    } catch (err) { console.error("Failed to load books", err); }
    finally { setLoading(false); }
  };
  fetchBooks();
}, [search, author, category, sort]);
```

There is **no** `useState` for the filters at all. The consequences:

- A filtered view is a shareable, bookmarkable URL: `/collection?category=Fantasy&sort=rating`.
- The browser back button undoes a filter change, because each change is a history entry.
- Refreshing preserves the view.
- There is exactly one source of truth, so the inputs and the results can never disagree.
- The effect's dependency array is derived from the URL, so **changing the URL is what triggers the fetch**. Clicking a category button does not fetch — it rewrites the URL, React Router re-renders, the derived values change, and the effect fires.

Note `updateParam` deletes the key when the value is empty or `"All"`, which keeps URLs clean (`/collection`, not `/collection?category=All&search=`).

> **TRADE-OFF TO ACKNOWLEDGE**
>
> Typing in the search box fires a request per keystroke — there is **no debounce** in the code. On a small catalogue it is imperceptible. The honest interview answer is: "I'd add a 300 ms debounce with `useEffect` + `setTimeout` cleanup before this saw real traffic."

3.5 The three Contexts

AuthContext — the hub

Exposes: `firebaseUser`, `profile`, `points`, `loading`, `redirectError`, `signup`, `login`, `loginWithGoogle`, `logout`, `refreshProfile`, `refreshPoints`.

The distinction between `firebaseUser` and `profile` is the single most important idea in the frontend:

	`FIREBASEUSER`	`PROFILE`
Comes from	Firebase Auth (`onAuthStateChanged`)	MongoDB, via `POST /api/auth/sync`
Contains	`uid`, `email`, `displayName`	`_id`, `firebaseUid`, `name`, `email`, `role`, `createdAt`
Answers	"Is someone signed in?"	"What is this person allowed to do?"
Used by	ProtectedRoute session check, Cart/Wishlist load triggers, "log in to…" prompts	Role checks, display name, ownership comparisons

CartContext — server-owned state mirrored locally

`frontend/src/context/CartContext.jsx`

```
const load = useCallback(async () => {
  if (!firebaseUser) { setCart(EMPTY); return; }
  setLoading(true);
  try { setCart(await cartApi.fetchCart()); }
  catch (err) { console.error("Failed to load cart", err); }
  finally { setLoading(false); }
}, [firebaseUser]);

useEffect(() => { load(); }, [load]);

const add         = async (bookId, quantity = 1) => setCart(await cartApi.addToCart(bookId, quantity));
const setQuantity = async (bookId, quantity)     => setCart(await cartApi.updateCartItem(bookId, quantity));
```

```
const remove      = async (bookId)              => setCart(await cartApi.removeFromCart(bookId));
const clear       = async ()                    => setCart(await cartApi.clearCart());
```

Every mutation is one line, because **every cart endpoint returns the entire rebuilt cart**. The client never recomputes a subtotal, never patches an item in place, never guesses. It replaces its state with whatever the server says. That is why the displayed total can never disagree with what Stripe will charge (§15.5).

`useCallback` with `[firebaseUser]` is load-bearing: `load` is in the effect's dependency array, so without memoisation a new function identity every render would make the effect fire on every render — an infinite request loop. Memoising means it re-runs only when the user actually changes (sign-in, sign-out, account switch).

WishlistContext — optimistic updates

frontend/src/context/WishlistContext.jsx

```
const toggle = async (book) => {
  const key = (i) => i.source === book.source && i.bookId === book.bookId;
  const already = items.some(key);
  if (already) {
    setItems((prev) => prev.filter((i) => !key(i)));      remove locally FIRST
    try { await apiRemove(book.source, book.bookId); }
    catch (err) { console.error(...); load(); }            failed → resync from server
  } else {
    const optimistic = { ...book, id: `temp-${Date.now()}` };
    setItems((prev) => [optimistic, ...prev]);             add locally FIRST
    try {
      const saved = await apiAdd(book);
      setItems((prev) => prev.map((i) => (i.id === optimistic.id ? saved : i)));
    } catch (err) { console.error(...); load(); }
  }
};
```

The heart fills **instantly** instead of after a network round-trip. The temporary id (`temp-1724...`) is a placeholder so React has a stable key; it is swapped for the real Firestore document id when the server responds. Any failure calls `load()`, which discards local guesses and re-reads the truth.

> **WHY CART IS PESSIMISTIC BUT WISHLIST IS OPTIMISTIC**
>
> A wrong wishlist heart for 200 ms is harmless. A wrong **price or quantity** is not — it could mean charging the wrong amount. Optimism is applied where the cost of being briefly wrong is zero.

3.6 The Axios layer

frontend/src/services/api.js

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5050/api",
});

api.interceptors.request.use(async (config) => {
  if (auth) await auth.authStateReady();
  const user = auth?.currentUser;
  if (user) {
    const token = await user.getIdToken();
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Twelve lines that carry an enormous amount of weight. Because this interceptor exists, **not one component in the entire application ever handles a token**. Line-by-line analysis is in §22.2 — it is worth memorising.

3.7 The canonical data path

1. **User acts** — clicks "Add to cart" on a Store product page.
   `pages/StoreBookDetails.jsx → handleAdd()`
2. **Component asks the Context** — `const { add } = useCart(); await add(id);`
   `context/CartContext.jsx`
3. **Context calls the service** — `cartApi.addToCart(bookId, quantity)`
   `services/cartService.js`
4. **Axios interceptor** waits for `authStateReady()`, fetches a fresh ID token, sets the `Authorization` header.
   `services/api.js`
5. **HTTP** — `POST {VITE_API_URL}/cart` with body `{ bookId, quantity }`
6. **Express** routes it, `verifyFirebaseToken` populates `req.user`, `addToCart` runs.
   `routes/cart.js → controllers/cartController.js`
7. **Firestore** — the cart doc is merged, then `buildCart()` re-reads every price from `storeBooks`.
8. **Response** — `201` with the complete rebuilt cart `{ items, subtotal, count }`.
9. **Context replaces state** — `setCart(response)`.
10. **React re-renders every consumer** — the navbar badge, the button ("Added to cart"), and the cart page all update from one state change.

3.8 Loading and error handling conventions

The same three-part shape appears in almost every data-fetching component:

```
const [data, setData]       = useState(initialShape);
const [loading, setLoading] = useState(true);
const [error, setError]     = useState("");

try   { setData(await fetchSomething()); }
catch (err) { setError(err.response?.data?.message || "Fallback message"); }
finally { setLoading(false); }
```

`err.response?.data?.message` is the important idiom. The Express `errorHandler` always replies with `{ message }`, so this reads the server's own human-readable explanation — "Not enough points. This book costs 200, you have 150." — rather than showing a generic failure. Optional chaining matters because a network failure produces an error with **no** `response` at all, and `err.response.data` would throw.

Loading states are **skeletons**, not spinners, on list pages — grey pulsing blocks in the exact grid shape of the real content, so the layout does not jump when data lands:

```
{Array.from({ length: 8 }).map((_, i) => (
  <div key={i} className="aspect-[3/4] animate-pulse rounded-2xl bg-navy-100/50" />
))}
```

3.9 Framer Motion — where and why

WHERE	TECHNIQUE	PURPOSE
`PageTransition.jsx`	`initial/animate/exit` fade + slide	Wraps every page body; paired with `AnimatePresence` in App.jsx
`BookCard.jsx`	`whileInView` + `viewport={{ once: true }}`	Cards animate in as they scroll into view, once only
Card grids	`delay: (index % 8) * 0.06`	Staggered cascade; the modulo caps the delay so item 200 isn't 12 s late
Modals	`AnimatePresence` + scale/slide	Exit animation runs before unmount
Cart items	`layout` prop	Remaining rows glide up when one is removed
MyLibrary tabs	`layoutId="mylib-tab"`	A single underline element that slides between tabs
Buttons	`whileTap={{ scale: 0.95 }}`	Tactile press feedback

`layoutId` is worth understanding: two elements in different positions sharing one `layoutId` are treated by Framer Motion as the *same* element moving, so it interpolates position automatically — a "magic move" with no manual animation code.

SECTION FOUR

Backend Architecture

The Express application, the vocabulary of its layers, and how a request is transformed into a response — using Bookverse's own files.

4.1 The application file

backend/app.js

```
require("dotenv").config();          ① env vars first — everything below reads them
const express = require("express");
const cors    = require("cors");
const connectDB = require("./config/db");
const { errorHandler } = require("./middleware/errorHandler");
... 13 route requires ...

const app = express();

connectDB();                         ② open the Mongo connection (not awaited)

app.use(cors());                     ③ allow the separately-hosted frontend
app.use(express.json());             ④ parse JSON bodies into req.body

app.use("/api/auth",     authRoutes); ⑤ mount each router on its prefix
app.use("/api/books",    bookRoutes);
app.use("/api/borrowed", borrowRoutes);
... 10 more ...

app.get("/", (req, res) => res.send("Bookverse API is running"));

app.use(errorHandler);               ⑥ LAST — 4 args makes it the error handler

module.exports = app;
```

Order is the whole design here. Express runs middleware top to bottom:

- `dotenv` must precede every `require` that reads `process.env` at module load — `lib/firebaseAdmin.js` and `lib/stripe.js` both do exactly that.
- `express.json()` must precede the routers, or every `req.body` is `undefined`.
- `errorHandler` must come last. Express identifies error middleware purely by its **arity**: a function taking `(err, req, res, next)` — four parameters — is an error handler. Registered earlier, routes below it would never be reached.

`connectDB()` is intentionally not awaited. Mongoose buffers commands issued before the connection is ready and flushes them once it opens, so the server can start accepting requests immediately.

4.2 The layers, defined precisely

These definitions come up in nearly every interview. Each is illustrated with real Bookverse code.

Route

Maps an HTTP **method + URL pattern** to a handler chain, and declares which guards apply. It contains no business logic.

backend/routes/store.js

```
router.get("/categories",   listCategories);                                  public
router.get("/books",        listStoreBooks);                                  public
router.get("/books/:id",    getStoreBook);                                    public
```

```
router.post("/books",       verifyFirebaseToken, requireRole("admin"), createStoreBook);
router.put("/books/:id",    verifyFirebaseToken, requireRole("admin"), updateStoreBook);
router.delete("/books/:id", verifyFirebaseToken, requireRole("admin"), deleteStoreBook);
```

You can read the entire security posture of the Store off these six lines — reads are open, writes require an admin. That legibility *is* the reason routes and controllers are separate.

Controller

The function that fulfils the request: validate input, enforce rules, touch the database, shape the response. It does not know its own URL.

backend/controllers/wishlistController.js

```
async function addToWishlist(req, res, next) {
  try {
    const { source, bookId, title, author, coverImage } = req.body;
    if (!SOURCES.includes(source) || !bookId || !title) {
      return res.status(400).json({ message: "source, bookId and title are required" });
    }
    const existing = await db.collection(COLLECTIONS.wishlist)
      .where("userUid", "==", req.user.firebaseUid)
      .where("source", "==", source)
      .where("bookId", "==", bookId)
      .get();
    if (!existing.empty) {
      const doc = existing.docs[0];
      return res.json({ id: doc.id, ...doc.data() });    idempotent
    }
    const item = { userUid: req.user.firebaseUid, source, bookId, title,
                   author: author || "", coverImage: coverImage || "",
                   createdAt: FieldValue.serverTimestamp() };
    const ref = await db.collection(COLLECTIONS.wishlist).add(item);
    res.status(201).json({ id: ref.id, ...item });
  } catch (err) { next(err); }
}
```

Notice `req.user.firebaseUid` — the owner is taken from the **verified token**, never from the request body. A client cannot add an item to someone else's wishlist because it has no way to express whose wishlist it is.

Middleware

A function `(req, res, next)` that runs *before* the handler and either enriches the request, ends it, or passes control on.

backend/middleware/auth.js

```
async function verifyFirebaseToken(req, res, next) {
  try {
    const authHeader = req.headers.authorization || "";
    const token = authHeader.startsWith("Bearer ") ? authHeader.slice(7) : null;
    if (!token) return res.status(401).json({ message: "Missing authorization token" });

    const decoded = await admin.auth().verifyIdToken(token);
    req.firebaseUser = decoded;

    const user = await User.findOne({ firebaseUid: decoded.uid });
    if (!user) return res.status(401).json({ message: "User not found. Please complete signup." });

    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}
```

```
function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Forbidden: insufficient role" });
    }
    next();
  };
}
```

`requireRole` is a **higher-order function** — it takes roles and *returns* the middleware. That closure is what lets you write `requireRole("admin")` at the route and have the roles available later when the request arrives. It is the clearest example of a closure in the codebase (§21.2).

Model

A Mongoose schema plus the model compiled from it: the shape, types, defaults and validation of a MongoDB document.

backend/models/User.js

```
const userSchema = new mongoose.Schema({
  firebaseUid: { type: String, required: true, unique: true, index: true },
  name:        { type: String, required: true, trim: true },
  email:       { type: String, required: true, lowercase: true, trim: true },
  role:        { type: String, enum: ["member", "admin"], default: "member" },
  createdAt:   { type: Date, default: Date.now },
});
module.exports = mongoose.model("User", userSchema);
```

Every field is doing a job: `unique + index` makes the token→user lookup fast and stops duplicate accounts; `enum` means an invalid role cannot be written even by a bug; `default: "member"` means new users are never accidentally privileged; `lowercase` normalises emails so casing never splits an identity. There is **no password field** — Firebase owns credentials entirely.

Library / helper

Shared infrastructure created once and reused, living in `lib/`.

backend/lib/stripe.js

```
const CURRENCY = "bdt";
const toStripeAmount = (taka) => Math.round(Number(taka) * 100);

const key = process.env.STRIPE_SECRET_KEY;
const stripe = key ? new Stripe(key) : null;

if (!stripe) {
  console.warn("STRIPE_SECRET_KEY is missing — the Store will load but checkout is disabled until it is set.");
}
```

The `key ? … : null` guard is a deliberate design choice: a missing Stripe key **degrades** the app (browse yes, checkout no) instead of crashing it at boot. The controller then returns a clear `503` rather than a stack trace.

Request and Response

OBJECT / PROPERTY	WHAT IT HOLDS	REAL BOOKVERSE USE
`req.params`	Named URL segments	`req.params.bookId` in `POST /points/purchase/:bookId`
`req.query`	Query string, parsed	`const { search, author, category, sort } = req.query`
`req.body`	Parsed JSON body	`req.body.shippingName`, `req.body.quantity`
`req.headers`	HTTP headers	`req.headers.authorization`, `req.headers["x-admin-secret"]`
`req.user`	**Custom** — attached by middleware	The Mongo user doc; the basis of every ownership check
`req.firebaseUser`	**Custom** — the decoded token	Set alongside `req.user`; available if raw claims are needed
`res.json(x)`	Sends JSON, status 200	The default success reply everywhere
`res.status(n).json(x)`	Explicit status + JSON	`400` validation, `401` auth, `403` role/ownership, `404` missing, `503` Stripe unconfigured
`next(err)`	Hands off to error middleware	Every controller's `catch` block

4.3 Status codes as used in this project

CODE	MEANING	CONCRETE EXAMPLE FROM THE CODE
200	OK	Any successful `GET`; also a duplicate wishlist add (idempotent)
201	Created	Borrow record, used book, review, cart add, confirmed order
400	Bad request	"Rating must be between 1 and 5"; "Your cart is empty"; "You have already reviewed this book"; insufficient points
401	Unauthenticated	Missing or invalid token; user not found in Mongo
403	Forbidden	Not an admin; not your notification; not your book; not your checkout session
404	Not found	Unknown book id; a Collection book already sold
500	Server error	The `errorHandler` default
503	Unavailable	Checkout attempted with no `STRIPE_SECRET_KEY`

401 VS 403 — GET THIS RIGHT

401 = "I don't know who you are" (authentication failed). **403** = "I know exactly who you are, and you still may not do this" (authorisation failed). Bookverse uses both correctly: a bad token is 401; a valid member trying to delete a Store book is 403.

4.4 Error handling

`backend/middleware/errorHandler.js`

```
function errorHandler(err, req, res, next) {
  console.error(err);
  const status = err.status || 500;
  res.status(status).json({ message: err.message || "Internal server error" });
}
```

Newer controllers use `catch (err) { next(err); }`, which funnels everything here — one place that logs, one consistent `{ message }` response shape. The frontend relies on that shape via `err.response?.data?.message`.

A SECOND HONEST INCONSISTENCY

> The older inline routers (`books.js` , `borrow.js` , `stats.js`) do **not** use `next(err)` . They catch locally and reply `res.status(500).json({ message, error: err.message })` — note the extra `error` field, which leaks raw internals. Being able to say "the newer half funnels through `next(err)` , the older half predates it and should be migrated" demonstrates genuine familiarity with your own code.

4.5 Complete endpoint reference

M	ENDPOINT	GUARD	STORE	PURPOSE
PST	`/api/auth/sync`	token (inline)	MONGO	Create-or-update the Mongo user from a verified token
GET	`/api/auth/me`	token	MONGO	Return the current profile (incl. role)
PST	`/api/auth/bootstrap-admin`	token + secret	MONGO	One-time self-promotion to admin
GET	`/api/books`	public	MONGO	Search / filter / sort the Collection
GET	`/api/books/:id`	public	MONGO	One book
PST	`/api/books`	admin	MONGO	Create a book
PUT	`/api/books/:id`	admin	MONGO	Update a book
DEL	`/api/books/:id`	admin	MONGO	Delete a book
PST	`/api/books/:id/borrow`	token	MONGO	Borrow, with shipping details
PST	`/api/books/:id/return`	token	MONGO	Return your own loan
GET	`/api/borrowed/me`	token	MONGO	Your loans, with snapshot fallback
GET	`/api/stats`	public	both	Collection counts + sold total from Firestore
GET	`/api/community/books`	public	FIRE	Browse the community shelf
GET	`/api/community/books/mine`	token	FIRE	Your contributions
GET	`/api/community/books/:id`	public	FIRE	One used book
PST	`/api/community/books`	token	FIRE	Share a book, award 50 points
DEL	`/api/community/books/:id`	token + owner	FIRE	Withdraw your own book
PST	`/api/community/books/:id/request`	token	FIRE	Ask to borrow; notifies owner
GET	`/api/community/requests`	token	FIRE	Incoming + outgoing requests
PST	`/api/community/requests/:id/respond`	token + owner	FIRE	Approve or decline
PST	`/api/community/requests/:id/return`	token + borrower	FIRE	Return a borrowed used book
GET	`/api/points/me`	token	FIRE	Balance + both economy rates
GET	`/api/points/purchases`	token	FIRE	Books you own permanently
PST	`/api/points/purchase/:bookId`	token	both	Buy an official book for 200 points
GET	`/api/reviews/mine`	token	both	Your reviews, enriched across both shelves
GET	`/api/reviews/:source/:bookId`	public	FIRE	Reviews + average + count
PST	`/api/reviews/:source/:bookId`	token	both	Post one review (duplicate-checked)
PST	`/api/notifications/:id/read`	token + owner	FIRE	Mark one read
PST	`/api/notifications/read-all`	token	FIRE	Batch mark all read
GET	`/api/wishlist`	token	FIRE	Your saved books
PST	`/api/wishlist`	token	FIRE	Save a book (idempotent)
DEL	`/api/wishlist/:source/:bookId`	token	FIRE	Unsave by book, not by doc id
GET	`/api/store/categories`	public	FIRE	Category names + counts
GET	`/api/store/books`	public	FIRE	Catalogue with filters + sort

M	ENDPOINT	GUARD	STORE	PURPOSE
GET	`/api/store/books/:id`	public	FIRE	One product
PST	`/api/store/books`	admin	FIRE	Add a product
PUT	`/api/store/books/:id`	admin	FIRE	Edit a product
DEL	`/api/store/books/:id`	admin	FIRE	Remove a product
GET	`/api/cart`	token	FIRE	Rebuilt cart with live prices
PST	`/api/cart`	token	FIRE	Add / increase quantity
PCH	`/api/cart/:bookId`	token	FIRE	Set exact quantity (0 removes)
DEL	`/api/cart/:bookId`	token	FIRE	Remove one line
DEL	`/api/cart`	token	FIRE	Empty the cart
PST	`/api/checkout/session`	token	FIRE	Create a Stripe Checkout Session
GET	`/api/checkout/confirm/:sessionId`	token + owner	FIRE	Verify payment, record order once, empty cart
GET	`/api/checkout/orders`	token	FIRE	Order history
GET	`/api/admin/members`	admin	MONGO	All members + their loans

46 endpoints. Two guard styles appear: per-route middleware (`router.get("/x", verifyFirebaseToken, handler)`) and router-wide (`router.use(verifyFirebaseToken)` at the top of `cart.js` , `checkout.js` and `admin.js`). Router-wide is used where *every* route needs the same guard — it makes it impossible to forget one when adding a route later.

4.6 Route ordering — a real bug class, avoided twice

backend/routes/community.js

```
// "mine" is declared before "/books/:id" so it isn't swallowed as an id.
router.get("/books/mine", verifyFirebaseToken, listMyUsedBooks);
router.get("/books",      listUsedBooks);
router.get("/books/:id",  getUsedBook);
```

backend/routes/reviews.js

```
// Declared before the "/:source/:bookId" param route so "mine" isn't captured as a source.
router.get("/mine", verifyFirebaseToken, listMyReviews);
router.get("/:source/:bookId", listReviews);
```

Express matches routes **in registration order** and stops at the first match. If `/books/:id` were registered first, a request to `/books/mine` would match it with `req.params.id === "mine"` — and the handler would look up a Firestore document literally named "mine", return 404, and the user's contributions page would be permanently empty. Both files carry a comment explaining the ordering, because the code alone looks arbitrary.

> **RULE**
>
> **Static path segments must always be registered before dynamic ones at the same depth.**

SECTION FIVE

Security Architecture

Every protective mechanism actually present in the codebase — what it protects, why it is necessary, where it lives, and how a request travels through it.

THE GOVERNING PRINCIPLE

The client is never trusted with a decision, an identity, or a price. Identity comes from a cryptographically verified token; authorisation comes from a database role; prices come from the server's own catalogue read. Everything in this section is an application of that one rule.

5.1 The complete security stack

#	MECHANISM	PROTECTS AGAINST	IMPLEMENTED IN
1	Firebase Authentication	Credential theft, weak password storage, OAuth implementation bugs	Google's infrastructure; `frontend/src/firebase.js`
2	ID token in an `Authorization` header	Anonymous access to private endpoints	`services/api.js` interceptor
3	`verifyIdToken()`	Forged, tampered, or expired tokens	`middleware/auth.js`
4	Mongo user lookup by `firebaseUid`	A valid Firebase token with no Bookverse account	`middleware/auth.js`
5	`requireRole("admin")`	Privilege escalation by ordinary members	`middleware/auth.js`
6	`ADMIN_SECRET` bootstrap	Anyone self-promoting to admin	`controllers/authController.js`
7	Ownership checks	Acting on another user's book, request, or notification	community / notifications / checkout controllers
8	Identity from token, never from body	Impersonation via a forged `userId` field	Every controller (`req.user.firebaseUid`)
9	`firestore.rules` — all writes denied	Browser-console manipulation of points, requests, prices	`firestore.rules`
10	Firestore per-document read scoping	Reading other people's addresses, orders, points	`firestore.rules`
11	Server-side price computation	Paying ৳1 for a ৳1890 book	`cartController.buildCart()`
12	Stripe session ownership check	Claiming somebody else's paid order	`checkoutController.confirmCheckout`
13	`payment_status` verification	Recording an order that was never paid	`checkoutController.confirmCheckout`
14	Order idempotency by session id	Duplicate orders from a page refresh	`checkoutController.confirmCheckout`
15	Firestore transaction on points	Double-spending points via concurrent requests	`pointsController.purchaseOfficialBook`
16	Atomic conditional claim	Two buyers acquiring the same single copy	`pointsController.purchaseOfficialBook`
17	One-review-per-user check	Rating manipulation by repeat reviews	`reviewController.addReview`
18	Input validation	Malformed / out-of-range data reaching the database	All controllers
19	Secret/public env separation	Leaking the Admin private key to the browser	The two `.env` files
20	CORS	(Permissive here — see 5.9)	`app.js`

5.2 Authentication: how identity reaches Express

1. **Sign-in happens outside Bookverse.** The Firebase Web SDK talks directly to Google. Bookverse's server never sees the password.
   `context/AuthContext.jsx → signInWithEmailAndPassword / signInWithPopup`
2. **Firebase issues an ID token** — a JWT signed by Google's private key, carrying `uid`, `email`, `iss`, `aud`, `exp`. Valid for one hour.
3. **The interceptor attaches it** to every outgoing request: `Authorization: Bearer <token>`. `getIdToken()` silently refreshes it when near expiry.
   `services/api.js`
4. **Express extracts it**, rejecting anything not prefixed `Bearer` with a 401.
   `middleware/auth.js`
5. `verifyIdToken()` checks the signature against Google's rotating public keys, plus expiry, issuer and audience. Any tampering invalidates the signature.
6. **Mongo lookup** — `User.findOne({ firebaseUid: decoded.uid })` converts a Google identity into a Bookverse account carrying a `role`.
7. `req.user` **is set** and `next()` runs. From here, every controller can treat the caller's identity as proven.

WHY STEP 6 EXISTS AT ALL

Firebase knows *who* you are but has no concept of a Bookverse role. Bookverse stores `role` in MongoDB. This lookup is the join between the two systems — and it is also a second gate: a valid Google token for someone who never completed signup is rejected with `401 "User not found. Please complete signup."`

5.3 Authorisation: three distinct layers

Layer 1 — Role-based (RBAC)

```
router.use(verifyFirebaseToken, requireRole("admin"));   routes/admin.js — whole router
router.post("/books", verifyFirebaseToken, requireRole("admin"), createStoreBook);
```

Nine endpoints are admin-gated: 3 official-book writes, 3 Store writes, and `/api/admin/members`. Role comes from MongoDB, never from the token or the request.

Layer 2 — Ownership

Role is not enough when the resource belongs to an individual member. Four separate ownership checks exist:

```
// communityController.deleteUsedBook — only the contributor may withdraw a book
if (doc.data().ownerUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "You can only remove your own books" });

// communityController.respondToRequest — only the owner may approve/decline
if (request.ownerUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "Only the book's owner can answer this request" });

// communityController.returnUsedBook — only the borrower may return
if (request.requesterUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "This is not your borrow record" });

// routes/notifications.js — only the recipient may mark one read
if (doc.data().userUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "Not your notification" });
```

Every one follows the same shape: **fetch the document, compare its stored owner UID against the verified caller, 403 on mismatch.** The comparison is only trustworthy because `req.user.firebaseUid` originates from a verified token.

Layer 3 — Implicit ownership through query scoping

```
// You cannot ask for someone else's data, because the query hard-codes yours
.where("userUid", "==", req.user.firebaseUid)
```

Used by wishlist, purchases, orders, reviews-mine, requests and the cart (keyed by UID as the document id). There is no endpoint anywhere that accepts a user id as a parameter — the API simply has no vocabulary for "someone else's data".

5.4 The admin bootstrap — a deliberately unusual design

`backend/controllers/authController.js`

```
// One-time-per-account self-promotion, gated by a server-only shared secret.
// This is the ONLY admin endpoint that trusts a secret instead of a role —
// every subsequent admin action is attributable to this specific signed-in
// account (audit trail), not "whoever has the secret."
async function bootstrapAdmin(req, res, next) {
  try {
    const providedSecret = req.headers["x-admin-secret"];
    if (!process.env.ADMIN_SECRET || providedSecret !== process.env.ADMIN_SECRET) {
      return res.status(403).json({ message: "Invalid admin secret." });
    }
    if (req.user.role !== "admin") {
      req.user.role = "admin";
      await req.user.save();
    }
    res.json(req.user);
  } catch (err) { next(err); }
}
```

This solves the classic **bootstrap problem**: only an admin can create an admin, so where does the first one come from? The design has several defensible properties:

- **The route still requires a valid token** — `router.post("/bootstrap-admin", verifyFirebaseToken, bootstrapAdmin)`. The secret alone is useless; you must also be signed in as a real account.
- **The secret is converted into durable, attributable state.** After promotion the secret is irrelevant to that account — the role is what authorises everything afterwards, and it is tied to a named user.
- **The secret never leaves the server.** It is compared against `process.env.ADMIN_SECRET`; it is not in the bundle, not in Firestore, not in Mongo.
- **Missing-secret is handled.** If `ADMIN_SECRET` is unset, the condition short-circuits and the endpoint refuses everyone — it cannot accidentally fail open.
- **It is idempotent.** Calling it as an existing admin is a no-op that returns the profile.
- It travels in a **header**, not the URL, so it never lands in server logs or browser history.

WEAKNESSES TO ACKNOWLEDGE HONESTLY

There is **no rate limiting**, so the secret is brute-forceable given enough attempts; a long random value is the only defence. The comparison is a plain `!==`, not a constant-time compare, so it is theoretically timing-attackable. And the endpoint stays open forever rather than disabling itself after the first promotion. Naming these unprompted is far stronger than being caught by them.

5.5 Firestore: the write lockdown

Every one of the ten Firestore collections carries `allow write: if false;`. Not "authenticated users may write their own" — **nobody** may write from a client. All writes flow through Express using the Admin SDK, which bypasses rules because it authenticates as a service account.

If client writes were allowed

Open devtools on any signed-in session and run:

```
updateDoc(doc(db,"users",myUid), { points: 999999
});
updateDoc(doc(db,"borrowRequests",id), {
status:"approved" });
updateDoc(doc(db,"storeBooks",id), { price: 1 });
```

Infinite points, self-approved borrowing, ₹1 books.

What actually happens

Every one of those calls is rejected by Firestore before it reaches storage — `PERMISSION_DENIED`. The only path to those documents is an Express endpoint that first verifies who you are and then decides whether the operation is legitimate.

And even if a price *were* rewritten, checkout re-reads prices server-side, so the tampered value would never be charged. Two independent defences.

5.6 Payment security

Prices are never accepted from the client

backend/controllers/checkoutController.js

```
// Rebuilt from the catalogue, so every amount charged comes from our own
// data — the client never gets to say what anything costs.
const cart = await buildCart(req.user.firebaseUid);
```

The cart document in Firestore stores only `{ bookId, quantity }`. Title, author, cover and **price** are re-read from `storeBooks` on every single call. Even the request body for adding to the cart carries no price field — the API has no parameter through which a price could be supplied.

The session belongs to the person who paid

```
const session = await stripe.checkout.sessions.retrieve(req.params.sessionId, { expand: ["line_items"] });

if (session.metadata?.userUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "This checkout doesn't belong to you." });

if (session.payment_status !== "paid")
  return res.status(400).json({ message: "This payment hasn't completed." });
```

The `userUid` was written into the session's metadata at creation time, from the verified token. On return, it is compared against the verified token again. Guessing another shopper's session id therefore achieves nothing. And the payment status is read **from Stripe**, not from the URL — landing on `/cart?session_id=…` by hand proves nothing.

Idempotency

```
// Reloading the success page must not create a second order.
const existing = await db.collection(COLLECTIONS.orders)
  .where("stripeSessionId", "==", session.id).get();
if (!existing.empty) {
  const doc = existing.docs[0];
  return res.json({ id: doc.id, ...doc.data(), alreadyRecorded: true });
}
```

Three independent layers stop duplicate orders: this server-side check (authoritative), a `useRef` latch in the Cart page, and stripping `session_id` from the URL after confirmation. The server-side one is the only one that actually matters — the other two are polish.

5.7 Concurrency as a security property

Points are money in this system, so double-spending is a security bug. The purchase flow defends against it twice.

```
// ① Atomic conditional claim — only one request can win this update
const claimed = await Book.findOneAndUpdate(
  { _id: book._id, available: true },
  { $set: { available: false } },
  { new: true }
);
if (!claimed) return res.status(400).json({ message: "This book was just taken by someone else..." });

// ② Firestore transaction — the balance is re-read INSIDE the transaction
result = await db.runTransaction(async (tx) => {
  const userDoc = await tx.get(userRef);
  const balance = userDoc.exists ? userDoc.data().points || 0 : 0;
  if (balance < POINTS_TO_PURCHASE) return { ok: false, balance };
  tx.set(userRef, { points: FieldValue.increment(-POINTS_TO_PURCHASE) }, { merge: true });
  tx.set(db.collection(COLLECTIONS.purchases).doc(), { ... });
  return { ok: true, balance: balance - POINTS_TO_PURCHASE };
});
```

The `{ _id, available: true }` filter makes find-and-update a single atomic database operation. Two simultaneous buyers both run it; MongoDB serialises them; the second one's filter no longer matches, so it receives `null` and is politely refused. A naive `if (book.available) { book.available = false; save(); }` would let both through, because the check and the write would be two separate operations with a gap between them.

The Firestore transaction re-reads the balance inside the transaction and aborts/retries on conflict, so two concurrent purchases cannot both see "200 points" and both spend it.

THE COMPENSATING ACTION

Points live in Firestore and books in MongoDB, so no single transaction can span both. The code handles this with a claim → pay → release pattern: if the payment leg fails, `releaseClaim(book._id)` puts the copy back on the shelf. This is a hand-rolled saga, and it is the most architecturally interesting code in the project. Full treatment in §11.5.

5.8 Input validation inventory

WHERE	VALIDATION ACTUALLY PERFORMED
`reviewController.addReview`	`source` ∈ {official, community}; rating is a number 1–5; text trimmed; duplicate review rejected
`storeController.validateBody`	5 required fields present and non-empty; price is a positive number
`cartController.addToCart`	`bookId` required; quantity coerced with `Math.max(1, Number(q) || 1)`; book must exist in the catalogue
`cartController.updateCartItem`	quantity must be a number ≥ 0; NaN rejected
`communityController.readShipping`	name, phone and address all present after trimming
`communityController.addUsedBook`	title, author, coverImage required; category and condition defaulted
`communityController.requestBorrow`	Not your own book; book available; no existing pending request from you
`routes/books.js` borrow	Book exists; book available; shipping complete; you haven't already borrowed it
`pointsController`	Shipping complete; book exists; book not on loan; balance sufficient
`wishlistController`	`source` valid; `bookId` and `title` present
Mongoose schemas	`required`, `enum`, `min` / `max` on rating, `unique` on firebaseUid, type coercion

Note the pattern `(req.body.shippingName || "").trim()` — it guards against `undefined` before calling a string method, then treats whitespace-only input as empty. A user cannot satisfy the address requirement by typing three spaces.

5.9 Known gaps — say these before an interviewer finds them

GAP	CURRENT STATE	WHAT YOU WOULD DO
CORS is wide open	`app.use(cors())` allows every origin	`cors({ origin: process.env.CLIENT_URL, credentials: true })`. Low severity here because auth is a bearer token, not a cookie — so CSRF does not apply — but it should still be scoped.
No rate limiting	Nothing throttles any endpoint	`express-rate-limit`, tightest on `/auth/bootstrap-admin`
No Stripe webhook	Orders are recorded only when the browser returns to `/cart`	Add a `checkout.session.completed` webhook with signature verification, so a customer who closes the tab after paying still gets their order. This is the single most important production gap. (§16.9)
No image validation	Cover images are arbitrary user-supplied URLs	Validate the URL, or proxy/host images. Note `BookCover` does handle broken images gracefully via `onError`.
Regex search unescaped	`{ $regex: search, $options: "i" }` passes user input straight to the engine	Escape regex metacharacters. A crafted pattern is a potential ReDoS vector. It is **not** injection — Mongoose still treats it as a value, not code.
Error detail leakage	Older routes return `error: err.message`	Route everything through `next(err)` and log details server-side only
No composite indexes file	Firestore queries stay simple to avoid needing them	Fine at this scale; would need `firestore.indexes.json` as data grows

SECTION SIX

Complete Authentication Explanation

The full Firebase Authentication architecture, then six complete request/response walkthroughs with real code and file paths.

6.1 Client configuration

frontend/src/firebase.js

```
const firebaseConfig = {
  apiKey:            import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain:        import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId:         import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket:     import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId:             import.meta.env.VITE_FIREBASE_APP_ID,
};

const isConfigured = Boolean(firebaseConfig.apiKey && firebaseConfig.projectId);
if (!isConfigured) console.warn("Firebase config is missing — ... Auth is disabled until then.");

const app = isConfigured ? initializeApp(firebaseConfig) : null;
export const auth = app ? getAuth(app) : null;
export const firestore = app ? getFirestore(app) : null;
```

The `isConfigured` guard means a developer who clones the repo without a `.env` gets a readable console warning and a browsable site, instead of a white screen from an uncaught initialisation error. This is why `auth` is checked for null in several places (`if (auth) await auth.authStateReady()`, `auth?.currentUser`).

`authDomain` deserves attention: it points at **the Bookverse site's own domain**, not at `*.firebaseapp.com`. That, combined with the proxy rewrite in `frontend/vercel.json`, is what makes redirect-based Google sign-in work at all. Full story in §28.1.

6.2 AuthProvider — the state machine

frontend/src/context/AuthContext.jsx

```
const [firebaseUser, setFirebaseUser] = useState(null);
const [profile, setProfile]           = useState(null);
const [points, setPoints]             = useState(null);
const [loading, setLoading]           = useState(true);
const [redirectError, setRedirectError] = useState(null);

useEffect(() => {
  if (!auth) { setLoading(false); return; }

  getRedirectResult(auth).catch((err) => {
    console.error("Google redirect sign-in failed", err);
    setRedirectError(err);
  });

  const unsubscribe = onAuthStateChanged(auth, async (fbUser) => {
    setFirebaseUser(fbUser);
    if (fbUser) {
      try {
        const { data } = await api.post("/auth/sync", { name: fbUser.displayName });
        setProfile(data);
        refreshPoints();
      } catch (err) {
        console.error("Failed to sync user profile", err);
```

```
        setProfile(null);
      }
    } else {
      setProfile(null);
      setPoints(null);
    }
    setLoading(false);
  });
  return unsubscribe;
}, []);
```

Why this shape:

- `onAuthStateChanged` is the **single** place a session is recognised. It fires on initial load (restoring persistence), after sign-up, after sign-in, after a Google popup, after a redirect returns, and on sign-out. Because every path converges here, profile syncing is written once rather than in six places.
- Returning `unsubscribe` from the effect is the cleanup — without it, a remount would stack listeners and every auth change would fire N times.
- `[]` dependencies: this must run exactly once for the app's lifetime.
- `setLoading(false)` sits **after** the sync attempt and outside the `if`, so it runs on every path including failure. If it were inside the `try`, a failed sync would leave the app spinning forever.
- `getRedirectResult` is called for its **error**, not its value. `onAuthStateChanged` already picks up a successful redirect session; this call exists so a *failed* redirect surfaces in the UI instead of vanishing. It feeds `redirectError`, which Login and Register display.

6.3 Session persistence

Firebase's default is `browserLocalPersistence` — the session survives tab closes and browser restarts until it is explicitly signed out. **Bookverse never sets persistence explicitly and never stores a token itself.** There is no `localStorage.setItem("token", …)` anywhere in the codebase, which is exactly right: the SDK owns storage and refresh.

ID tokens expire after one hour. `getIdToken()` in the interceptor returns the cached token if valid and transparently exchanges the long-lived refresh token for a new one if not. A user reading for three hours never notices.

6.4 `authStateReady()` — the race this prevents

frontend/src/services/api.js

```
// On a hard page load, Firebase hasn't yet restored the persisted session
// at the moment a mounting component's first request fires — reading
// auth.currentUser synchronously here would silently send the request
// unauthenticated. authStateReady() waits for that initial restore.
if (auth) await auth.authStateReady();
```

The sequence being defended against:

1. User presses F5 on `/my-library`.
2. JS loads. Firebase begins restoring the session from IndexedDB — **asynchronously**.
3. `MyLibrary` mounts and its effect immediately fires five parallel API calls.
4. **Without the guard:** `auth.currentUser` is still `null`, no header is attached, and all five return `401`. The page renders empty despite a perfectly valid session.
5. **With the guard:** the interceptor awaits the restore, `currentUser` is populated, tokens attach, all five succeed.

INTERVIEW GOLD

This is a genuinely subtle async bug and a great thing to be able to explain. The symptom — "works when I navigate there, 401s when I refresh" — is memorable and diagnostic.

6.5 `syncUser` — create-or-update, and the name race

`backend/controllers/authController.js`

```
async function syncUser(req, res, next) {
  try {
    const authHeader = req.headers.authorization || "";
    const token = authHeader.startsWith("Bearer ") ? authHeader.slice(7) : null;
    if (!token) return res.status(401).json({ message: "Missing authorization token" });

    const decoded = await admin.auth().verifyIdToken(token);
    const { name } = req.body;

    let user = await User.findOne({ firebaseUid: decoded.uid });
    if (!user) {
      user = await User.create({
        firebaseUid: decoded.uid,
        email: decoded.email,
        name: name || decoded.name || decoded.email.split("@")[0],
      });
    } else if (name && name !== user.name) {
      // The signup form's explicit call and the auth-state-change listener's
      // call can race (the listener fires before updateProfile() finishes
      // setting displayName). Treat an explicitly provided name as authoritative
      // whenever it arrives, instead of only setting it at creation time.
      user.name = name;
      await user.save();
    }
    res.json(user);
  } catch (err) { next(err); }
}
```

Three things to notice:

1. **The UID comes from the decoded token, never from the body.** The comment in the source says it outright: otherwise "the client could claim to be any user it liked." The body supplies only a display name — cosmetic data.
2. **It verifies the token inline** rather than using `verifyFirebaseToken` middleware. It must: the middleware requires an existing Mongo user, and this is the endpoint whose job is to *create* that user. Using the middleware would be a chicken-and-egg deadlock.
3. **The name fallback chain** `name || decoded.name || decoded.email.split("@")[0]` guarantees a non-empty name (the schema requires one). Google sign-in supplies `decoded.name`; email signup supplies `name`; a user with neither becomes "uday" from "uday@example.com".

THE RACE THIS `ELSE IF` FIXES

During email signup, two calls to `/auth/sync` happen almost simultaneously: `onAuthStateChanged` fires the moment the account is created — **before** `updateProfile()` has finished setting `displayName`, so it posts `{ name: null }`; and `signup()` posts the real name explicitly. If the name were only applied at creation time, whichever call arrived first would win, and the user could end up permanently named "uday" instead of "Uday Barua". Accepting a later explicit name fixes it regardless of arrival order.

6.6 Complete authentication flows

Six end-to-end walkthroughs. Each names the real file at every hop.

Flow A — New user registers with email and password

1. User fills name / email / password / confirm and submits.
   `pages/Register.jsx → handleSubmit`
2. **Client validation:** passwords match, length ≥ 6. Fails here never reach the network.
   `Register.jsx lines 34–41`
3. `signup(name, email, password)` → `createUserWithEmailAndPassword(auth, email, password)`. The password goes **to Google**, never to Express.
   `context/AuthContext.jsx`
4. Firebase creates the account, returns a credential, and starts a session. `updateProfile(cred.user, { displayName: name })` attaches the name.
5. **Two things now race:** `onAuthStateChanged` fires and posts `/auth/sync { name: displayName }`; `signup()` also posts `/auth/sync { name }`.
6. Axios interceptor: `authStateReady()` → `getIdToken()` → `Authorization: Bearer eyJhbGci…`
   `services/api.js`
7. `POST /api/auth/sync` → `syncUser` verifies the token inline.
   `routes/auth.js → controllers/authController.js`
8. **MongoDB:** no user with that `firebaseUid` → `User.create({ firebaseUid, email, name })` with `role: "member"` by default.
9. Response `200` with the full user document.
10. `setProfile(data)`; `refreshPoints()` fetches `/points/me` → `{ points: 0, pointsPerContribution: 50, pointsToPurchase: 200 }`.
11. `navigate("/", { replace: true })`. `replace` means Back does not return to the signup form.
12. **User sees:** the home page, navbar showing their first name, a 0-point badge, and cart/wishlist now loading for their account.

Flow B — Returning user logs in with email and password

1. Submit email + password.
   `pages/Login.jsx → handleSubmit`
2. `signInWithEmailAndPassword(auth, email, password)` — Firebase verifies the credential.
3. **On failure:** throws with a code like `auth/invalid-credential`. `friendlyAuthError(err)` maps it to "Incorrect email or password."
   `utils/authError.js`
4. On success, `onAuthStateChanged` fires → `/auth/sync`. The user exists, so no document is created; the profile is returned as-is.
5. `navigate(from, { replace: true })` where `from = location.state?.from || "/"` — so a user bounced from `/wishlist` lands back on `/wishlist`, not the home page.

`friendlyAuthError` deliberately does not swallow unknown codes — it returns `"Something went wrong (auth/xyz). Please try again."` so a real failure remains diagnosable from a screenshot. That is a deliberate trade of a little polish for a lot of supportability.

Flow C — Google sign-in (popup, with redirect fallback)

`frontend/src/context/AuthContext.jsx`

```
const POPUP_ENV_FAILURES = new Set([
  "auth/popup-blocked",
  "auth/operation-not-supported-in-this-environment",
  "auth/web-storage-unsupported",
  "auth/internal-error",
]);

async function loginWithGoogle() {
  const provider = new GoogleAuthProvider();
  try {
    const cred = await signInWithPopup(auth, provider);
    return cred.user;
  } catch (err) {
    if (!POPUP_ENV_FAILURES.has(err?.code)) throw err;
    await signInWithRedirect(auth, provider);
    return null; page navigates to Google; nothing to return here
  }
}
```

1 User clicks "Continue with Google".
`pages/Login.jsx → handleGoogle`

2 `signInWithPopup` opens Google's account chooser in a popup window.

3a **Popup succeeds:** returns a credential. `handleGoogle` sees a non-null user and navigates to `from`.

3b **Popup blocked (environmental):** the error code is in `POPUP_ENV_FAILURES`, so the code falls back to `signInWithRedirect` and the whole page navigates to Google. `loginWithGoogle` returns `null`, so `handleGoogle` skips navigation — there is nothing to navigate, the browser is already leaving.

3c **User cancelled** (`auth/popup-closed-by-user`): *not* in the set, so it rethrows and is shown as "Google sign-in was cancelled." A cancellation must never trigger a redirect — that would be an infinite loop of unwanted navigations.

4 After a redirect, the browser returns to the site. `onAuthStateChanged` fires with the restored session; `getRedirectResult` settles and would surface any error.

5 An effect in Login/Register watches restored state:
`useEffect(() => { if (!authLoading && firebaseUser) navigate(from, { replace: true }); }, [...])`
This is what routes the user away after the return leg — the click handler is long gone.

6 `/auth/sync` creates the Mongo user on first Google sign-in, taking the name from `decoded.name`.

WHY THE REDIRECT LEG WORKS HERE BUT IS BROKEN IN MOST PROJECTS

`signInWithRedirect` completes its handshake through an iframe on the Firebase auth domain. Served cross-origin, that storage is third-party — which Chrome 115+, Firefox 109+ and Safari 16.1+ block, silently dropping the session. Bookverse fixes this by reverse-proxying `/__/auth/*` to the Firebase auth domain in `frontend/vercel.json` and pointing `VITE_FIREBASE_AUTH_DOMAIN` at its own site, making the handler **first-party**. This took two commits to get right and is the project's headline bug — see §28.1.

Flow D — Page refresh while already logged in

1. F5 on `/my-library`. All React state is destroyed.
2. App boots. `AuthProvider` mounts with `loading: true`, `firebaseUser: null`.
3. `ProtectedRoute` sees `loading === true` and renders a spinner. **It does not redirect** — this is the gate that prevents refresh-logout.
4. Firebase restores the session from IndexedDB and fires `onAuthStateChanged(fbUser)`.
5. `/auth/sync` returns the profile; `refreshPoints()` runs; `loading` flips to `false`.
6. `ProtectedRoute` re-renders, sees a user, renders `MyLibrary`.
7. `MyLibrary`'s effect fires five parallel calls via `Promise.all`. Each waits on `authStateReady()` — already resolved — and carries a token.

Flow E — Accessing a protected API endpoint

Example: `GET /api/wishlist`

1. `fetchWishlist()` → `api.get("/wishlist")`
   `services/wishlistService.js`
2. Interceptor awaits `authStateReady()`, calls `getIdToken()`, sets the header.
3. Request on the wire:
   `GET /api/wishlist HTTP/1.1`
   `Authorization: Bearer eyJhbGciOiJSUzI1NiIs…`
4. Express matches `app.use("/api/wishlist", wishlistRoutes)` → `router.get("/", verifyFirebaseToken, listWishlist)`.
5. **Middleware:** extract → `verifyIdToken` → Mongo lookup → `req.user` → `next()`. Any failure ends here with 401.
6. **Controller:** `.where("userUid", "==", req.user.firebaseUid)` — scoped to the caller by construction.
7. Docs mapped, sorted newest-first in memory, returned as JSON.
8. `WishlistContext` calls `setItems(data)`; every heart icon across the app re-renders correctly.

Flow F — Logout

1. Click Logout in the account menu.
   `components/Navbar.jsx → handleLogout`
2. `await logout()` → `signOut(auth)` clears Firebase's persisted session.
   `context/AuthContext.jsx`
3. `onAuthStateChanged` fires with `null` → `setFirebaseUser(null)`, `setProfile(null)`, `setPoints(null)`.
4. Both other contexts react: their `load` callbacks are keyed on `firebaseUser`, so cart resets to `EMPTY` and wishlist to `[]`. **No stale data from the previous account survives.**
5. `NotificationBell`'s effect cleanup runs `unsubscribe()`, closing the Firestore listener.
6. `navigate("/")`. If the user was on a protected page, `ProtectedRoute` would have redirected them anyway.

WHY NO MANUAL CLEANUP IS NEEDED

Because all three contexts derive from `firebaseUser`, signing out is a single state change that cascades everywhere. There is no "clear the cart, clear the wishlist, close the listener" checklist to forget — the data flow does it.

SECTION SEVEN

Every Feature in the Application

A complete inventory derived by reading the code, not by reading a plan. Sections 8–18 then take the major ones apart.

7.1 The full feature list

#	FEATURE	PRIMARY UI	DETAILED IN
1	Email + password registration	`pages/Register.jsx`	§6 Flow A
2	Email + password login	`pages/Login.jsx`	§6 Flow B
3	Google sign-in (popup + redirect fallback)	Login / Register	§6 Flow C, §28.1
4	Session persistence across refresh	App-wide	§6 Flow D
5	Logout	`components/Navbar.jsx`	§6 Flow F
6	Browse the official Collection	`pages/Collection.jsx`	§8
7	Search by title / by author	Collection, Community, Store	§8.3, §9.2, §14.4
8	Category filtering	Collection, Community, Store	§8.3
9	Sorting (newest / oldest / A–Z / rating)	Collection	§8.4
10	Official book detail page	`pages/BookDetails.jsx`	§8.6
11	Borrow an official book	BookDetails + ShippingModal	§8.7
12	Return a borrowed book	`pages/MyLibrary.jsx`	§8.8
13	Buy an official book with points	BookDetails	§11.5
14	Browse the community shelf	`pages/Community.jsx`	§9.2
15	Contribute a used book (+50 points)	`AddUsedBookModal`	§9.3
16	Withdraw your own used book	MyLibrary → Contributions	§9.4
17	Request to borrow a community book	CommunityBookDetails	§9.5
18	Approve / decline a request	MyLibrary → Requests	§9.6
19	Return a borrowed community book	MyLibrary → Requests	§9.7
20	Live notifications (real-time)	`NotificationBell`	§10
21	Mark notifications read / read-all	NotificationBell	§10.5
22	Contribution points balance	Navbar, MyLibrary, AccountInfo	§11
23	Purchase history	MyLibrary → Purchased	§11.7
24	Write a review + star rating	`ReviewSection`	§12
25	Average rating + review count	ReviewSection	§12.4
26	My Reviews (both shelves)	`pages/MyReviews.jsx`	§12.6
27	Wishlist add / remove	`WishlistButton`	§13
28	Wishlist page (both sources)	`pages/Wishlist.jsx`	§13.4
29	Store catalogue + sidebar filters	`pages/Store.jsx`	§14
30	Store categories with counts	Store sidebar	§14.3
31	Price / rating filtering + sorting	Store sidebar	§14.4
32	Add to cart	StoreBookCard, StoreBookDetails	§15.3
33	Quantity +/−, remove, clear cart	`pages/Cart.jsx`	§15.4
34	Stripe Checkout payment	Cart → Stripe	§16
35	Payment confirmation + receipt	Cart (same page)	§16.6
36	Order history	`pages/Orders.jsx`	§16.8
37	Shipping details capture	`ShippingModal`	§17
38	Admin bootstrap via secret	`pages/AdminSetup.jsx`	§18.2
39	Admin: manage Collection + stats	`AdminCollectionTab`	§18.3

#	FEATURE	PRIMARY UI	DETAILED IN
40	Admin: manage Store	`AdminStoreTab`	§18.4
41	Admin: member directory + loans	`AdminMembersTab`	§18.5
42	Account info page	`pages/AccountInfo.jsx`	§7.2
43	Home page + editorial content	`pages/Home.jsx`	§7.2
44	Genres / About / 404	Genres, About, NotFound	§7.2
45	Navbar search → Collection	Navbar	§7.2
46	Broken-cover fallback image	`BookCover`	§7.2

7.2 The smaller features, briefly

Home page

Hero over a library photograph, then `api.get("/books", { params: { sort: "rating" } })` sliced to the top 8. Editorial cards and the stat strip ("10K+ books", "50K+ readers") are **hard-coded marketing copy** in the component, not database values — worth knowing so you never claim they are live figures.

Navbar search

Submitting the navbar search navigates to `/collection?search=<term>`. It does not fetch anything itself — it just writes the URL and lets Collection's effect do the work. A clean illustration of URL-as-state.

BookCover fallback

Covers are remote URLs from `covers.openlibrary.org` or user input, so some will 404. `onError` swaps in an inline SVG data-URI, and sets `onerror = null` first to prevent an infinite error loop if the fallback itself failed. Also uses `loading="lazy"`.

AccountInfo

Pure presentation — reads `profile` and `points` straight from `useAuth()` and makes **no API calls of its own**. Shows name, email, "Administrator" or "Member", join date, and links to Reviews / Wishlist / My Library.

PLANNED BUT NOT CURRENTLY IMPLEMENTED

- **Pagination** — no page/limit/skip anywhere. Every list endpoint returns all matches.
- **Related / recommended books** — no such endpoint or component exists.
- **Editing or deleting your own review** — reviews are write-once.
- **Shipping on Store orders** — Stripe collects an email; no address is gathered for Store purchases.
- **Order status beyond "paid"** — the field exists but is never advanced to shipped/delivered.
- **Admin management of community books** — deliberately absent; owners control their own.
- **Stripe webhooks**, **email notifications**, **password reset UI**, **profile editing**, **image upload**, **tests** — none present.

SECTION EIGHT

Official Library Collection

The MongoDB-backed shelf: browsing, searching, sorting, borrowing, returning — and where each operation actually executes.

8.1 What the user can do

Browse every book the library owns; search by title and by author simultaneously; filter to one of eight categories; sort four ways; open a detail page; borrow (if available); return; buy permanently with points; wishlist; and read or write reviews. All of it except borrowing, buying, wishlisting and reviewing works signed out.

8.2 The data model

backend/models/Book.js

```
const bookSchema = new mongoose.Schema({
  title:         { type: String, required: true, trim: true },
  author:        { type: String, required: true, trim: true },
  description:   { type: String, required: true },
  category:      { type: String, required: true },
  publishedYear: { type: Number, required: true },
  pages:         { type: Number, required: true },
  coverImage:    { type: String, required: true },
  rating:        { type: Number, default: 4, min: 0, max: 5 },

  // A borrowed book is still the library's — it comes back. A book bought with
  // points does not, so it is deleted outright rather than flagged; the only
  // trace left is the buyer's purchase record. Anything that needs to survive
  // that delete (borrow history, reviews) keeps its own snapshot of the book.
  available:     { type: Boolean, default: true },
  createdAt:     { type: Date, default: Date.now },
});
```

That comment encodes a real design decision, arrived at over two commits (§28.3). The model has exactly one lifecycle flag, and it means *on loan* — not *sold*. Sold books do not exist.

backend/models/Borrow.js

```
const borrowSchema = new mongoose.Schema({
  userId:     { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  bookId:     { type: mongoose.Schema.Types.ObjectId, ref: "Book", required: true },
  borrowedAt: { type: Date, default: Date.now },
  returnedAt: { type: Date, default: null },
  status:     { type: String, enum: ["borrowed", "returned"], default: "borrowed" },

  shipping:   { name: String, phone: String, address: String },

  // A copy of the book as it was when borrowed. If the title is later bought
  // with points the Book document is deleted, and without this the reader's
  // history would populate to null and render as "Untitled".
  bookSnapshot: { title: String, author: String, coverImage: String },
});
```

`Borrow` is a **join document with attributes** — it links a user and a book *and* records facts about that specific loan (when, where to deliver, whether returned). Storing shipping here rather than on the user means a delivery address is tied to the one loan it was given for, instead of lingering on the account. That is a privacy decision, and it is stated in the source comment.

8.3 Searching and filtering — where it happens

backend/routes/books.js

```
router.get("/", async (req, res) => {
  try {
    const { search, author, category, sort } = req.query;
    const query = {};

    if (search)   query.title  = { $regex: search, $options: "i" };
    if (author)   query.author = { $regex: author, $options: "i" };
    if (category && category !== "All") query.category = category;

    let sortOption = { createdAt: -1 };
    if      (sort === "oldest") sortOption = { createdAt: 1 };
    else if (sort === "az")     sortOption = { title: 1 };
    else if (sort === "rating") sortOption = { rating: -1 };
    else if (sort === "newest") sortOption = { createdAt: -1 };

    const books = await Book.find(query).sort(sortOption);
    res.json(books);
  } catch (err) {
    res.status(500).json({ message: "Failed to fetch books", error: err.message });
  }
});
```

ANSWER TO "WHERE DOES THE COLLECTION SEARCH HAPPEN?"

Inside MongoDB. The filters are compiled into a MongoDB query object and executed by the database engine. React sends parameters; Express translates them into a query; MongoDB does the work. **Nothing is filtered in JavaScript.** Contrast this with the Store and Community shelves (§14.4, §9.2), where filtering *is* done in JavaScript — and be ready to explain why the two differ.

The query object is built **conditionally**. With no parameters it stays `{}`, which matches everything. Each supplied parameter adds a key. Combining search + author + category produces:

```
{
  title:    { $regex: "gats", $options: "i" },
  author:   { $regex: "fitz", $options: "i" },
  category: "Fiction"
}
```

MongoDB ANDs top-level keys implicitly, so all three must match. `$options: "i"` makes it case-insensitive, and `$regex` without anchors matches a **substring** — so "gats" finds "The Great Gatsby".

8.4 Sorting — a complete trace

Scenario: "Uday selects Newest First."

1. Uday picks "Newest First" from the sort `<select>`.
   `pages/Collection.jsx line ~128`
2. `onChange` → `updateParam("sort", "newest")`. A new `URLSearchParams` is built from the current one and the `sort` key is set.
3. `setSearchParams(next)` — the URL becomes `/collection?sort=newest` (plus any existing filters). This is a router navigation, so it enters browser history.
4. React Router re-renders Collection. `const sort = searchParams.get("sort")` now yields `"newest"`.
5. The dependency array `[search, author, category, sort]` changed → the effect re-runs → `setLoading(true)` → skeletons appear.
6. `api.get("/books", { params: { search, author, category, sort } })`. Axios serialises this to a query string; the interceptor attaches a token if signed in (this endpoint is public either way).
7. **On the wire:** `GET /api/books?search=&author=&category=All&sort=newest`
8. **Express:** `search` and `author` are empty strings (falsy → skipped); `category === "All"` → skipped. `query` stays `{}`.
9. `sort === "newest"` → `sortOption = { createdAt: -1 }`. **`-1` is descending**, so the most recent `createdAt` comes first.
10. **MongoDB executes** `db.books.find({}).sort({ createdAt: -1 })` and returns the ordered array.
11. `res.json(books)` → an array of book documents, newest first.
12. `setBooks(data)`; `setLoading(false)` in the `finally`.
13. **Uday sees:** "24 books found", then the grid re-renders in the new order, each `BookCard` animating in with a staggered delay. The URL is shareable and Back undoes the sort.

UI LABEL	PARAM	MONGO SORT	MEANING
Newest First	`newest`	`{ createdAt: -1 }`	Descending by creation date (also the default)
Oldest First	`oldest`	`{ createdAt: 1 }`	Ascending by creation date
Title A–Z	`az`	`{ title: 1 }`	Ascending alphabetical
Highest Rated	`rating`	`{ rating: -1 }`	Descending by the book's stored rating

> **DETAIL WORTH NOTICING**
>
> "Highest Rated" sorts by `Book.rating` — the **seeded/admin-set** value stored in MongoDB — **not** by the average of user reviews, which live in Firestore. The two rating systems are independent. If asked "why not sort by real review average?", the honest answer is: reviews are in a different database, so sorting by them would require reading every review and sorting in memory. Being able to state that trade-off precisely is the point.

8.5 The book card

`BookCard` is presentational. It receives `book` and `index`, links to `/books/{book._id}`, and shows an availability badge (green "Available" / dark "Borrowed"), the category, the title, author, `RatingStars` and the published year. `index` exists only to stagger the entry animation.

8.6 The detail page

`BookDetails` reads `const { id } = useParams()` and fetches `/books/{id}`. Interesting behaviours:

- **One modal, two intents.** A single `intent` state holds `"borrow" | "purchase" | null`, and one `ShippingModal` serves both. The modal's title, subtitle and button label are computed from `intent`, and `handleConfirm` branches on it. Two flows that need identical input therefore cannot drift apart.
- **Points come from context**, not from a local fetch — so the buy button, the navbar badge and My Library always agree.
- **The buy button is disabled while the book is on loan**, with a tooltip explaining why. The library owns one copy; it cannot be posted to a buyer while someone is reading it.
- **After a purchase the page navigates away** — the book has been deleted, so there is nothing left to render. It routes to `/my-library` with `state: { purchased: "..." }`, and MyLibrary opens on the Purchased tab and shows the toast.

```
const handleConfirm = async (shipping) => {
  setSubmitting(true); setModalError("");
  try {
    if (intent === "purchase") {
      const res = await purchaseBook(id, shipping);
      setIntent(null);
      refreshPoints();
      navigate("/my-library", { replace: true,
        state: { purchased: `Purchased! ${res.pointsSpent} points spent — it's yours to keep.` } });
      return;
    }
    await api.post(`/books/${id}/borrow`, shipping);
    setMessage({ type: "success", text: "Book borrowed! Check My Library." });
    setIntent(null);
    fetchBook();        re-read so the badge flips to "Borrowed"
    refreshPoints();
  } catch (err) {
    setModalError(err.response?.data?.message || "Something went wrong.");
  } finally { setSubmitting(false); }
};
```

8.7 Borrowing — complete flow

backend/routes/books.js — POST /api/books/:id/borrow

```
router.post("/:id/borrow", verifyFirebaseToken, async (req, res) => {
  try {
    const book = await Book.findById(req.params.id);
    if (!book) return res.status(404).json({ message: "Book not found" });
    if (!book.available)
      return res.status(400).json({ message: "This book is currently unavailable" });

    const name    = (req.body.shippingName    || "").trim();
    const phone   = (req.body.shippingPhone   || "").trim();
    const address = (req.body.shippingAddress || "").trim();
    if (!name || !phone || !address)
      return res.status(400).json({ message: "Name, phone number and home address are required to borrow" });

    const alreadyBorrowed = await Borrow.findOne({
      userId: req.user._id, bookId: book._id, status: "borrowed",
    });
    if (alreadyBorrowed)
      return res.status(400).json({ message: "You have already borrowed this book" });

    const borrow = await Borrow.create({
      userId: req.user._id,
      bookId: book._id,
      shipping: { name, phone, address },
      bookSnapshot: { title: book.title, author: book.author, coverImage: book.coverImage },
    });
    book.available = false;
    await book.save();
```

```
    res.status(201).json(borrow);
  } catch (err) {
    res.status(500).json({ message: "Failed to borrow book", error: err.message });
  }
});
```

Four guards, in order: the book exists → it is available → shipping is complete → you do not already have it. Only then is the record created and the book marked unavailable.

A REAL RACE CONDITION, HONESTLY STATED

Borrowing uses `if (!book.available) … ; book.available = false; await book.save();` — a **check-then-act** across two operations. Two simultaneous requests could both pass the check and both create a borrow record. Compare this with `purchaseOfficialBook`, which uses an atomic `findOneAndUpdate({ _id, available: true }, …)` and is immune. The purchase path was hardened because points are money; the borrow path was not. If asked "what would you improve?", this is your best answer — you can name the exact fix and point at code in the same repository that already does it correctly.

8.8 Returning

```
router.post("/:id/return", verifyFirebaseToken, async (req, res) => {
  const borrow = await Borrow.findOne({
    bookId: req.params.id, userId: req.user._id, status: "borrowed",
  });
  if (!borrow) return res.status(404).json({ message: "No active borrow record found for you on this book" });

  borrow.status = "returned";
  borrow.returnedAt = new Date();
  await borrow.save();

  await Book.findByIdAndUpdate(req.params.id, { available: true });
  res.json(borrow);
});
```

The query itself is the authorisation: it looks for a borrow matching **this book**, **this user**, **still active**. Someone attempting to return a book they never borrowed simply finds no record and receives a 404. There is no separate ownership check because the query cannot match another person's loan. This directly answers the interview question *"how do you prevent a user returning someone else's book?"*

8.9 "My loans", and the snapshot fallback

backend/routes/borrow.js — GET /api/borrowed/me

```
const records = await Borrow.find({ userId: req.user._id })
  .populate("bookId")
  .sort({ borrowedAt: -1 });

// populate() yields null once a book has been bought with points and
// deleted, so fall back to the snapshot taken when the loan was made. The
// shape stays the same either way, and `bookGone` lets the UI drop the link.
const withFallback = records.map((r) => {
  const doc = r.toObject();
  if (!doc.bookId && doc.bookSnapshot) {
    doc.bookId = {
      _id: null,
      title: doc.bookSnapshot.title,
      author: doc.bookSnapshot.author,
      coverImage: doc.bookSnapshot.coverImage,
    };
    doc.bookGone = true;
  }
```

```
  return doc;
});
```

`.populate("bookId")` replaces the stored ObjectId with the full Book document — Mongoose's equivalent of a join. But `populate` **yields** `null` **when the referenced document no longer exists**, and Bookverse deliberately deletes books that are bought with points. Without this fallback, a reader's history would render "Untitled" with a broken image.

The fallback **reshapes the snapshot to look exactly like a populated book**, so the React component needs no special case for rendering — it reads `r.bookId.title` either way. The extra `bookGone` flag is the only thing the UI branches on, and only to drop the link and add a "No longer in the library" label. That is a very clean way to handle a dangling reference.

8.10 Collection statistics

backend/routes/stats.js

```
const totalBooks     = await Book.countDocuments();
const availableBooks = await Book.countDocuments({ available: true });
const borrowedBooks  = totalBooks - availableBooks;
const categories     = (await Book.distinct("category")).length;

// Sold copies are deleted from the collection, so the only remaining record
// of them is the purchases ledger.
const purchasedSnap = await db.collection(COLLECTIONS.purchases).count().get();
const purchasedBooks = purchasedSnap.data().count;

res.json({ totalBooks, availableBooks, borrowedBooks, purchasedBooks, categories });
```

This one endpoint reads from **both databases** — a compact illustration of the dual-store architecture. `borrowedBooks` is derived by subtraction rather than a third query. `.count().get()` is a Firestore **aggregation query**: the server returns only the number, never the documents, so it stays cheap as the ledger grows.

SECTION NINE

Community / Used-Book System

Peer-to-peer lending in Firestore: contributing, requesting, approving, returning — and the ownership rules that hold it together.

9.1 The Firestore data model

`usedBooks/{autoId}`

```
{
  title, author, description,
  category, coverImage, condition,
  ownerUid:  "firebase uid",
  ownerName: "Uday Barua",
  available: true,
  createdAt: serverTimestamp
}
```

`borrowRequests/{autoId}`

```
{
  bookId, bookTitle, bookCover,
  ownerUid, requesterUid, requesterName,
  shipping: { name, phone, address },
  status: "pending" | "approved"
        | "declined" | "returned",
  createdAt, respondedAt?, returnedAt?
}
```

Both documents are **denormalised on purpose**. A request stores `bookTitle` and `bookCover` rather than only `bookId`, so the requests list renders in one read instead of N follow-up lookups. It also stores `ownerName` / `requesterName` so no user lookup is needed to display "Requested by Uday". This is standard NoSQL practice: duplicate read-heavy display data to avoid joins a document database cannot do efficiently.

9.2 Browsing and searching — filtered in JavaScript

`backend/controllers/communityController.js`

```
async function listUsedBooks(req, res, next) {
  try {
    const { search, author, category } = req.query;
    const snap = await db.collection(COLLECTIONS.usedBooks).orderBy("createdAt", "desc").get();
    let books = snap.docs.map((d) => ({ id: d.id, ...d.data() }));

    // Firestore has no case-insensitive contains, and the community shelf is
    // small, so the text filters are applied here rather than as queries.
    if (search)   { const q = search.toLowerCase();  books = books.filter((b) =>
b.title?.toLowerCase().includes(q)); }
    if (author)   { const q = author.toLowerCase();  books = books.filter((b) =>
b.author?.toLowerCase().includes(q)); }
    if (category && category !== "All") books = books.filter((b) => b.category === category);

    res.json(books);
  } catch (err) { next(err); }
}
```

ANSWER TO "WHERE DOES THE COMMUNITY-BOOK SEARCH HAPPEN?"

In Node.js memory, on the server — not in Firestore, and not in the browser. Firestore fetches *all* used books ordered by date, and JavaScript's `Array.filter` narrows them. **Why:** Firestore has no case-insensitive substring operator. It offers equality, ranges, `array-contains` and `in` — but nothing resembling SQL's `LIKE '%gats%'`. A prefix search is possible with a range trick (`>= "gats"` , `< "gats"`) but it is case-**sensitive** and prefix-only. The honest engineering answer is: at this scale, fetch-and-filter is correct and simple; at scale, you denormalise a lowercased search field or put the data in Algolia / Elasticsearch. Note it is filtered **on the server**, so the browser still receives only matching results.

Note also `b.title?.toLowerCase()` — optional chaining, because a document written without a title would otherwise throw `Cannot read properties of undefined` and take down the whole listing. Firestore is schemaless, so defensive reads are appropriate.

9.3 Contributing a used book

```
async function addUsedBook(req, res, next) {
  try {
    const { title, author, description, category, coverImage, condition } = req.body;
    if (!title || !author || !coverImage)
      return res.status(400).json({ message: "Title, author and cover image are required" });

    const book = {
      title: title.trim(), author: author.trim(),
      description: (description || "").trim(),
      category: category || "Fiction",
      coverImage: coverImage.trim(),
      condition: condition || "Good",
      ownerUid:  req.user.firebaseUid,     ← from the VERIFIED token
      ownerName: req.user.name,
      available: true,
      createdAt: FieldValue.serverTimestamp(),
    };
    const ref = await db.collection(COLLECTIONS.usedBooks).add(book);

    // The award is applied server-side; clients cannot write their own points.
    await db.collection(COLLECTIONS.users).doc(req.user.firebaseUid).set(
      { points: FieldValue.increment(POINTS_PER_CONTRIBUTION),
        name: req.user.name, email: req.user.email },
      { merge: true }
    );

    res.status(201).json({ id: ref.id, ...book, pointsAwarded: POINTS_PER_CONTRIBUTION });
  } catch (err) { next(err); }
}
```

Three things to be able to explain here:

- `FieldValue.increment(50)` is an **atomic server-side operation**. It is not read-then-write from Node — Firestore performs the addition itself, so two concurrent contributions both count. A naive `points = current + 50` would lose one of them.
- `{ merge: true }` makes `.set()` behave as an upsert that preserves other fields. A user contributing for the first time has no `users` document yet; merge creates it. An existing user keeps their other fields untouched.
- The response echoes `pointsAwarded`, so the UI can say "You earned 50 points" using the **server's** number rather than a hard-coded 50.

9.4 Withdrawing your own book

```
async function deleteUsedBook(req, res, next) {
  const ref = db.collection(COLLECTIONS.usedBooks).doc(req.params.id);
  const doc = await ref.get();
  if (!doc.exists) return res.status(404).json({ message: "Book not found" });
  if (doc.data().ownerUid !== req.user.firebaseUid)
    return res.status(403).json({ message: "You can only remove your own books" });
  await ref.delete();
  res.json({ message: "Book removed" });
}
```

The classic ownership pattern. The UI additionally disables the Remove button while a book is on loan (`disabled={busyId === b.id || !b.available}`) — though note that is a **client-side** restriction only; the server would permit deleting a book that is out on loan. A small, genuine gap worth mentioning.

Points are deliberately **not** deducted when a book is withdrawn. Defensible: the points were earned for the act of sharing. Also exploitable: add and remove repeatedly to farm points. A good thing to raise yourself, with the fix — decrement on delete, or award only once per unique title.

9.5 Requesting to borrow

```
async function requestBorrow(req, res, next) {
  const shipping = readShipping(req.body);
  if (!shipping) return res.status(400).json({ message: "Name, phone number and home address are required to borrow"
});

  const bookRef = db.collection(COLLECTIONS.usedBooks).doc(req.params.id);
  const bookDoc = await bookRef.get();
  if (!bookDoc.exists) return res.status(404).json({ message: "Book not found" });

  const book = bookDoc.data();
  if (book.ownerUid === req.user.firebaseUid)
    return res.status(400).json({ message: "This is your own book" });
  if (!book.available)
    return res.status(400).json({ message: "This book is currently unavailable" });

  const existing = await db.collection(COLLECTIONS.borrowRequests)
    .where("bookId", "==", req.params.id)
    .where("requesterUid", "==", req.user.firebaseUid)
    .where("status", "==", "pending").get();
  if (!existing.empty)
    return res.status(400).json({ message: "You already have a pending request for this book" });

  const request = { bookId: req.params.id, bookTitle: book.title, bookCover: book.coverImage,
                    ownerUid: book.ownerUid, requesterUid: req.user.firebaseUid,
                    requesterName: req.user.name, shipping, status: "pending",
                    createdAt: FieldValue.serverTimestamp() };
  const ref = await db.collection(COLLECTIONS.borrowRequests).add(request);

  await notify(book.ownerUid, {
    type: "borrow_request",
    title: "New borrow request",
    body: `${req.user.name} would like to borrow "${book.title}".`,
    requestId: ref.id,
  });

  res.status(201).json({ id: ref.id, ...request });
}
```

Four guards: shipping complete → book exists → not your own book → book available → no duplicate pending request. The duplicate check is a **compound query** on three equality filters, which Firestore serves from its automatic single-field indexes without needing a composite index.

9.6 Approving or declining

```
const { action } = req.body;
if (!["approve", "decline"].includes(action))
  return res.status(400).json({ message: "Action must be approve or decline" });

const doc = await ref.get();
if (!doc.exists) return res.status(404).json({ message: "Request not found" });

const request = doc.data();
if (request.ownerUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "Only the book's owner can answer this request" });
if (request.status !== "pending")
  return res.status(400).json({ message: "This request was already answered" });
```

```
const status = action === "approve" ? "approved" : "declined";
await ref.update({ status, respondedAt: FieldValue.serverTimestamp() });

if (status === "approved")
  await db.collection(COLLECTIONS.usedBooks).doc(request.bookId).update({ available: false });

await notify(request.requesterUid, { type: "request_" + status, ... });
```

The `status !== "pending"` check is a **state-machine guard** — it stops an already-answered request being answered again, which would send a duplicate notification and could flip availability incorrectly. The allowed transitions are:

pending → approved → returned or pending → declined (terminal)

9.7 Returning

```
if (request.requesterUid !== req.user.firebaseUid)
  return res.status(403).json({ message: "This is not your borrow record" });
if (request.status !== "approved")
  return res.status(400).json({ message: "This book is not currently on loan to you" });

await ref.update({ status: "returned", returnedAt: FieldValue.serverTimestamp() });
await db.collection(COLLECTIONS.usedBooks).doc(request.bookId).update({ available: true });
await notify(request.ownerUid, { type: "book_returned", ... });
```

Note the **asymmetry of authority**: the *owner* approves, but the *borrower* returns. Each action is restricted to the person who can actually perform it in the physical world. This directly answers *"how does the system prevent unauthorised return of another user's book?"*

9.8 Complete scenario — the full lifecycle

Uday shares a book; Rafi borrows it; Rafi returns it.

1. **Uday** opens My Library → "Add a used book", fills the form and submits.
   `pages/MyLibrary.jsx → components/AddUsedBookModal.jsx`
2. `addUsedBook(form)` → `POST /api/community/books`
   `services/communityService.js`
3. **Firestore write #1:** a new `usedBooks` doc with `ownerUid = Uday`, `available: true`.
4. **Firestore write #2:** `users/{uday}` merged with `points: increment(50)`. Uday goes 0 → 50.
5. Response `201 { ..., pointsAwarded: 50 }`. Toast: "Thanks for sharing! You earned 50 points." `loadAll()` refreshes every tab; `refreshPoints()` updates the navbar badge.
6. **Rafi** opens `/community`, types "atomic". The server fetches all used books and filters in memory; Uday's book appears.
   `pages/Community.jsx`
7. Rafi opens the detail page and clicks "Request to borrow". `isOwner` is false, so the button renders.
   `pages/CommunityBookDetails.jsx`
8. `ShippingModal` collects name, phone, address. `POST /api/community/books/{id}/request`
9. **Firestore write #3:** a `borrowRequests` doc, `status: "pending"`, carrying Rafi's **shipping address**.
10. **Firestore write #4:** a `notifications` doc addressed to `userUid: Uday`.
11. **Uday's browser reacts instantly.** His `onSnapshot` listener is watching `notifications where userUid == uday`; Firestore pushes the new document. The bell badge turns 1 **with no page reload and no polling**.
    `components/NotificationBell.jsx`
12. Uday opens My Library → Requests, sees "Requested by Rafi", clicks **Approve**.
13. **Firestore writes #5–7:** request → `approved` + `respondedAt`; used book → `available: false`; a notification to Rafi.
14. The UI now reveals the delivery address to Uday — **only after approval**:
    `{r.status === "approved" && (<p>Deliver to: {r.shipping?.name}, {r.shipping?.address}…</p>)}`
15. Rafi's bell lights up: "Borrow request approved". The community card now shows "On loan"; the Request button is disabled for everyone.
16. Rafi finishes the book, opens My Library → Requests → outgoing, clicks **Return book**.
17. **Firestore writes #8–10:** request → `returned` + `returnedAt`; used book → `available: true`; a notification to Uday.
18. **Final state:** the book is back on the shelf and requestable again; both users hold a complete audit trail; Uday keeps his 50 points.

Ten Firestore writes across the lifecycle — **every one performed by Express through the Admin SDK**. The browsers involved only ever *read*.

9.9 Incoming vs outgoing requests

```
const [incomingSnap, outgoingSnap] = await Promise.all([
  db.collection(COLLECTIONS.borrowRequests).where("ownerUid",     "==", req.user.firebaseUid).get(),
  db.collection(COLLECTIONS.borrowRequests).where("requesterUid", "==", req.user.firebaseUid).get(),
]);

const map = (snap) => snap.docs
  .map((d) => ({ id: d.id, ...d.data() }))
  .sort((a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0));
```

```
res.json({ incoming: map(incomingSnap), outgoing: map(outgoingSnap) });
```

Two queries in `Promise.all` so they run concurrently rather than sequentially. Sorting is done in JavaScript rather than with `.orderBy()` because combining a `where` with an `orderBy` on a different field requires a **composite index** in Firestore — and there is no `firestore.indexes.json` in this project. Sorting in memory sidesteps that entirely. Note the very defensive `b.createdAt?.toMillis?.() || 0`: a document whose `serverTimestamp()` has not yet materialised has a null `createdAt`, and this treats it as 0 instead of throwing. That exact expression appears eight times across the codebase.

SECTION TEN

Notification System

How a borrow request reaches its owner in real time — and precisely what technology does and does not make that happen.

ACCURACY FIRST — NO FIREBASE CLOUD FUNCTIONS EXIST

There is **no** `functions/` **directory, no** `firebase.json` **, and no Cloud Function of any kind** in this repository. Notifications are **not** triggered by a serverless Firestore trigger. Every notification document is written explicitly by the Express API, inline, during the request that causes it. The real-time delivery comes from a **client-side Firestore listener**, which is a completely different mechanism. If an interviewer assumes Cloud Functions are involved, correcting them precisely is a strong moment.

10.1 The two halves

Write path — Express, Admin SDK

A notification is created as a side-effect of a business action, in the same controller, in the same request. Three call sites, all in `communityController.js`.

Read path — browser, client SDK

`onSnapshot` holds an open connection. Firestore **pushes** changes to the browser. No polling, no refresh, no WebSocket code written by hand.

10.2 Creating a notification

`backend/controllers/communityController.js`

```
async function notify(userUid, payload) {
  await db.collection(COLLECTIONS.notifications).add({
    userUid,
    read: false,
    createdAt: FieldValue.serverTimestamp(),
    ...payload,
  });
}
```

Six lines, one job. `userUid` is **the recipient** — not the actor. This is the field the listener and the security rules both key on. `read: false` gives every notification a consistent starting state so the unread count never has to handle `undefined`. The spread `...payload` comes last so a caller could override defaults if it ever needed to.

All four notification types

TYPE	RECIPIENT	TRIGGERED BY	BODY
`borrow_request`	Book owner	`requestBorrow`	"Rafi would like to borrow "Atomic Habits"."
`request_approved`	Requester	`respondToRequest`	"Uday approved your request… It will be parcelled to you."
`request_declined`	Requester	`respondToRequest`	"Uday declined your request for "…"."
`book_returned`	Book owner	`returnUsedBook`	"Rafi has returned "Atomic Habits"."

The type string for approve/decline is built dynamically: `type: "request_" + status`. Only the community system generates notifications — borrowing an *official* book produces none, because there is no counterparty to inform.

10.3 The live listener

`frontend/src/components/NotificationBell.jsx`

```
// A live Firestore listener, so a borrow request reaches the owner the moment
// it is created rather than on their next page load. Writes still go through
// the API — this subscription is read-only.
useEffect(() => {
  if (!firestore || !firebaseUser) { setItems([]); return; }

  const q = query(
    collection(firestore, "notifications"),
    where("userUid", "==", firebaseUser.uid)
  );

  const unsubscribe = onSnapshot(
    q,
    (snap) => {
      const list = snap.docs
        .map((d) => ({ id: d.id, ...d.data() }))
        .sort((a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0));
      setItems(list);
    },
    (err) => console.error("Notification listener failed", err)
  );

  return unsubscribe;
}, [firebaseUser]);
```

Line by line:

- `if (!firestore || !firebaseUser)` — two guards. No Firebase config (dev without `.env`) or nobody signed in: clear the list and open no connection.
- `query(collection(...), where(...))` — the modular v9+ SDK style. The query is declarative and is sent to Firestore, which watches **only** the matching documents.
- `where("userUid", "==", firebaseUser.uid)` — this is simultaneously a filter **and** the thing that satisfies the security rule. A listener without this filter would try to read documents belonging to others and Firestore would reject the entire query.
- `onSnapshot` fires immediately with the current state, then again on **every** change to the matching set — server-pushed, no polling.
- The third argument is an **error callback**. Without it, a permission error would surface as an unhandled rejection; with it, the failure is logged and the UI simply shows nothing.
- `return unsubscribe` — the cleanup. `onSnapshot` returns its own unsubscribe function, so returning it directly closes the connection when the component unmounts or when `firebaseUser` changes. **Omitting this leaks an open connection per sign-in.**
- `[firebaseUser]` — re-subscribes when the user changes, so switching accounts never shows the previous user's notifications.

WHY A LISTENER INSTEAD OF POLLING

Polling `GET /api/notifications` every 10 seconds would mean a request per user per 10 s forever, most returning nothing, and up to 10 s of delay. `onSnapshot` holds one connection and delivers in well under a second, at zero cost when nothing happens. This is the single strongest justification for Firestore being in this project at all.

10.4 Complete real-time flow

1. **Uday's browser** has an open `onSnapshot` on `notifications where userUid == uday`. He is reading an unrelated page.
2. **Rafi**, elsewhere, submits a borrow request → `POST /api/community/books/{id}/request`.
3. `requestBorrow` creates the request document, then calls `notify(book.ownerUid, {...})`.
4. **Firestore** stores a notification with `userUid: uday`.
5. **Firestore matches it against every open listener.** Uday's query matches → the document is pushed down his existing connection.
6. The snapshot callback fires in Uday's browser. `setItems(list)` — React re-renders.
7. `const unread = items.filter((i) => !i.read).length` is recomputed → `1`.
8. **Uday sees** a coral badge appear on the bell. He never reloaded, and Rafi's request took under a second to arrive.

10.5 Marking as read — and why it needs the API

Reading is direct; **writing is not**. The rules deny client writes, so marking read must go through Express.

backend/routes/notifications.js

```
// The bell reads notifications live from Firestore on the client; these routes
// exist for the writes, which clients are not permitted to make directly.
router.post("/:id/read", verifyFirebaseToken, async (req, res, next) => {
  const ref = db.collection(COLLECTIONS.notifications).doc(req.params.id);
  const doc = await ref.get();
  if (!doc.exists) return res.status(404).json({ message: "Notification not found" });
  if (doc.data().userUid !== req.user.firebaseUid)
    return res.status(403).json({ message: "Not your notification" });
  await ref.update({ read: true });
  res.json({ message: "Marked read" });
});

router.post("/read-all", verifyFirebaseToken, async (req, res, next) => {
  const snap = await db.collection(COLLECTIONS.notifications)
    .where("userUid", "==", req.user.firebaseUid)
    .where("read", "==", false)
    .get();

  const batch = db.batch();
  snap.docs.forEach((d) => batch.update(d.ref, { read: true }));
  await batch.commit();

  res.json({ message: "All marked read", count: snap.size });
});
```

`db.batch()` groups up to 500 writes into one atomic commit — one network round trip instead of N, and either all succeed or none do. Marking 12 notifications read is a single operation, not 12.

The UI marks all read on opening the dropdown:

```
onClick={() => { setOpen((s) => !s); if (!open) markAllRead(); }}
```

`if (!open)` reads the value *before* the toggle, so the call fires only when opening. And `markAllRead` short-circuits with `if (!unread) return;` — no pointless request when there is nothing unread.

THE SATISFYING PART

The component **never updates its own list after marking read**. It does not need to: the API writes to Firestore, Firestore pushes the change back down the open listener, and the badge clears by itself. The write path and read path close the loop automatically. That is the unidirectional data flow working exactly as intended.

10.6 Notification security

```
match /notifications/{notificationId} {
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
  allow write: if false;
}
```

`resource.data.userUid` is the **stored document's** field, evaluated per document. A user querying without the matching `where` clause has their whole query rejected — Firestore requires that a query provably cannot return documents the rules would refuse. Writes are impossible, so no one can mark someone else's notification read or fabricate one.

SECTION ELEVEN

Contribution Point System

The internal currency: earning, storing, protecting and spending — including the most sophisticated piece of code in the project.

11.1 The economy

backend/lib/firestore.js

```
// A contributor earns this for each used book they share; an official book
// costs this many points to own permanently. Kept here so the economy has a
// single source of truth on the server.
const POINTS_PER_CONTRIBUTION = 50;
const POINTS_TO_PURCHASE = 200;
```

Four shared books buy one owned book. Both numbers live in **one server-side module** and are **served to the client** rather than duplicated in it:

```
async function getMyPoints(req, res, next) {
  const doc = await db.collection(COLLECTIONS.users).doc(req.user.firebaseUid).get();
  res.json({
    points: doc.exists ? doc.data().points || 0 : 0,
    pointsPerContribution: POINTS_PER_CONTRIBUTION,
    pointsToPurchase: POINTS_TO_PURCHASE,
  });
}
```

Hence the UI writes `{points?.pointsToPurchase ?? 200}` — the server's value with a fallback for the moment before it loads. Change 200 to 250 in one file and every screen updates. The `doc.exists ? … : 0` handles a user who has never contributed and therefore has no `users` document, and `|| 0` handles a document that exists without a `points` field.

11.2 Storage

```
users/{firebaseUid}
{
  points: 150,
  name:  "Uday Barua",
  email: "uday@example.com"
}
```

The **document id is the Firebase UID**, which means no query is ever needed to find someone's balance — it is a direct document lookup, the fastest read Firestore offers. It also makes the security rule trivial: `request.auth.uid == userId` compares the caller against the document id itself.

TWO `USERS` STORES — DO NOT CONFUSE THEM

MongoDB's `users` collection holds identity and **role**. Firestore's `users` collection holds **points** (with name/email denormalised for convenience). They are linked only by the Firebase UID. This is the least tidy part of the architecture, and §19.5 explains the reasoning honestly.

11.3 Earning points

```
await db.collection(COLLECTIONS.users).doc(req.user.firebaseUid).set(
  { points: FieldValue.increment(POINTS_PER_CONTRIBUTION), name: req.user.name, email: req.user.email },
```

```
  { merge: true }
);
```

Contributing a used book is currently the **only** way to earn points. `increment()` is atomic on the server; `merge: true` upserts. Both matter, and both are covered in §9.3.

11.4 How points are protected

ATTACK	WHY IT FAILS
Write points from the browser console	`firestore.rules`: `match /users/{userId} { allow write: if false; }`
Send `{ points: 99999 }` to an API	No endpoint anywhere accepts a points value. The only mutations are server-computed `increment(+50)` and `increment(-200)`.
Read someone else's balance	`allow read: if signedIn() && request.auth.uid == userId`
Spend the same 200 points twice concurrently	The balance is re-read **inside** a Firestore transaction, which retries on conflict
Change the price of a book in points	`POINTS_TO_PURCHASE` is a server constant; the client only ever displays it
Award yourself by replaying a contribution	Each award requires a real `usedBooks` document to be created first, in the same request

11.5 Spending points — the cross-database purchase

This is the most architecturally interesting function in Bookverse. The problem it solves: **the book is in MongoDB and the points are in Firestore, so no single transaction can cover both.**

backend/controllers/pointsController.js — POST /api/points/purchase/:bookId

```
// ── Phase 1: validate ──────────────────────────────────────
const name = (req.body.shippingName || "").trim();  (+ phone, address)
if (!name || !phone || !address)
  return res.status(400).json({ message: "Name, phone number and home address are required to purchase" });

const book = await Book.findById(req.params.bookId);
if (!book) return res.status(404).json({
  message: "This book is no longer in the collection — it may have just been purchased." });

// The library lends one copy; it can't be posted to a buyer while someone
// else is reading it.
if (!book.available) {
  const mine = await Borrow.findOne({ bookId: book._id, userId: req.user._id, status: "borrowed" });
  return res.status(400).json({ message: mine
    ? "You're currently borrowing this book. Return it first, then you can buy it."
    : "This book is currently borrowed by another reader, so it can't be purchased yet." });
}

// ── Phase 2: CLAIM (atomic) ────────────────────────────────
// MongoDB holds the book and Firestore holds the points, so there is no
// single transaction spanning both. Reserve the copy first with one atomic
// conditional update — whoever wins this update owns the sale — then spend
// the points, and put it back if that fails.
const claimed = await Book.findOneAndUpdate(
  { _id: book._id, available: true },
  { $set: { available: false } },
  { new: true }
);
if (!claimed) return res.status(400).json({
  message: "This book was just taken by someone else. Please refresh and try again." });

// ── Phase 3: PAY (transactional) ───────────────────────────
const userRef = db.collection(COLLECTIONS.users).doc(req.user.firebaseUid);
let result;
try {
```

```
  result = await db.runTransaction(async (tx) => {
    const userDoc = await tx.get(userRef);
    const balance = userDoc.exists ? userDoc.data().points || 0 : 0;
    if (balance < POINTS_TO_PURCHASE) return { ok: false, balance };

    tx.set(userRef, { points: FieldValue.increment(-POINTS_TO_PURCHASE) }, { merge: true });
    tx.set(db.collection(COLLECTIONS.purchases).doc(), {
      userUid: req.user.firebaseUid, userName: req.user.name,
      bookId: req.params.bookId, bookTitle: book.title,
      bookAuthor: book.author, bookCover: book.coverImage,
      pointsSpent: POINTS_TO_PURCHASE,
      shipping: { name, phone, address },
      createdAt: FieldValue.serverTimestamp(),
    });
    return { ok: true, balance: balance - POINTS_TO_PURCHASE };
  });
} catch (err) {
  await releaseClaim(book._id);     ← COMPENSATE on exception
  throw err;
}

// ── Phase 4: COMPENSATE or COMMIT ──────────────────────────
if (!result.ok) {
  await releaseClaim(book._id);     ← COMPENSATE on insufficient funds
  return res.status(400).json({
    message: `Not enough points. This book costs ${POINTS_TO_PURCHASE}, you have ${result.balance}.` });
}

// Paid for — the copy leaves the library for good. Snapshot it onto the
// records that outlive it before removing the document.
await preserveReferences(book);
await Book.findByIdAndDelete(book._id);

res.status(201).json({ message: "Book purchased", pointsSpent: POINTS_TO_PURCHASE, points: result.balance });
```

NAME THIS PATTERN IN AN INTERVIEW

This is a **saga with a compensating transaction** — the standard answer to "how do you keep two databases consistent without a distributed transaction?". Claim the scarce resource atomically, perform the payment transactionally, and undo the claim if the payment fails. It is exactly how real payment systems reserve inventory.

Why each phase is built that way

PHASE	MECHANISM	WHAT BREAKS WITHOUT IT
Claim	`findOneAndUpdate({ _id, available: true })`	Two buyers both pass an `if` check and both pay 200 points for one physical copy
Pay	`runTransaction` re-reading the balance inside	Two concurrent purchases both read "200" and both deduct, taking the balance to −200
Compensate	`releaseClaim()` in both failure paths	A failed purchase strands the book as permanently unavailable — sold but never paid for
Preserve	`preserveReferences()` before delete	Borrow history renders "Untitled"; reviews render "Unknown book"; wishlists hold dead links

`preserveReferences` — cleaning up dangling references

```
async function preserveReferences(book) {
  const snapshot = { title: book.title, author: book.author, coverImage: book.coverImage };

  // ① Borrow records: only fill snapshots that are missing
  await Borrow.updateMany(
    { bookId: book._id, "bookSnapshot.title": { $exists: false } },
    { $set: { bookSnapshot: snapshot } }
```

```
  );

  const bookId = String(book._id);

  // ② Reviews: batch-update every review of this book
  const reviews = await db.collection(COLLECTIONS.reviews)
    .where("source", "==", "official").where("bookId", "==", bookId).get();
  const reviewBatch = db.batch();
  reviews.docs.forEach((d) => reviewBatch.update(d.ref, {
    bookTitle: snapshot.title, bookAuthor: snapshot.author, bookCover: snapshot.coverImage,
  }));
  await reviewBatch.commit();

  // ③ Wishlists: the book can never be borrowed or bought again, so leaving
  // it on anyone's wishlist would only produce a dead link.
  const wishes = await db.collection(COLLECTIONS.wishlist)
    .where("source", "==", "official").where("bookId", "==", bookId).get();
  const wishBatch = db.batch();
  wishes.docs.forEach((d) => wishBatch.delete(d.ref));
  await wishBatch.commit();
}
```

Three different responses to the same event, each correct for its context: borrows and reviews are **historical records** that must survive, so they are snapshotted; a wishlist is a **forward-looking intention** that is now impossible, so it is deleted. `String(book._id)` is required because Firestore stores the id as a string while Mongoose holds an ObjectId — comparing them without conversion silently matches nothing.

Note this function touches **both databases** and is deliberately run *before* the delete. Reversing the order would lose the data it needs to copy.

11.6 A worked numerical example

UDAY'S LEDGER

ACTION	OPERATION	BALANCE	RESULT
Registers	—	**0**	No `users` doc exists yet; `/points/me` returns 0
Shares "Atomic Habits"	`increment(+50)`	**50**	Document created via `merge: true`
Shares "Deep Work"	`increment(+50)`	**100**	
Tries to buy "1984"	transaction reads 100	**100**	**400** "Not enough points. This book costs 200, you have 100." Claim released — the book stays available.
Shares "Sapiens"	`increment(+50)`	**150**	
Shares "Educated"	`increment(+50)`	**200**	Now exactly affordable
Buys "1984"	`increment(-200)`	**0**	**201** Purchase doc written; snapshots preserved; the Book document is **deleted**; Uday keeps the book forever

Note the boundary condition: `balance < POINTS_TO_PURCHASE` uses strictly-less-than, so exactly 200 points **is** enough. Off-by-one bugs in currency checks are a classic interview probe.

11.7 Purchase history

```
const snap = await db.collection(COLLECTIONS.purchases)
  .where("userUid", "==", req.user.firebaseUid).get();
```

```
const purchases = snap.docs.map((d) => ({ id: d.id, ...d.data() }))
  .sort((a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0));
```

Because the Book document is gone, the purchase record is the **only** surviving evidence of the title — which is exactly why it stores `bookTitle`, `bookAuthor` and `bookCover` rather than just `bookId`. MyLibrary's "Purchased" tab renders straight from these fields. The same documents are counted by `/api/stats` to report how many books have been sold.

> **WHY COMMUNITY BOOKS CANNOT BE BOUGHT**
>
> The source comment is explicit: "*Community books are borrow-only and deliberately have no equivalent route.*" A used book belongs to a member, not to the library — the platform has no right to sell it. The UI states this too: "Community books are borrow-only — they can't be purchased with points." A rule enforced in the routing table, not just the interface.

SECTION TWELVE

Reviews and Ratings

One review system serving two shelves whose ids live in different namespaces.

12.1 The `source` discriminator

backend/controllers/reviewController.js

```
// Reviews cover both shelves, so a review is keyed by the book's id AND which
// collection it came from — an official Mongo _id and a Firestore usedBook id
// live in different id spaces and could otherwise collide.
const SOURCES = ["official", "community"];
```

A MongoDB ObjectId (`507f1f77bcf86cd799439011`) and a Firestore auto-id (`a7Kd9xPqR2mNb4Lz`) are drawn from entirely separate id spaces. Without `source` , an id would be ambiguous. Every review therefore carries a **composite key**: `(source, bookId)` . This shows up in the endpoint shape itself — `/api/reviews/:source/:bookId` — and in the wishlist for the same reason.

`SOURCES.includes(source)` is validated on both the read and write endpoints; an unknown source is a `400` . That is a whitelist, which is always safer than a blacklist.

12.2 The review document

```
reviews/{autoId}
{
  source:   "official" | "community",
  bookId:   "Mongo _id string OR Firestore doc id",
  userUid:  "firebase uid",
  userName: "Uday Barua",
  rating:   5,
  text:     "Completely changed how I think about habits.",
  createdAt: serverTimestamp,

  // snapshot — only for official books
  bookTitle?, bookAuthor?, bookCover?
}
```

12.3 Posting a review

```
const rating = Number(req.body.rating);
const text = (req.body.text || "").trim();
if (!rating || rating < 1 || rating > 5)
  return res.status(400).json({ message: "Rating must be between 1 and 5" });

// One review per person per book, so a single user can't skew a rating.
const existing = await db.collection(COLLECTIONS.reviews)
  .where("source",  "==", source)
  .where("bookId",  "==", bookId)
  .where("userUid", "==", req.user.firebaseUid)
  .get();
if (!existing.empty)
  return res.status(400).json({ message: "You have already reviewed this book" });

// Snapshot the book alongside the review. An official book is deleted when
// someone buys it with points, and the reader's own review should still say
// what it was about afterwards.
let snapshot = {};
```

```
if (source === "official") {
  const b = await Book.findById(bookId).select("title author coverImage");
  if (b) snapshot = { bookTitle: b.title, bookAuthor: b.author, bookCover: b.coverImage };
}

const review = { source, bookId, userUid: req.user.firebaseUid, userName: req.user.name,
                 rating, text, ...snapshot, createdAt: FieldValue.serverTimestamp() };
const ref = await db.collection(COLLECTIONS.reviews).add(review);
```

Notes:

- `!rating` catches `NaN`, `0`, `null` and `undefined` in one check, because all are falsy. Combined with the range test, only 1–5 survives.
- Review text is **optional** — a bare star rating is valid. The textarea says "(optional)".
- The duplicate check is the answer to *"how do you prevent duplicate reviews?"*: a three-field equality query before insert.
- `.select("title author coverImage")` is a **projection** — fetch only those three fields, not the whole document including the long description.
- Only official books are snapshotted, because only they can be deleted. Community books are removed by their owner, and that case is handled at read time instead.

> **HONEST LIMITATION**
>
> The duplicate check is **read-then-write**, not transactional. Two simultaneous submissions could both find nothing and both insert. In practice a user cannot double-submit (the button disables via `submitting`), but the correct fix is a deterministic document id like `` `${source}_${bookId}_${userUid}` `` with `create()`, which makes the database itself reject the second one.

12.4 Reading reviews and computing the average

```
const snap = await db.collection(COLLECTIONS.reviews)
  .where("source", "==", source).where("bookId", "==", bookId).get();

const reviews = snap.docs.map((d) => ({ id: d.id, ...d.data() }))
  .sort((a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0));

const average = reviews.length
  ? reviews.reduce((sum, r) => sum + (r.rating || 0), 0) / reviews.length
  : 0;

res.json({ reviews, average: Number(average.toFixed(2)), count: reviews.length });
```

The average is computed **on demand in JavaScript** with `reduce`, not stored on the book. Trade-off: always correct and never needs maintenance, but recomputed on every page view. Correct at this scale; at scale you would keep a running total and count on the book document and update both inside a transaction.

`reviews.length ? … : 0` guards against division by zero. `Number(average.toFixed(2))` rounds to two decimals and converts back to a number — `toFixed` returns a *string*, and returning `"4.33"` instead of `4.33` would break the client's `data.average.toFixed(1)` call.

> **TWO INDEPENDENT RATING SYSTEMS — BE PRECISE ABOUT THIS**
>
> `Book.rating` in MongoDB (shown on cards, used for "Highest Rated" sorting) is a **seeded/admin-set** value. The review average from Firestore is shown only in the `ReviewSection`. They are never reconciled — posting a 1-star review does not lower the book's card rating. An interviewer may well spot this; the answer is that it's a known simplification and the fix is to recompute `Book.rating` from the review average, or drop the stored field entirely.

12.5 The review UI

`ReviewSection` takes `source` and `bookId` as props and is used unchanged by both `BookDetails` (`source="official"`) and `CommunityBookDetails` (`source="community"`) — one component, two shelves.

```
const [rating, setRating] = useState(0);
const [hovered, setHovered] = useState(0);

{[1,2,3,4,5].map((n) => (
  <button key={n} type="button"
          onClick={() => setRating(n)}
          onMouseEnter={() => setHovered(n)}
          onMouseLeave={() => setHovered(0)}>
    <FiStar className={n <= (hovered || rating) ? "fill-coral-400 text-coral-400" : "text-navy-200"} />
  </button>
))}
```

`hovered || rating` is the whole star-picker in one expression: while hovering, preview the hovered value; otherwise show the committed one. `hovered` resets to `0` (falsy) on mouse-out, so the fallback takes over. `type="button"` is essential — inside a `<form>`, a button defaults to `type="submit"`, so clicking a star would submit the form.

After a successful post the component resets local state and calls `load()` to re-fetch, so the new review, the new count and the new average all arrive from the server rather than being guessed locally.

12.6 My Reviews — resolving two id spaces efficiently

```
const officialIds  = [...new Set(reviews.filter((r) => r.source === "official").map((r) => r.bookId))];
const communityIds = [...new Set(reviews.filter((r) => r.source === "community").map((r) => r.bookId))];

const officialBooks = officialIds.length
  ? await Book.find({ _id: { $in: officialIds } }).select("title coverImage author")
  : [];
const officialMap = Object.fromEntries(officialBooks.map((b) => [String(b._id), b]));

const communityMap = {};
await Promise.all(communityIds.map(async (id) => {
  const doc = await db.collection(COLLECTIONS.usedBooks).doc(id).get();
  if (doc.exists) communityMap[id] = doc.data();
}));

const enriched = reviews.map((r) => {
  const book = r.source === "official" ? officialMap[r.bookId] : communityMap[r.bookId];
  // Live book first so edits show through, then the snapshot taken when the
  // review was written — which is all that's left once a book is sold.
  return {
    ...r,
    bookTitle:  book?.title      || r.bookTitle  || "Unknown book",
    bookAuthor: book?.author     || r.bookAuthor || "",
    bookCover:  book?.coverImage || r.bookCover  || "",
    bookExists: Boolean(book),
  };
});
```

This function is a small masterclass — four techniques worth knowing:

1. `new Set` **de-duplication.** Ten reviews of three books produce three lookups, not ten.
2. `$in` **batching.** One MongoDB query fetches every official book, instead of N round trips. The source comment says it: "*so N reviews don't fan out into N round-trips per store.*" This is the classic **N+1 query problem**, solved.
3. **A lookup map.** `Object.fromEntries` converts the array into an id-keyed object so enrichment is O(1) per review instead of a nested `find`.
4. **Three-level fallback.** Live book (so edits appear) → snapshot (survives deletion) → "Unknown book". `bookExists` then tells the UI whether to render a link, since a deleted book's page is gone. `MyReviews.jsx` greyscales the cover and adds "No longer in the

library".

Firestore has no `$in`-by-document-id equivalent as convenient as Mongo's, so community books are fetched with `Promise.all` over individual `.doc(id).get()` calls — still concurrent, still one round of latency rather than N sequential ones.

SECTION THIRTEEN

Wishlist

A small feature that demonstrates idempotency, denormalisation and optimistic UI.

13.1 The document

```
wishlist/{autoId}
{
  userUid:    "firebase uid",
  source:     "official" | "community",
  bookId:     "...",
  title:      "The Great Gatsby",     ← denormalised
  author:     "F. Scott Fitzgerald",  ← denormalised
  coverImage: "https://...",          ← denormalised
  createdAt:  serverTimestamp
}
```

backend/controllers/wishlistController.js

```
// A wishlist item stores who saved it, which shelf the book is on, its id, and
// a denormalized copy of the display fields so the wishlist page renders in one
// read without resolving two different id spaces. Firestore, per the project's
// "new features use Firestore" rule.
```

That comment states both the technique and the convention. Without denormalisation, rendering a wishlist of 10 mixed books would need a Mongo query *and* a set of Firestore reads before anything could be displayed. With it, one query returns everything the page needs. The trade-off is staleness — if an admin renames a book, the wishlist keeps the old title. For a wishlist that is entirely acceptable.

13.2 Adding — idempotent by design

```
// POST /api/wishlist — save a book. Idempotent: saving the same book twice
// returns the existing entry instead of creating a duplicate.
const existing = await db.collection(COLLECTIONS.wishlist)
  .where("userUid", "==", req.user.firebaseUid)
  .where("source",  "==", source)
  .where("bookId",  "==", bookId).get();
if (!existing.empty) {
  const doc = existing.docs[0];
  return res.json({ id: doc.id, ...doc.data() });   200, not 201
}
... otherwise create and return 201 ...
```

Idempotent means calling it repeatedly has the same effect as calling it once. Note the status codes carry meaning: `201 Created` for a genuinely new entry, `200 OK` for "already there, here it is". The client can treat both as success without caring which happened — which is exactly what makes a double-click harmless.

13.3 Removing — by book, not by document id

```
// DELETE /api/wishlist/:source/:bookId — remove by the book it points at, so
// the client can toggle without tracking the wishlist document id.
const snap = await db.collection(COLLECTIONS.wishlist)
  .where("userUid", "==", req.user.firebaseUid)
  .where("source",  "==", source)
  .where("bookId",  "==", bookId).get();
```

```
const batch = db.batch();
snap.docs.forEach((d) => batch.delete(d.ref));
await batch.commit();

res.json({ message: "Removed", count: snap.size });
```

This is a genuinely good API design decision. A heart button on a book card knows the **book** it represents; it has no idea what the wishlist *document* id is. Keying deletion on `(source, bookId)` means the toggle needs no extra state. The `userUid` filter is also the authorisation — you can only ever delete your own entries, because the query cannot match anyone else's. Deleting the whole result set (rather than just the first) means any duplicates from a past bug are cleaned up too.

13.4 The UI

`WishlistButton` takes a `book` object carrying `{ source, bookId, title, author, coverImage }` and a `variant` (`"pill"` for detail pages, `"icon"` for card corners).

```
const handleClick = (e) => {
  e.preventDefault();
  e.stopPropagation();
  if (!firebaseUser) { navigate("/login"); return; }
  toggle(book);
};
```

`preventDefault` + `stopPropagation` are essential: the heart sits **inside a** `<Link>` on card corners, so without them clicking it would navigate to the book instead of toggling. Signed-out users are sent to login rather than silently failing.

The Wishlist page renders both sources in one grid with a badge ("Official" indigo / "Community" coral) and computes the destination per item:

```
const linkFor = (i) => (i.source === "community" ? `/community/${i.bookId}` : `/books/${i.bookId}`);
```

The React key is `` `${item.source}-${item.bookId}` `` — a composite key again, for exactly the same reason the database uses one: the two id spaces could otherwise collide.

13.5 Flow — Uday wishlists a book

1. Uday taps the heart on a Collection card.
   components/WishlistButton.jsx
2. **The heart fills immediately** — `toggle()` inserts an optimistic item with a temporary id before any request goes out.
   context/WishlistContext.jsx
3. `POST /api/wishlist` with `{ source, bookId, title, author, coverImage }`
4. No existing entry → a Firestore document is created with `userUid` from the verified token.
5. `201` with the real document. The temporary item is swapped for it by id.
6. Every `WishlistButton` on screen re-renders from the shared context, and `/wishlist` already contains the book when Uday opens it.

✗ **If step 3 failed:** `catch` logs and calls `load()`, discarding the optimistic guess and re-reading the truth from the server. The heart un-fills.

SECTION FOURTEEN

The Store

A real e-commerce catalogue in Bangladeshi Taka — filtering, sorting, and admin management.

14.1 What the Store is

Brand-new books for sale with real money, entirely separate from the lending Collection. The source comment on the create endpoint is explicit: "*Store stock is separate from the library's lending copies, so this never touches the Collection.*" The same title can appear in both — "The Great Gatsby" is borrowable from the Collection **and** purchasable in the Store — as two unrelated documents in two different databases.

14.2 The document

```
storeBooks/{autoId}
{
  title:       "Designing Data-Intensive Applications",
  author:      "Martin Kleppmann",
  description: "The ideas behind reliable, scalable and maintainable systems.",
  category:    "Technology",
  price:       1890,           ← Taka, stored as a plain number
  rating:      4.9,
  coverImage:  "https://covers.openlibrary.org/b/isbn/9781449373320-L.jpg",
  currency:    "BDT",
  createdAt:   serverTimestamp
}
```

There is **no stock quantity field**. The product page hard-codes "In stock" and the cart imposes no upper bound. A defensible simplification for a portfolio project, and worth naming as such rather than being caught by it.

`seedStore.js` populates 24 books across 8 categories at realistic Dhaka prices (৳360–৳1890), with its own comment: "*Prices are Bangladeshi Taka, in the range a real Dhaka bookshop charges for English paperbacks.*"

14.3 Categories with live counts

```
// GET /api/store/categories — drives the "Shop by category" filter, so the list
// always reflects what is actually for sale.
const snap = await db.collection(COLLECTIONS.storeBooks).get();
const counts = {};
snap.docs.forEach((d) => {
  const c = d.data().category;
  if (c) counts[c] = (counts[c] || 0) + 1;
});
const categories = Object.entries(counts)
  .map(([name, count]) => ({ name, count }))
  .sort((a, b) => a.name.localeCompare(b.name));
res.json({ categories, total: snap.size });
```

The sidebar shows "Fiction 5", "Technology 3" and so on, computed from the actual catalogue. Compare this with the Collection and Community pages, which use a **hard-coded** array of eight category names — so an empty category still appears there. The Store's approach is better; the difference is a good thing to notice about your own code.

`(counts[c] || 0) + 1` is the standard tally idiom, and `localeCompare` gives correct alphabetical ordering rather than raw code-point comparison.

14.4 Filtering and sorting — in JavaScript, and why

```
// GET /api/store/books
// Filters: category, minRating (4 => "4 stars & up"), search
// Sort: price-asc | price-desc | rating | newest
const { category, minRating, search, sort } = req.query;

const snap = await db.collection(COLLECTIONS.storeBooks).get();
let books = snap.docs.map((d) => ({ id: d.id, ...d.data() }));

// The catalogue is small and Firestore can't combine a range filter with
// arbitrary ordering without composite indexes, so filtering happens here.
if (category && category !== "All") books = books.filter((b) => b.category === category);
if (minRating) { const min = Number(minRating); books = books.filter((b) => (b.rating || 0) >= min); }
if (search) {
  const q = search.toLowerCase();
  books = books.filter((b) => b.title?.toLowerCase().includes(q) || b.author?.toLowerCase().includes(q));
}

const sorters = {
  "price-asc":  (a, b) => a.price - b.price,
  "price-desc": (a, b) => b.price - a.price,
  rating:       (a, b) => (b.rating || 0) - (a.rating || 0),
  newest:       (a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0),
};
books.sort(sorters[sort] || sorters.newest);
res.json(books);
```

ANSWER TO "WHERE DOES STORE FILTERING HAPPEN, AND WHY NOT IN THE DATABASE?"

In Node.js memory on the server. The reason is a genuine Firestore constraint: a query that combines a **range filter** (`rating >= 4`) with an **ordering on a different field** (`price`) requires a composite index — and Firestore additionally requires that the first `orderBy` match the range field. Supporting every combination of category × rating × sort would mean maintaining a matrix of composite indexes. With a 24-book catalogue, one read plus in-memory filtering is simpler, faster to develop, and behaves identically from the client's point of view. At scale the answer changes — you would add `firestore.indexes.json` , or move the catalogue to a search service.

The `sorters` object is a clean **strategy map**: it turns four branches into one lookup, and `sorters[sort] || sorters.newest` gives a safe default for an unknown or missing value. Also note the search here covers **title OR author in one box** — unlike the Collection, which has two separate inputs.

14.5 The Store UI

`Store.jsx` uses the same URL-as-state pattern with four parameters (`search` , `category` , `minRating` , `sort` , default `price-asc`). Two effects: one loads categories **once** (`[]`), one reloads books whenever a filter changes.

Details worth noticing:

- **The sidebar is a variable, not a component** — `const Sidebar = (<div>…)` is a JSX element assigned once and rendered twice (desktop sticky column, mobile collapsible panel). Zero duplication.
- **The rating filter toggles.** `updateParam("minRating", String(minRating) === String(r) ? "" : r)` — clicking the active rating clears it. The `String()` coercion on both sides matters because URL params are always strings while `r` is a number.
- **"Clear all filters" preserves sort** — `setSearchParams({ sort })` . Clearing filters is not the same as resetting your chosen ordering.
- Only two sort options are exposed in the UI (price low→high, high→low) even though the API also supports `rating` and `newest` . The comment says why: *"the two options a bookshop actually needs."*

14.6 Currency formatting

frontend/src/utils/currency.js

```
// Every Store price is Bangladeshi Taka, formatted in one place so the symbol
// and grouping never drift between the grid, the cart and the order summary.
export const taka = (amount) =>
  `৳${Number(amount || 0).toLocaleString("en-BD", { maximumFractionDigits: 0 })}`;
```

`toLocaleString("en-BD")` applies locale-correct digit grouping; whole Taka only, since poisha are not used in practice. `Number(amount || 0)` means a missing price renders "৳0" instead of "৳NaN". Used in four places, defined in one.

14.7 Admin Store management

```
const REQUIRED_FIELDS = ["title", "author", "category", "price", "coverImage"];

function validateBody(body) {
  for (const f of REQUIRED_FIELDS) {
    if (body[f] === undefined || body[f] === null || body[f] === "") return `${f} is required`;
  }
  if (Number.isNaN(Number(body.price)) || Number(body.price) <= 0) return "price must be a positive number";
  return null;
}
```

The explicit `undefined / null / ""` comparison rather than a falsy check is deliberate — a plain `if (!body[f])` would reject a legitimate price of `0`. (Price 0 is rejected anyway by the next rule, but the pattern is correct and would matter for other fields.) Returning a **message or null** lets the caller write `if (err) return 400`.

Update merges old and new before validating, so a partial edit does not fail on untouched fields:

```
const err = validateBody({ ...doc.data(), ...req.body });
```

All three write endpoints are guarded by `verifyFirebaseToken, requireRole("admin")`. `AdminStoreTab.jsx` provides the table, a create/edit modal, delete confirmation, and computed stat cards (average price, average rating) — all through `storeBooksAdmin.create / update / remove` in `adminService.js`.

SECTION FIFTEEN

Cart System

A server-owned cart that stores almost nothing — and why that is the whole point.

15.1 The design decision

backend/controllers/cartController.js

```
// The cart stores only ids and quantities. Prices and titles are re-read from
// the catalogue on every load, so a stale cart can never carry an old price and
// a client can never suggest one.
```

```
carts/{firebaseUid}
{
  userUid: "firebase uid",
  items: [
    { bookId: "abc123", quantity: 2 },
    { bookId: "def456", quantity: 1 }
  ],
  updatedAt: serverTimestamp
}
```

That is the **entire** stored cart. No title, no price, no subtotal. Two consequences follow, and both are important:

- **Stale prices are impossible.** Add a book at ৳420, leave the tab open for a week, and an admin changes the price to ৳480 — you are charged ৳480, because the price is read fresh on every single cart operation.
- **Client price manipulation is impossible.** There is no price field in any request body, so there is nothing to tamper with. The API has no vocabulary for a client-supplied price.

The document id is the user's Firebase UID, so a cart lookup is a direct document read and the security rule is `request.auth.uid == userId`. It also means one cart per user by construction — duplicates cannot exist.

15.2 `buildCart()` — the heart of the system

```
const cartRef = (uid) => db.collection(COLLECTIONS.carts).doc(uid);

async function buildCart(uid) {
  const doc = await cartRef(uid).get();
  const raw = doc.exists ? doc.data().items || [] : [];
  if (raw.length === 0) return { items: [], subtotal: 0, count: 0 };

  const books = await Promise.all(
    raw.map(async (i) => {
      const b = await db.collection(COLLECTIONS.storeBooks).doc(i.bookId).get();
      return b.exists ? { id: b.id, ...b.data() } : null;
    })
  );

  const items = [];
  raw.forEach((i, idx) => {
    const book = books[idx];
    if (!book) return;              dropped from the catalogue since it was added
    items.push({
      bookId: book.id, title: book.title, author: book.author,
      coverImage: book.coverImage, category: book.category,
      price: book.price,                          ← from the CATALOGUE
```

```
      quantity: i.quantity,                    ← from the CART
      lineTotal: book.price * i.quantity,      ← computed server-side
    });
  });

  const subtotal = items.reduce((sum, i) => sum + i.lineTotal, 0);
  const count    = items.reduce((sum, i) => sum + i.quantity, 0);
  return { items, subtotal, count };
}
```

Five things to be able to explain:

1. **Early return for an empty cart** avoids doing any work — and avoids `Promise.all([])` ceremony.
2. `Promise.all` **over the items** fetches every book **concurrently**. Three items cost one round-trip of latency, not three sequential ones.
3. **Deleted books are dropped silently.** If an admin removes a product from the Store while it sits in someone's cart, `b.exists` is false and the line simply disappears rather than crashing checkout with `undefined.price`. Graceful degradation.
4. **Every money value is computed here.** `lineTotal` and `subtotal` come from catalogue prices, server-side. The client displays them; it never calculates them.
5. `count` **is total units, not distinct titles** — 2 × Gatsby + 1 × 1984 is a badge of **3**. That is what a shopper expects.

Crucially, `buildCart` is also imported by `checkoutController`, so **the cart you see and the cart Stripe charges are produced by the same function**. They cannot disagree.

15.3 Adding to the cart

```
const { bookId } = req.body;
const quantity = Math.max(1, Number(req.body.quantity) || 1);
if (!bookId) return res.status(400).json({ message: "bookId is required" });

const book = await db.collection(COLLECTIONS.storeBooks).doc(bookId).get();
if (!book.exists) return res.status(404).json({ message: "Book not found in the Store" });

const ref = cartRef(req.user.firebaseUid);
const doc = await ref.get();
const items = doc.exists ? doc.data().items || [] : [];

const existing = items.find((i) => i.bookId === bookId);
if (existing) existing.quantity += quantity;    already there → bump it
else items.push({ bookId, quantity });          new line

await ref.set({ userUid: req.user.firebaseUid, items, updatedAt: FieldValue.serverTimestamp() },
              { merge: true });

res.status(201).json(await buildCart(req.user.firebaseUid));
```

`Math.max(1, Number(q) || 1)` is a compact triple-guard: `Number("abc")` is `NaN` → falsy → becomes 1; `0` → falsy → becomes 1; `-5` → survives `||` but is clamped by `Math.max`. **Adding a negative quantity to reduce a total is impossible.**

Duplicate handling: adding a book already in the cart increases its quantity rather than creating a second line — which is why `items` can never contain the same `bookId` twice.

The book's existence is verified **before** anything is written, so a bogus id can never enter the cart.

15.4 Quantity, removal, clearing

```
// PATCH /api/cart/:bookId — set an exact quantity; 0 removes the line
const quantity = Number(req.body.quantity);
if (Number.isNaN(quantity) || quantity < 0)
  return res.status(400).json({ message: "quantity must be 0 or more" });
```

```
items = quantity === 0
  ? items.filter((i) => i.bookId !== req.params.bookId)
  : items.map((i) => (i.bookId === req.params.bookId ? { ...i, quantity } : i));
```

`PATCH` **sets an absolute quantity;** `POST` **adds a relative amount.** That distinction is why the cart page's +/− buttons send `PATCH` with `item.quantity ± 1` — the server is told the new value, not a delta, so a double-click cannot compound.

`{ ...i, quantity }` creates a **new** object rather than mutating in place — immutable update style, consistent with React conventions even though this is server code.

The UI decrement button is disabled at `quantity <= 1`, so removal happens through the explicit Remove button rather than by decrementing to zero — though the API supports both. `clearCart` writes `items: []` with `merge: true`, keeping the document (and its `userUid`) rather than deleting it.

15.5 Cart persistence

The cart lives in **Firestore, keyed by user** — not in `localStorage`, not in React state. It therefore survives page refreshes, browser restarts, and device changes: sign in on your phone and your cart is there. The cost is that a signed-out visitor has **no cart at all**; "Add to cart" redirects to login. A guest cart would require a local fallback and a merge-on-login step, neither of which exists here.

15.6 Complete flow — "User adds one book to cart"

1. Uday clicks "Add to cart" on the ৳1890 book.
   `pages/StoreBookDetails.jsx → handleAdd`
2. `if (!firebaseUser) navigate("/login", { state: { from: `/store/${id}` } })` — signed out users are sent to login and returned here afterwards.
3. `setAdding(true)` → the button reads "Adding…" and is disabled, preventing a double submit.
4. `await add(id)` → `cartApi.addToCart(bookId, 1)`
   `context/CartContext.jsx → services/cartService.js`
5. Interceptor: `authStateReady()` → `getIdToken()` → `Authorization` header.
6. **Request:** `POST /api/cart`
   `{ "bookId": "a7Kd9xPqR2mNb4Lz", "quantity": 1 }`
   Note: **no price field exists in this payload.**
7. `router.use(verifyFirebaseToken)` guards the whole cart router → `req.user` is set.
   `routes/cart.js`
8. **Firestore read:** the book exists in `storeBooks`. **Firestore read:** the current cart. Not present → `items = []`.
9. **Firestore write:** `carts/{uid}` set with `items: [{ bookId, quantity: 1 }]`, merged.
10. `buildCart()` re-reads the book and computes `price: 1890`, `lineTotal: 1890`, `subtotal: 1890`, `count: 1`.
11. **Response 201:** the complete cart object.
12. `setCart(response)` in the context — one state update.
13. **Uday sees:** the button turns green "Added to cart" with a check icon, a "Go to cart" link appears, and the **navbar cart badge shows 1** — because `Navbar` consumes the same context.

SECTION SIXTEEN

Stripe Payment System

From clicking Checkout to a stored order — every file, every guard, and exactly what each party sends and receives.

16.1 The Stripe client

backend/lib/stripe.js — the entire file

```
const Stripe = require("stripe");

// Prices throughout the Store are Bangladeshi Taka. Stripe takes amounts in the
// currency's smallest unit, and BDT has two decimals, so ৳450 is sent as 45000.
const CURRENCY = "bdt";
const toStripeAmount = (taka) => Math.round(Number(taka) * 100);

const key = process.env.STRIPE_SECRET_KEY;
const stripe = key ? new Stripe(key) : null;

if (!stripe) {
  console.warn("STRIPE_SECRET_KEY is missing — the Store will load but checkout is disabled until it is set.");
}

module.exports = { stripe, CURRENCY, toStripeAmount };
```

- **The secret key never leaves the server.** It is read from `process.env` and is not prefixed `VITE_`, so it can never reach the bundle. A leaked secret key would let an attacker issue refunds and read every customer record.
- **Conditional initialisation, not lazy initialisation.** The client is created once at module load if a key exists, otherwise `null`. This is a graceful-degradation guard, not lazy-loading — worth being precise about.
- **The currency conversion is the classic Stripe gotcha.** Stripe takes amounts in the smallest unit. BDT is a two-decimal currency, so ৳450 → `45000` poisha. `Math.round` guards against floating-point drift — `4.35 * 100 === 434.99999999999994` in IEEE-754, and Stripe rejects non-integers.

> **THERE IS DELIBERATELY NO STRIPE BROWSER SDK**
>
> `@stripe/stripe-js` was installed and then **removed** in commit `aef233a`: "*Checkout is Stripe-hosted: the server creates the session and the browser is redirected to session.url, which needs no publishable key and no client SDK.*" Bookverse never renders a card field, so it never handles card data, so its PCI scope is minimal. See §28.11.

16.2 Creating the Checkout Session

backend/controllers/checkoutController.js

```
async function createCheckoutSession(req, res, next) {
  try {
    if (!stripe) return res.status(503).json({
      message: "Payments aren't configured yet. STRIPE_SECRET_KEY is missing." });

    // Rebuilt from the catalogue, so every amount charged comes from our own
    // data — the client never gets to say what anything costs.
    const cart = await buildCart(req.user.firebaseUid);
    if (cart.items.length === 0) return res.status(400).json({ message: "Your cart is empty" });

    const session = await stripe.checkout.sessions.create({
      mode: "payment",
      customer_email: req.user.email,
      line_items: cart.items.map((i) => ({
```

```
        price_data: {
          currency: CURRENCY,
          product_data: {
            name: i.title,
            description: `by ${i.author}`,
            images: i.coverImage ? [i.coverImage] : undefined,
          },
          unit_amount: toStripeAmount(i.price),
        },
        quantity: i.quantity,
      })),
      // The uid is read back from the session on return, so a returning browser
      // can't claim someone else's order.
      metadata: { userUid: req.user.firebaseUid, itemCount: String(cart.count) },
      // Both outcomes return to the cart, which confirms the payment and
      // switches itself into a receipt rather than bouncing through a
      // separate page.
      success_url: `${CLIENT_URL}/cart?session_id={CHECKOUT_SESSION_ID}`,
      cancel_url:  `${CLIENT_URL}/cart?canceled=1`,
    });

    res.json({ url: session.url, sessionId: session.id });
  } catch (err) { next(err); }
}
```

PARAMETER	WHY IT IS SET THIS WAY
`mode: "payment"`	A one-time charge, as opposed to `"subscription"` or `"setup"`
`customer_email`	Taken from `req.user.email` — the **verified** account email, so the receipt cannot be redirected to an attacker's inbox
`line_items[].price_data`	Prices are defined **inline** rather than referencing pre-created Stripe Price objects, so the Store's own catalogue stays the single source of truth
`unit_amount`	`toStripeAmount(i.price)` where `i.price` came from `buildCart` — i.e. from Firestore, never from the request
`images`	Conditional — `undefined` when there is no cover, because Stripe rejects an array containing an empty string
`metadata.userUid`	The security linchpin: it binds this session to one verified account, and is checked again on return
`success_url`	`{CHECKOUT_SESSION_ID}` is a **Stripe placeholder**, substituted by Stripe at redirect time — it is not JavaScript interpolation
`cancel_url`	Returns to the same cart with a flag, so the shopper's items are still waiting

The response is deliberately minimal — just `{ url, sessionId }`. The browser needs nothing else.

16.3 Handing off to Stripe

frontend/src/pages/Cart.jsx

```
const handleCheckout = async () => {
  setCheckingOut(true); setError("");
  try {
    const { url } = await startCheckout();
    // Stripe hosts the payment page, so we hand the browser over to it.
    window.location.href = url;
  } catch (err) {
    setError(err.response?.data?.message || "Could not start checkout. Please try again.");
    setCheckingOut(false);
  }
};
```

`window.location.href` — a full browser navigation, deliberately **not** React Router, because the destination is a different origin (`checkout.stripe.com`). Note `setCheckingOut(false)` appears only in the `catch`: on success the page is leaving, so resetting the

button would just flash it.

16.4 Confirming the payment

```
async function confirmCheckout(req, res, next) {
  try {
    if (!stripe) return res.status(503).json({ message: "Payments aren't configured yet." });

    const session = await stripe.checkout.sessions.retrieve(req.params.sessionId, {
      expand: ["line_items"],
    });

    // ① OWNERSHIP
    if (session.metadata?.userUid !== req.user.firebaseUid)
      return res.status(403).json({ message: "This checkout doesn't belong to you." });

    // ② PAYMENT ACTUALLY COMPLETED
    if (session.payment_status !== "paid")
      return res.status(400).json({ message: "This payment hasn't completed." });

    // ③ IDEMPOTENCY — Reloading the success page must not create a second order.
    const existing = await db.collection(COLLECTIONS.orders)
      .where("stripeSessionId", "==", session.id).get();
    if (!existing.empty) {
      const doc = existing.docs[0];
      return res.json({ id: doc.id, ...doc.data(), alreadyRecorded: true });
    }

    const items = (session.line_items?.data || []).map((li) => ({
      title: li.description,
      quantity: li.quantity,
      lineTotal: li.amount_total / 100,     back from poisha to Taka
    }));

    const order = {
      userUid: req.user.firebaseUid, userName: req.user.name,
      email: session.customer_details?.email || req.user.email,
      items,
      total: session.amount_total / 100,
      currency: (session.currency || CURRENCY).toUpperCase(),
      stripeSessionId: session.id,
      paymentStatus: session.payment_status,
      status: "paid",
      createdAt: FieldValue.serverTimestamp(),
    };
    const ref = await db.collection(COLLECTIONS.orders).add(order);

    await db.collection(COLLECTIONS.carts).doc(req.user.firebaseUid)
      .set({ items: [], updatedAt: FieldValue.serverTimestamp() }, { merge: true });

    res.status(201).json({ id: ref.id, ...order });
  } catch (err) { next(err); }
}
```

THE THREE GUARDS, IN PLAIN LANGUAGE

① **Ownership** — the session's stored `userUid` must equal the caller's verified uid, so guessing a session id gets you nothing. ② **Payment status** — read from **Stripe**, so manually visiting `/cart?session_id=…` without paying records nothing. ③ **Idempotency** — an order already exists for this session id, so a refresh returns the existing order with `alreadyRecorded: true` rather than creating a duplicate.

The order amounts are taken from **Stripe's** response (`session.amount_total`), not recomputed locally — so the stored order records what was genuinely charged. `/ 100` converts back from poisha. The cart is emptied only *after* the order is written, so a failure at that

point leaves the cart intact.

16.5 The return leg in React

```
const sessionId = searchParams.get("session_id");
const [order, setOrder] = useState(null);
const [confirming, setConfirming] = useState(Boolean(sessionId));
const confirmed = useRef(false);

useEffect(() => {
  if (!sessionId || confirmed.current) return;
  confirmed.current = true;

  confirmCheckout(sessionId)
    .then((o) => {
      setOrder(o);
      reload();                              the server emptied the cart — mirror that here
      setSearchParams({}, { replace: true }); drop the id so refresh doesn't re-confirm
    })
    .catch((err) => setError(err.response?.data?.message || "We couldn't confirm that payment."))
    .finally(() => setConfirming(false));
}, [sessionId]);
```

`useRef` **as a latch.** In React 18/19 StrictMode, effects run twice in development. A `useState` flag would not help — the second run happens before a re-render. `useRef` persists across renders and mutating it does **not** trigger one, so `confirmed.current = true` reliably blocks the duplicate call. It is defence-in-depth on top of the authoritative server-side idempotency check.

`setSearchParams({}, { replace: true })` strips `?session_id=` from the URL using `replace` so the Back button does not return to a URL that would re-trigger confirmation.

16.6 The cart becomes the receipt

There is no `/checkout/success` route — it was deliberately removed (§28.5). The cart page renders one of several states:

```
{confirming || (order && cart.items.length === 0) ? null : loading ? <Skeletons />
  : cart.items.length === 0 ? <EmptyCart /> : <CartWithSummary />}
```

That condition suppresses the "your cart is empty" panel while confirming or while a receipt is on screen — otherwise a successful payment would show a celebratory receipt directly above the words "Your cart is empty", which reads as failure.

16.7 The complete purchase — a realistic example

Uday buys *Clean Code* (৳1250 × 1) and *Atomic Habits* (৳620 × 2).

1. **Browser → API.** `POST /api/checkout/session`, `Authorization: Bearer …`, **empty body**. The client sends no items and no amounts.
2. `router.use(verifyFirebaseToken)` → token verified → `req.user` set.
3. `buildCart(uid)` reads `carts/{uid}` → `[{clean-code,1},{atomic,2}]`, then reads both from `storeBooks` → prices **1250** and **620**. Subtotal **2490**, count **3**.
4. **API → Stripe.** A session with two line items: `unit_amount: 125000` qty 1, and `unit_amount: 62000` qty 2. Metadata `{ userUid: "uday…", itemCount: "3" }`.
5. **Stripe → API.** `{ id: "cs_test_a1B2c3…", url: "https://checkout.stripe.com/c/pay/cs_test_a1B2c3…" }`
6. **API → Browser.** `{ url, sessionId }` → `window.location.href = url`.
7. **On Stripe's domain.** Uday sees both books with covers, a total of ৳**2,490.00**, and his email pre-filled. He enters card details — **on Stripe's page, never on Bookverse's**.
8. Stripe authorises and redirects to `https://…/cart?session_id=cs_test_a1B2c3…`, substituting the placeholder.
9. `ProtectedRoute` runs. If the session has not restored yet it bounces to login — **preserving** `?session_id=` via `location.search`, so the id survives (§28.5).
10. Cart mounts, reads `session_id`, sets `confirming: true` → "Confirming your payment…", fires `GET /api/checkout/confirm/cs_test_a1B2c3…`.
11. **API → Stripe.** `sessions.retrieve(id, { expand: ["line_items"] })`. Returns `payment_status: "paid"`, `amount_total: 249000`, `currency: "bdt"`, metadata intact.
12. **Three guards pass:** metadata uid matches Uday; status is "paid"; no existing order carries this session id.
13. **Firestore write:** an `orders` document — total **2490**, currency **"BDT"**, two items with `lineTotal` 1250 and 1240, `status: "paid"`.
14. **Firestore write:** `carts/{uid}` → `items: []`.
15. **API → Browser.** `201` with the full order.
16. `setOrder(o)`; `reload()` mirrors the emptied cart; `session_id` is stripped from the URL.
17. **Uday sees:** a green "Payment successful" panel with a spring-animated check, ৳**2,490**, an itemised list, and two buttons — "Continue to Store" and "View my orders". The navbar cart badge is gone.

16.8 Order history

```
const snap = await db.collection(COLLECTIONS.orders)
  .where("userUid", "==", req.user.firebaseUid).get();
const orders = snap.docs.map((d) => ({ id: d.id, ...d.data() }))
  .sort((a, b) => (b.createdAt?.toMillis?.() || 0) - (a.createdAt?.toMillis?.() || 0));
```

`Orders.jsx` shows an order number (`#{o.id.slice(0,10).toUpperCase()}`), the date, total and status badge. Its date helper handles Firestore timestamps arriving as JSON over HTTP:

```
const formatDate = (ts) => {
  const ms = ts?._seconds ? ts._seconds * 1000 : ts?.seconds ? ts.seconds * 1000 : null;
  return ms ? new Date(ms).toLocaleDateString(undefined, { dateStyle: "medium" }) : "—";
};
```

A Firestore `Timestamp` serialises as `{ _seconds, _nanoseconds }` through the Admin SDK but as `{ seconds, nanoseconds }` through the client SDK. Handling both, and falling back to an em-dash, is exactly the kind of real-world detail worth pointing out.

16.9 The webhook gap — the most important thing to volunteer

SAY THIS BEFORE YOU ARE ASKED

Orders are recorded **only** when the browser returns to `/cart` and calls `confirm`. If Uday pays and then closes the tab, loses connectivity, or his battery dies, **Stripe has his money and Bookverse has no order**. The cart is not even emptied.

The production fix is a **Stripe webhook**: an endpoint Stripe calls server-to-server on `checkout.session.completed`, independent of the browser. It must verify the `Stripe-Signature` header with `stripe.webhooks.constructEvent` (using the raw body, so `express.json()` has to be bypassed for that route) to prove the call really came from Stripe. The existing `stripeSessionId` idempotency check already means the webhook and the browser confirmation can both run safely — whichever arrives first records the order, and the second is a no-op. **The architecture is already webhook-ready; the endpoint just isn't written.**

SECTION SEVENTEEN

Shipping / Delivery Information

Where addresses are collected, where they are stored, and who is allowed to read them.

17.1 Why it is collected

Bookverse posts physical books to people's homes. Three flows need a delivery address; one conspicuously does not.

FLOW	COLLECTED?	STORED IN	WHO CAN READ IT
Borrow an official book	**Yes**	MONGO `borrows.shipping`	The library (server-side). Not exposed by any endpoint.
Buy with points	**Yes**	FIRE `purchases.shipping`	Only the buyer (rules + query scoping)
Request a community book	**Yes**	FIRE `borrowRequests.shipping`	Only the two parties involved
Store purchase (Stripe)	**No**	—	Stripe collects an email only

That last row is a genuine functional gap: a paid Store order has no delivery address. The fix is Stripe's built-in `shipping_address_collection` parameter, which would return the address on the session and could be written onto the order.

17.2 One form, three flows

`frontend/src/components/ShippingModal.jsx`

```
// The library parcels books to people's homes, so borrowing and purchasing both
// need the same delivery details. One component serves both so the two flows
// can never drift apart.
const ShippingModal = ({ open, title, subtitle, confirmLabel, submitting, error, onClose, onSubmit }) => {
  const [form, setForm] = useState({ shippingName: "", shippingPhone: "", shippingAddress: "" });
  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
  const handleSubmit = (e) => { e.preventDefault(); onSubmit(form); };
  ...
};
```

The component is fully **controlled** and fully **generic**: it owns the three fields and receives everything contextual (title, subtitle, button label, error, submit handler) as props. `BookDetails` uses it for both borrowing and buying; `CommunityBookDetails` uses it for requesting. Three flows, one implementation, zero drift.

`handleChange` uses a **computed property name** — `[e.target.name]: e.target.value` — so one handler serves all three inputs, keyed by each input's `name` attribute.

Two clicking details worth noticing: the backdrop has `onClick={onClose}` while the panel has `onClick={(e) => e.stopPropagation()}`, so clicking outside closes the modal but clicking inside does not. And every field carries the HTML `required` attribute, so the browser blocks submission before any JavaScript runs.

17.3 Server-side validation

```
// backend/controllers/communityController.js
function readShipping(body) {
  const name    = (body.shippingName    || "").trim();
  const phone   = (body.shippingPhone   || "").trim();
  const address = (body.shippingAddress || "").trim();
  if (!name || !phone || !address) return null;
  return { name, phone, address };
}
```

The identical three lines appear in `routes/books.js` (borrow) and `pointsController.js` (purchase) — validated **three times**, once per entry point, because HTML `required` is trivially bypassed with a direct API call.

`(x || "").trim()` does two jobs: it prevents `TypeError` when the field is absent, and it makes whitespace-only input count as empty. Returning `null` as a sentinel lets the caller write `if (!shipping) return 400`.

17.4 Storage and privacy

backend/models/Borrow.js

```
// Where the library parcels the book. Stored on the borrow record itself
// rather than a new collection, so a delivery address is tied to the one
// loan it was given for instead of lingering on the account.
shipping: { name: String, phone: String, address: String },
```

That is a deliberate **data-minimisation** decision, and a strong thing to be able to articulate: the address is scoped to the transaction that needed it, not accumulated on the user profile. A user can send different books to different addresses, and no address becomes a permanent account attribute.

Who can read a shipping address

```
// firestore.rules
match /borrowRequests/{requestId} {
  // A borrow request carries a home address, so only the two people involved
  // may read it.
  allow read: if signedIn() &&
    (request.auth.uid == resource.data.ownerUid ||
     request.auth.uid == resource.data.requesterUid);
  allow write: if false;
}

match /purchases/{purchaseId} {
  // Purchases include a delivery address — owner only.
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
  allow write: if false;
}
```

Notice the rule for `borrowRequests` is the **only** two-party read rule in the file, and it exists precisely because the owner legitimately needs the borrower's address to post the book — but nobody else does.

And the UI reveals it only after approval

frontend/src/pages/MyLibrary.jsx

```
{r.status === "approved" && (
  <p>Deliver to: {r.shipping?.name}, {r.shipping?.address} (Phone: {r.shipping?.phone})</p>
)}
```

A **pending** request does not display the address to the owner. They see who is asking and for what, and only once they agree to lend does the address appear. That is a thoughtful privacy detail — the address is shown at the moment it becomes necessary, not before.

> **HONEST CAVEAT**
>
> This last protection is **client-side only**. The address *is* present in the API response for pending requests, so a determined owner could read it from the network tab. Proper enforcement would strip `shipping` server-side until `status === "approved"`. Naming this yourself shows you understand the difference between hiding data and protecting it — the same distinction as §18.6.

17.5 Why phone and not an ID number

An earlier version collected an ID card/passport number. Commit `5a0e3b4` replaced it: *"An ID card/passport number was confusing to test and unclear where a real user would get one for a demo. Phone number is the more realistic field for a delivery flow and needs no explanation."* Beyond usability, it is also better privacy practice — a courier needs a phone number, never a government ID.

SECTION EIGHTEEN

Admin System

Role-based access control end to end — and why hiding a button in React protects nothing.

18.1 How the role is stored

```
role: { type: String, enum: ["member", "admin"], default: "member" }
```

The role lives in **MongoDB**, not in the Firebase token. That is a real architectural choice with trade-offs worth knowing.

	ROLE IN MONGODB (CHOSEN)	ROLE IN FIREBASE CUSTOM CLAIMS
Cost per request	One extra Mongo lookup	None — it is inside the token
Revocation speed	**Immediate** — next request reads the new role	Up to 1 hour, until the token refreshes
Where it is enforced	Express only	Express **and** Firestore rules
Complexity	Simple — one field	Requires `setCustomUserClaims` and a refresh dance

Because Bookverse denies *all* client Firestore writes anyway, the main advantage of custom claims (letting rules check the role) is unnecessary here — so the simpler option is also the right one.

18.2 Becoming an admin

`frontend/src/pages/AdminSetup.jsx`

```
const handleSubmit = async (e) => {
  e.preventDefault(); setError(""); setSubmitting(true);
  try {
    await bootstrapAdmin(secret.trim());
    await refreshProfile();            ← essential
    navigate("/admin", { replace: true });
  } catch (err) {
    setError(err.response?.data?.message || "Could not verify that admin key.");
  } finally { setSubmitting(false); }
};
```

```
// services/adminService.js
// One-time self-promotion: the secret is sent once, and every admin action
// afterwards is authorised by the account's role, not by the secret.
export async function bootstrapAdmin(secret) {
  const { data } = await api.post("/auth/bootstrap-admin", {},
                                  { headers: { "x-admin-secret": secret } });
  return data;
}
```

WHY `AWAIT REFRESHPROFILE()` IS NOT OPTIONAL

The server has updated MongoDB, but React's `profile` state still says `role: "member"`. Navigating to `/admin` without refreshing would hit `ProtectedRoute roles={["admin"]}`, fail the check against stale state, and bounce the new admin straight back to the home page. `refreshProfile()` re-fetches `/auth/me` and updates the context **before** navigating — and it is `await`ed so the state update happens first. This is the **refreshProfile pattern**: after any server-side change to the user's own record, re-read it before routing on it.

The page also guards itself: it shows a spinner while `loading`, redirects to `/login` if signed out, and if the account is *already* an admin it replaces the form with a "Go to Admin Dashboard" button instead. It is deliberately not wrapped in an admin-only `ProtectedRoute` — a non-admin must be able to reach it in order to become one.

18.3 Collection management

`AdminCollectionTab` loads statistics and books in parallel:

```
const [statsRes, booksRes] = await Promise.all([
  api.get("/stats"),
  api.get("/books", { params: { sort: "newest" } }),
]);
```

It renders stat cards (total, available, borrowed, purchased, categories), a table of every book, and a modal that serves both create and edit — `editingId` being `null` means create, otherwise update:

```
if (editingId) await booksAdmin.update(editingId, payload);
else           await booksAdmin.create(payload);
```

All three operations map to `POST/PUT/DELETE /api/books`, each guarded by `verifyFirebaseToken, requireRole("admin")`. Note the admin table shows only books that still exist — books bought with points are deleted, and the purchases ledger is what accounts for them (§11.5).

The form's `handleChange` handles checkboxes correctly: `const { name, value, type, checked } = e.target`, using `checked` for the `available` toggle and `value` for everything else — a detail plain text handlers get wrong.

18.4 Store management

`AdminStoreTab` mirrors the Collection tab against `storeBooksAdmin.create/update/remove`, and adds computed stat cards — average price and average rating, calculated in the component with `reduce` over the loaded list. It fetches books and categories together with `Promise.all`.

18.5 Member management

backend/controllers/adminController.js

```
// GET /api/admin/members — every registered user with their borrow status, so
// the admin can see who currently has which book without cross-referencing
// the Borrow collection by hand.
const [users, borrows] = await Promise.all([
  User.find({}).sort({ createdAt: -1 }).lean(),
  Borrow.find({}).populate("bookId", "title author coverImage").lean(),
]);

const byUser = {};
for (const b of borrows) {
  const key = String(b.userId);
  (byUser[key] ||= []).push(b);      group loans by user
}

const describe = (b) => ({
  // populate() yields null once a book is bought with points and deleted;
  // bookSnapshot is what the borrow route stores for exactly that case.
  title:      b.bookId?.title      || b.bookSnapshot?.title      || "Untitled",
  author:     b.bookId?.author     || b.bookSnapshot?.author     || "",
  coverImage: b.bookId?.coverImage || b.bookSnapshot?.coverImage || "",
  borrowedAt: b.borrowedAt,
  returnedAt: b.returnedAt,
});
```

```
const members = users.map((u) => {
  const own = byUser[String(u._id)] || [];
  const active   = own.filter((b) => b.status === "borrowed");
  const returned = own.filter((b) => b.status === "returned");
  return { _id: u._id, name: u.name, email: u.email, role: u.role, createdAt: u.createdAt,
           activeBorrowCount: active.length, returnedCount: returned.length,
           totalBorrowCount: own.length,
           activeBorrows:  active.map(describe).sort(...),
           borrowHistory: returned.map(describe).sort(...) };
});
```

Three techniques worth naming:

- **Two queries, not N+1.** Fetch all users and all borrows once, then join in memory. The naive version would query borrows per user.
- `.lean()` returns plain JavaScript objects instead of full Mongoose documents — significantly faster and lighter when you only intend to read.
- `(byUser[key] ||= []).push(b)` — logical OR assignment. Initialise the array if absent, then push, in one expression.

`AdminMembersTab` renders the result with expandable rows (`expanded` holds one member id) showing active loans and full history, plus stat cards for total members, active loans and admin count. `<Fragment>` is imported specifically so a member row and its expanded detail row can be returned together with a single key.

18.6 Frontend protection vs backend protection

Frontend — `ProtectedRoute roles={["admin"]}`

Purpose: user experience. Don't show people doors they cannot open.

Bypass: trivial. Open React DevTools, set `profile.role = "admin"`, and the dashboard renders.

Also cosmetic: the navbar's conditional `{(profile?.role === "admin" ? [...accountLinks, ADMIN_LINK] : accountLinks)}`.

Backend — `requireRole("admin")`

Purpose: actual security. It decides whether the operation happens.

Bypass: none available to a client. The role is read from MongoDB using an identity proven by a Google-signed token.

Result of the DevTools trick: the dashboard renders as an empty shell. `/api/stats` is public so numbers appear, but `/admin/members` returns `403`, and every create/edit/delete returns `403`.

THE SENTENCE TO SAY OUT LOUD

"Client-side route guards are UX. Server-side middleware is security. I have both, and I know which one is load-bearing." Then give the concrete demonstration: forging `profile.role` in DevTools renders the page but every privileged request still returns 403, because the role is read from the database against a cryptographically verified identity — and nothing the browser says can change that.

18.7 Admin capability summary

CAPABILITY	ENDPOINT(S)	NOTES
Create / edit / delete official books	`/api/books`	Full CRUD on the Collection
View Collection statistics	`/api/stats`	Endpoint is public; only the UI is admin-only
Create / edit / delete Store books	`/api/store/books`	Includes setting prices
View all members + loans	`/api/admin/members`	The only endpoint under `/api/admin`
Self-promote once	`/api/auth/bootstrap-admin`	Requires `x-admin-secret`
Cannot: promote another user through the UI (no endpoint exists — it requires the secret or direct DB access) · edit or delete community books · read anyone's cart, wishlist or orders · change anyone's points · view shipping addresses for community requests		

SECTION NINETEEN

Database Architecture

Two databases, thirteen collections, and an honest account of why both exist.

19.1 The split at a glance

MONGODB (MONGOOSE) — 3 COLLECTIONS

users — identity + role | books — the official Collection | borrows — loan records + shipping

linked only by the Firebase UID string

CLOUD FIRESTORE — 10 COLLECTIONS

users — points | usedBooks | borrowRequests | notifications | reviews | purchases | wishlist | storeBooks | carts | orders

19.2 MongoDB — what and why

COLLECTION	HOLDS	WHY MONGODB
`users`	`firebaseUid`, name, email, **role**, createdAt	Needs a strict schema and an `enum` on role — a typo'd role would be a security hole. Mongoose enforces it.
`books`	title, author, description, category, year, pages, cover, rating, available	Every field required and typed; the original feature, written before Firestore entered the project.
`borrows`	userId + bookId refs, dates, status, shipping, bookSnapshot	A genuine **relational** record joining two entities. `populate()` resolves the book in one call.

References and `populate`

```
userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
bookId: { type: mongoose.Schema.Types.ObjectId, ref: "Book", required: true },
```

An **ObjectId** is MongoDB's 12-byte primary key, encoding a timestamp, a machine id, a process id and a counter — which is why sorting by `_id` is roughly chronological. `ref: "Book"` tells Mongoose which model to resolve against when `.populate("bookId")` is called.

`populate` **is not a database join.** Mongoose issues a *second* query for the referenced ids and stitches the results in application memory. Two important consequences appear in this codebase:

- It returns `null` for a deleted target — which is exactly why `bookSnapshot` exists (§8.9, §18.5).
- A projection can be passed as the second argument — `.populate("bookId", "title author coverImage")` — to avoid fetching whole documents.

19.3 Firestore — what and why

COLLECTION	DOC ID	KEY FIELDS	WHY FIRESTORE
`users`	**firebaseUid**	`points`, name, email	`FieldValue.increment()` is atomic; per-user reads are one document lookup
`usedBooks`	auto	ownerUid, ownerName, condition, available	Member-generated, loosely structured, publicly readable
`borrowRequests`	auto	bookId, ownerUid, requesterUid, shipping, status	Needs per-document, two-party read rules
`notifications`	auto	userUid, type, title, body, read	**The decisive reason Firestore is here** — `onSnapshot` real-time push
`reviews`	auto	source, bookId, userUid, rating, text	Spans both shelves; publicly readable; schema varies by source
`purchases`	auto	userUid, book snapshot, pointsSpent, shipping	Written inside the same transaction that debits points
`wishlist`	auto	userUid, source, bookId, denormalised display fields	Owner-scoped reads; heavy denormalisation
`storeBooks`	auto	title, author, price, currency, rating	Consistency with the newer half of the app
`carts`	**firebaseUid**	`items[]`, updatedAt	One document per user; direct lookup; rule is `uid == userId`
`orders`	auto	userUid, items[], total, stripeSessionId, status	Nested arrays of line items; owner-only reads

19.4 Firestore concepts used in this project

CONCEPT	WHERE USED	WHAT IT DOES
`FieldValue.serverTimestamp()`	Every `createdAt`	The **server's** clock, so a wrong device clock cannot forge ordering
`FieldValue.increment(n)`	Points earn and spend	Atomic server-side arithmetic; concurrency-safe
`{ merge: true }`	Points, carts	Upsert without destroying other fields
`db.batch()`	read-all, wishlist delete, review snapshots	Up to 500 writes in one atomic commit
`db.runTransaction()`	Point purchase	Read-then-write with automatic retry on conflict
`.where()` chains	Almost every query	Compound equality filters
`.orderBy()`	**Only** in `listUsedBooks`	Everything else sorts in memory to avoid composite indexes
`.count().get()`	`/api/stats`	Aggregation — returns a number, not documents
`onSnapshot()`	NotificationBell only	Real-time listener; the one client-side Firestore usage
`snap.empty`, `doc.exists`	Every existence check	Query-result vs single-document emptiness

19.5 Why both — the honest answer

ANSWER IT IN TWO PARTS: HISTORY, THEN JUSTIFICATION

Part 1 — the historical truth. Bookverse began as a conventional MERN app: MongoDB, JWT + bcrypt authentication, three collections. Commit `c1c4113` migrated authentication to Firebase. Once the Firebase Admin SDK was already initialised on the server, Firestore was available at zero additional cost — so **every feature built afterwards used it**. The comment in `wishlistController.js` even names the convention out loud: "*Firestore, per the project's 'new features use Firestore' rule.*" The split is a **chronological boundary**: everything before the migration is in MongoDB, everything after is in Firestore.

Part 2 — why it is nonetheless defensible. The two databases really do have different strengths, and the data landed on the right sides more often than not.

REQUIREMENT	WHICH ENGINE WINS, AND WHY	BOOKVERSE EXAMPLE
Real-time push to browsers	**Firestore** — `onSnapshot`; MongoDB change streams cannot be consumed directly by a browser	Notification bell
Per-document access rules	**Firestore** — declarative rules enforced by the database itself	Only the two parties may read a borrow request
Atomic counters	**Firestore** — `increment()` without a read	Contribution points
Strict schema + enums	**MongoDB** — Mongoose validation	`role: enum["member","admin"]`
Rich queries: regex, sort, range	**MongoDB** — expressive query language, no index matrix	Collection search runs in the database; Store search must run in memory
Relational joins	**MongoDB** — `ref` + `populate`	Borrow → User + Book
Aggregation	**MongoDB** — `countDocuments`, `distinct`	`/api/stats`

A POLISHED 30-SECOND INTERVIEW ANSWER

"MongoDB holds the library's core domain — users, books and loans — where I want a strict schema, relational references and expressive queries. Firestore holds everything social and transactional, because two things it does that MongoDB can't: real-time listeners, which is how a borrow request reaches its owner instantly without polling, and per-document security rules enforced by the database itself. The honest history is that the project started MongoDB-only and adopted Firestore with the Firebase Auth migration — so the boundary partly reflects chronology. If I were rebuilding it, I'd consolidate `storeBooks` into MongoDB, since the Store needs exactly the rich filtering MongoDB is good at and gains nothing from real-time. But I'd keep notifications, requests and points in Firestore — those genuinely benefit."

19.6 The `users` collision

Both databases have a `users` collection, and this is the least tidy part of the design.

MONGODB `users`

```
{
  _id: ObjectId("..."),
  firebaseUid: "abc123",
  name: "Uday Barua",
  email: "uday@example.com",
  role: "admin",
  createdAt: ISODate(...)
}
```

Read by `verifyFirebaseToken` on **every** authenticated request. Authoritative for identity and role.

FIRESTORE `users/{uid}`

```
{
  points: 150,
  name: "Uday Barua",      duplicated
  email: "uday@..."        duplicated
}
```

Read for the points balance. Name and email are denormalised copies, written on each contribution.

They are joined only by the Firebase UID string. The duplication means a name change would need writing in two places — though since Bookverse has no profile-editing feature, this never actually bites. **Say this proactively:** "The cleanest fix is to move `points` onto the Mongo user document, which would eliminate the duplicate collection entirely. I kept it in Firestore because `FieldValue.increment` and the transaction support made the point economy simpler to get right — but I recognise it as the weakest seam in the schema."

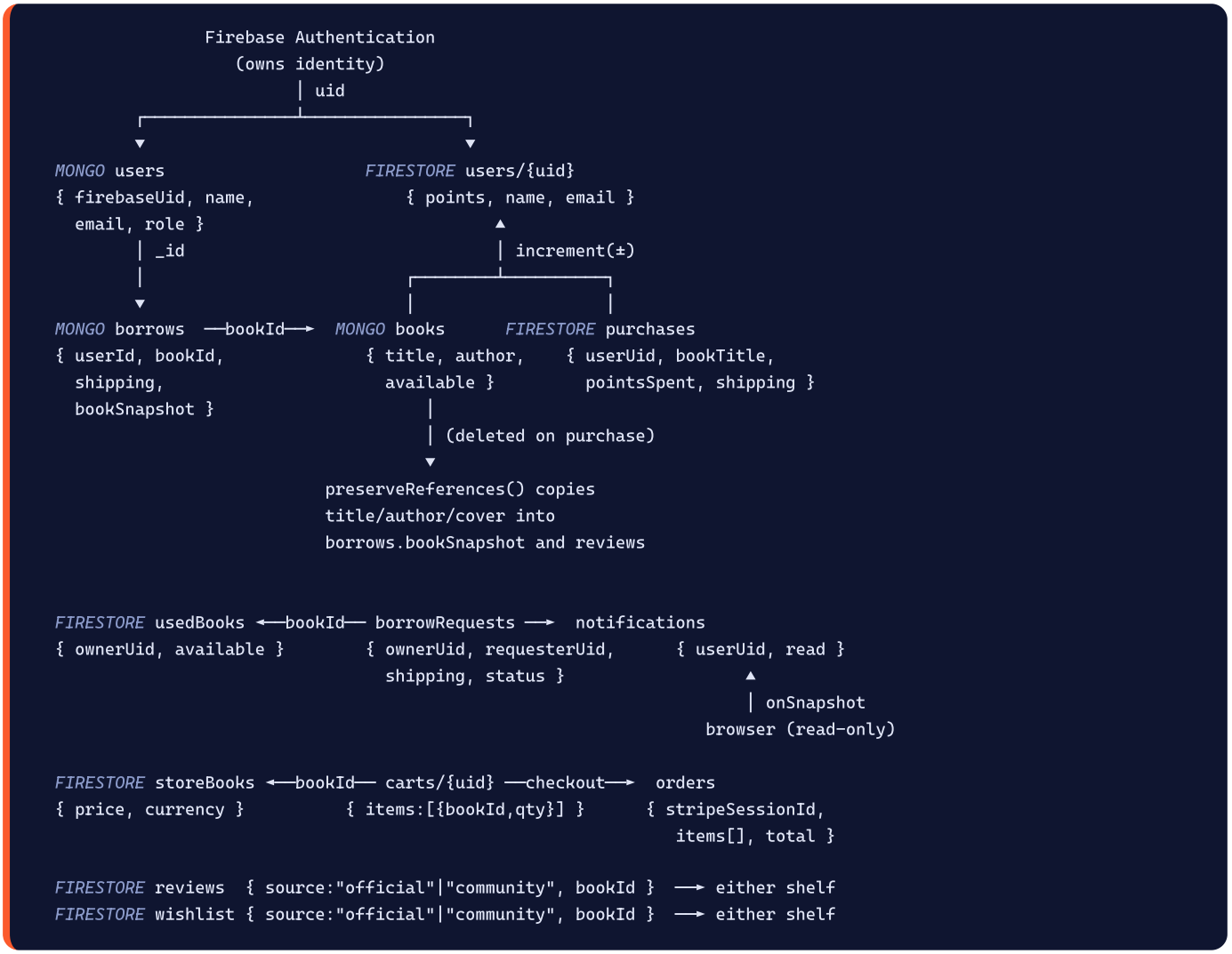
19.7 The three cross-database operations

Only three code paths touch both databases. Knowing them cold is worth a lot.

OPERATION	MONGODB SIDE	FIRESTORE SIDE	COORDINATION
`purchaseOfficialBook`	Claim, then delete the book; update borrow snapshots	Debit points; write the purchase; update reviews; delete wishlist entries	**Saga** — atomic claim, transactional payment, compensating release
`listMyReviews`	`Book.find({ _id: { $in: ids } })`	Fetch reviews; fetch used books	Read-only — no consistency concern
`/api/stats`	`countDocuments`, `distinct`	`purchases.count()`	Read-only — no consistency concern

Everything else stays within one database. That containment is what makes the dual-store design workable: there is exactly **one** write path that must span both, and it is explicitly engineered for it.

19.8 Entity relationship map



SECTION TWENTY

Firestore Security Rules

The complete file, rule by rule, with the attacks each one defeats.

20.1 The governing comment

firestore.rules

```
rules_version = '2';

// Bookverse Firestore rules.
//
// Every write goes through the Express API using the Firebase Admin SDK, which
// bypasses these rules. Clients therefore get READ access only — enough to run
// real-time listeners (the notification bell, live community shelf) while
// making it impossible for someone to award themselves contribution points or
// approve their own borrow request straight from the browser console.
service cloud.firestore {
  match /databases/{database}/documents {

    function signedIn() {
      return request.auth != null;
    }
```

`rules_version = '2'` selects the current rules language, in which `match /{document=**}` behaves recursively and `allow` statements are evaluated per operation.

`signedIn()` is a helper function — rules support them, and factoring the check out means it reads the same way in eight places. `request.auth` is populated by Firebase from the caller's ID token; it is `null` for anonymous callers.

THE ONE FACT THAT MAKES THE WHOLE DESIGN WORK

The Firebase Admin SDK bypasses security rules entirely. It authenticates as a service account with full project access. That is why the server can write to collections whose rules say `allow write: if false` — those rules govern **client SDK** access only. Understanding this is the difference between "the rules are broken" and "the rules are the architecture".

20.2 Rule by rule

`users` — points balances

```
// Points balances. Readable only by their owner; never client-writable.
match /users/{userId} {
  allow read: if signedIn() && request.auth.uid == userId;
  allow write: if false;
}
```

Because the document id **is** the UID, the check compares the caller against the id itself — no document read is required to evaluate it, which makes it both cheap and airtight.

Attack defeated: `updateDoc(doc(db,"users",myUid),{points:999999})` → `PERMISSION_DENIED`. Also: reading a rival's balance to see who is close to affording a book → denied.

`usedBooks` — the community shelf

```
// The community shelf is public to browse.
match /usedBooks/{bookId} {
```

```
  allow read: if true;
  allow write: if false;
}
```

Why public: `/community` is browsable signed-out, matching the Collection and Store. **Attack defeated:** flipping `available: true` on a book you have not returned, or editing someone else's listing.

`borrowRequests` — the two-party rule

```
// A borrow request carries a home address, so only the two people involved
// may read it.
match /borrowRequests/{requestId} {
  allow read: if signedIn() &&
    (request.auth.uid == resource.data.ownerUid ||
     request.auth.uid == resource.data.requesterUid);
  allow write: if false;
}
```

`resource.data` is the **stored document**, so this is evaluated per document against its own fields. It is the only rule in the file granting access to two different people, and it exists because the owner genuinely needs the borrower's address to post the book.

Attack defeated: harvesting home addresses and phone numbers by listing the collection. **The most serious data-exposure risk in the app, closed by four lines.** Note also that an unfiltered query is rejected outright — Firestore refuses queries it cannot prove will only return permitted documents.

`notifications`

```
match /notifications/{notificationId} {
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
  allow write: if false;
}
```

This is the rule the `onSnapshot` listener must satisfy — and it does, because the listener queries `where("userUid","==",firebaseUser.uid)`. **Attack defeated:** reading another user's notifications; marking them read; fabricating a fake "request approved" notice.

`reviews`

```
// Reviews are public to read, like the books they describe.
match /reviews/{reviewId} {
  allow read: if true;
  allow write: if false;
}
```

Attack defeated: writing reviews under another user's name; posting a hundred 5-star reviews to inflate an average; editing a review after the fact. The one-review-per-user rule in `reviewController` is only meaningful *because* this path is closed.

`purchases`, `wishlist`, `orders`

```
match /purchases/{purchaseId} {
  // Purchases include a delivery address — owner only.
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
  allow write: if false;
}
match /wishlist/{itemId} {
  // A user's saved books — owner only.
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
  allow write: if false;
}
match /orders/{orderId} {
  // Orders carry payment details — owner only.
  allow read: if signedIn() && request.auth.uid == resource.data.userUid;
```

```
  allow write: if false;
}
```

The same owner-only shape three times. **Attacks defeated:** fabricating a purchase to obtain a book for free; reading someone's purchase history and delivery address; reading what other people have bought and for how much.

`storeBooks` — the double defence

```
// The Store catalogue is public to browse. Prices are only ever read from
// here server-side when charging, so a client rewriting one would achieve
// nothing even if writes were open — they are not.
match /storeBooks/{bookId} {
  allow read: if true;
  allow write: if false;
}
```

That comment describes **defence in depth** explicitly. Even in a hypothetical where the write rule were misconfigured, checkout reads prices server-side from this same collection at charge time — so a tampered price would have to survive both layers. Two independent controls guarding one asset.

`carts`

```
// A cart is keyed by the owner's uid.
match /carts/{userId} {
  allow read: if signedIn() && request.auth.uid == userId;
  allow write: if false;
}
```

Attack defeated: adding items to someone else's cart, or inspecting what another shopper is about to buy.

The catch-all

```
match /{document=**} {
  allow read, write: if false;
}
```

`{document=**}` is a recursive wildcard matching every path at any depth. Placed last, it means **any collection added in future is denied by default** until someone writes an explicit rule for it. This is **fail-closed** design — the single most important habit in access control. A new feature that forgets its rules breaks loudly rather than leaking silently.

Firestore rules are not "first match wins" — every matching `allow` is evaluated and access is granted if **any** of them permits it. The catch-all therefore does not override the specific rules above; it only covers paths nothing else matched.

20.3 Attack scenarios, end to end

ATTEMPT (BROWSER CONSOLE, SIGNED IN)	RESULT	WHY
`updateDoc(doc(db,"users",myUid), {points:1e6})`	**Denied**	`allow write: if false` on `users`
`getDoc(doc(db,"users",someoneElse))`	**Denied**	`uid != userId`
`getDocs(collection(db,"borrowRequests"))`	**Denied**	Query cannot be proven to return only permitted docs
`getDocs(query(...,where("ownerUid","==",myUid)))`	**Allowed**	Provably returns only documents you may read
`updateDoc(requestRef, {status:"approved"})`	**Denied**	Writes closed; only `respondToRequest` can, after an ownership check
`updateDoc(storeBookRef,{price:1})`	**Denied**	Writes closed — and checkout re-reads prices anyway
`addDoc(collection(db,"orders"),{...})`	**Denied**	Fabricating a paid order is impossible
`addDoc(collection(db,"reviews"), {rating:5})` ×100	**Denied**	Rating manipulation blocked at the database
`getDocs(collection(db,"usedBooks"))`	**Allowed**	Deliberately public — it is a public shelf
POST `/api/wishlist` with a forged `userUid` in the body	**Ignored**	The controller uses `req.user.firebaseUid` from the verified token; the body field is never read

20.4 The read/write asymmetry, summarised

COLLECTION	CLIENT READ	CLIENT WRITE
`usedBooks`, `reviews`, `storeBooks`	**Public**	**NEVER — all ten collections** Every write flows through Express + Admin SDK
`users`, `carts`	Owner only (by document id)	
`notifications`, `purchases`, `wishlist`, `orders`	Owner only (by field)	
`borrowRequests`	Owner **or** requester	
anything else	Denied	Denied

WHY NOT ALLOW SCOPED CLIENT WRITES?

Firestore rules *could* express "a user may write their own cart". But they cannot express "points may only increase by exactly 50, and only when a used book was created in the same operation", or "a request may move to approved only by its owner, only from pending, and only alongside flipping the book's availability". **Business rules need a server.** Once the server must exist for the complex cases, routing *all* writes through it is simpler and safer than maintaining two parallel authorisation systems that could disagree.

SECTION TWENTY-ONE

Technical Topics I Must Know for the Interview

Every concept the project actually uses, taught through Bookverse's own code. Each entry: what it is, why it is here, where it lives, the code, the problem it solves, and a ready answer.

21.1 React

Components and props

Definition. A component is a function returning JSX. Props are its read-only inputs.

In Bookverse. Every UI element. `ReviewSection` is the sharpest example: the *same* component serves both shelves purely through props.

```
<ReviewSection source="official"  bookId={id} />   BookDetails.jsx
<ReviewSection source="community" bookId={id} />   CommunityBookDetails.jsx
```

Problem solved. One implementation, two contexts, zero duplication. `ShippingModal` does the same for three flows.

Interview Q: "props vs state?" → "Props come from the parent and are immutable inside the component; state is owned and mutated by the component itself. `ReviewSection` receives `source` as a prop but owns `rating` and `text` as state."

`useState`

```
const [book, setBook]         = useState(null);      object or null
const [loading, setLoading]   = useState(true);      starts true — data is coming
const [intent, setIntent]     = useState(null);      "borrow" | "purchase" | null
const [form, setForm]         = useState({ shippingName: "", shippingPhone: "", shippingAddress: "" });
```

Note the initial values. `loading: true` not `false` — the very first render must show a skeleton, not "0 results found". And updates are always immutable:

```
const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
setItems((prev) => [optimistic, ...prev]);   functional form — safe with stale closures
```

Interview Q: "when do you use the functional updater?" → "When the next value depends on the previous one, especially inside async code or event handlers that may hold a stale closure. `WishlistContext.toggle` uses `setItems(prev => …)` for exactly that reason."

`useEffect`

Three dependency patterns all appear in this codebase, and each means something different:

```
// ① [] — run once on mount
useEffect(() => { const unsub = onAuthStateChanged(auth, ...); return unsub; }, []);

// ② [deps] — re-run when inputs change
useEffect(() => { fetchBooks(); }, [search, author, category, sort]);

// ③ cleanup — undo a subscription
useEffect(() => {
  const onClick = (e) => { if (wrapRef.current && !wrapRef.current.contains(e.target)) setOpen(false); };
  document.addEventListener("mousedown", onClick);
```

```
  return () => document.removeEventListener("mousedown", onClick);
}, []);
```

Problem solved. Synchronising React with things outside React: the network, Firebase, the DOM.

Interview Q: "what happens without a cleanup?" → "A leak. The click-outside listener would accumulate on every mount; the Firestore `onSnapshot` connection would stay open forever and fire on a dead component. Returning the unsubscribe function is the fix — and in `NotificationBell` we return `onSnapshot`'s own return value directly."

`useContext` and the Context API

```
const AuthContext = createContext(null);

export function AuthProvider({ children }) { ...; return <AuthContext.Provider value={{...}}>{children}
</AuthContext.Provider>; }

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}
```

Problem solved. Prop drilling. Without it, `firebaseUser` would have to be threaded from `App` through every intermediate component to reach `WishlistButton`.

The guard is a custom-hook pattern worth knowing: throwing when the context is null converts a confusing "cannot read property of null" deep in a render into a precise error naming the actual mistake. All three contexts do it.

Interview Q: "why not Redux?" → "Three pieces of genuinely global state and no complex derived logic. Context is built in, has zero dependencies, and is sufficient. Redux would earn its place with time-travel debugging needs, complex reducers, or heavy cross-slice derivation — none of which this app has."

Known trade-off: any change to a context value re-renders every consumer. Splitting Auth into separate session and points contexts would reduce that, but at this scale it is not a measurable problem.

`useCallback`

```
const load = useCallback(async () => {
  if (!firebaseUser) { setCart(EMPTY); return; }
  ...
}, [firebaseUser]);

useEffect(() => { load(); }, [load]);
```

Problem solved. An infinite loop. `load` is in the effect's dependency array; without memoisation it would be a new function identity on every render, so the effect would fire every render, which sets state, which re-renders… `useCallback` keeps the identity stable until `firebaseUser` actually changes.

Interview Q: "isn't useCallback premature optimisation?" → "Often, yes — but not here. This isn't about render performance, it's about correctness: the function is a dependency of an effect, so its identity is load-bearing."

`useRef`

```
const confirmed = useRef(false);      Cart.jsx — a latch across renders
const wrapRef   = useRef(null);       NotificationBell.jsx — a DOM handle
const closeTimer = useRef(null);      Navbar.jsx — a timeout id
```

Two distinct uses, both present: holding a DOM node, and holding a mutable value that must survive re-renders *without* causing one.

Interview Q: "why `useRef` and not `useState` for the checkout latch?" → "Two reasons. Setting state triggers a re-render I don't need. More importantly, state updates are asynchronous — under StrictMode the effect runs twice before any re-render, so a state flag would still be `false` on the second run. `ref.current = true` takes effect immediately."

Custom hooks

Three exist: `useAuth`, `useCart`, `useWishlist`. All follow the same shape — call `useContext`, guard, return.

Interview Q: "what makes something a hook?" → "A function starting with `use` that calls other hooks. The naming convention is what lets the linter enforce the rules of hooks — top level only, never inside conditions or loops."

Conditional rendering

```
{loading ? <Skeletons /> : books.length === 0 ? <EmptyState /> : <Grid />}   ternary chain
{error && <p className="error">{error}</p>}                                   && short-circuit
{firebaseUser ? <ReviewForm /> : <LoginPrompt />}                             either/or
{tab === "borrowed" && <BorrowedTab />}                                       tab switching
```

> **A CLASSIC INTERVIEW TRAP**
>
> `{items.length && <List />}` renders a literal **0** when the array is empty, because `0` is falsy but still a valid React child. Bookverse avoids it by comparing explicitly — `items.length === 0 ? … : …` and `data.count > 0 && …`.

Controlled forms

```
<input required name="shippingName" value={form.shippingName} onChange={handleChange} />
```

React state is the single source of truth; the DOM only reflects it. Every form in the project is controlled. The computed property name in `handleChange` (`[e.target.name]: e.target.value`) is what allows one handler to serve every field.

Lists and keys

```
{books.map((book, i) => <BookCard key={book._id} book={book} index={i} />)}
{items.map((item) => <div key={`${item.source}-${item.bookId}`}>…</div>)}    composite key
```

Interview Q: "why not use the array index as a key?" → "Because it breaks when the list is reordered or filtered — React reuses the wrong DOM node and component state ends up attached to the wrong item. The one place indexes *are* used here is skeleton placeholders, where the items are identical and never reorder."

React Router

API	USED IN	PURPOSE
`<BrowserRouter>`	`main.jsx`	History API routing (no hash)
`<Routes> / <Route>`	`App.jsx`	Path → element table
`<Link>`	Everywhere	Client-side navigation without a reload
`useNavigate()`	Login, BookDetails, Navbar…	Programmatic navigation
`useParams()`	4 detail pages	`const { id } = useParams()`
`useSearchParams()`	Collection, Community, Store, Cart	Query string as state
`useLocation()`	ProtectedRoute, App, MyLibrary	Current path, search, and router `state`
`<Navigate>`	ProtectedRoute, AdminSetup	Declarative redirect during render

`replace` matters. `navigate(from, { replace: true })` after login means Back does not return to the login form. `navigate("/my-library", { replace: true })` after a purchase means Back does not return to a book page that no longer exists.

Router state passes data without putting it in the URL:

```
navigate("/login", { state: { from } });                          where to return to
navigate("/my-library", { state: { purchased: "Purchased! ..." } }); a one-off toast
const from = location.state?.from || "/";                          read it back
```

Framer Motion

Covered in §3.9. The three concepts to be able to name: `AnimatePresence` (enables exit animations by delaying unmount), `layout` (animates position changes automatically), and `layoutId` (shared-element transitions — the sliding tab underline).

21.2 JavaScript

async / await and Promises

```
const [borrowRes, mine, reqs, pts, purch] = await Promise.all([
  api.get("/borrowed/me"), fetchMyUsedBooks(), fetchRequests(), fetchMyPoints(), fetchMyPurchases(),
]);
```

Problem solved. Five sequential awaits would take the sum of the latencies; `Promise.all` takes the maximum. `MyLibrary` loads roughly five times faster for it. The same pattern appears in `buildCart`, `listRequests`, `listMyReviews`, and both admin tabs.

Interview Q: "`Promise.all` vs `Promise.allSettled`?" → "`all` rejects as soon as any promise rejects — so one failed call loses all five results. `allSettled` always resolves with per-promise outcomes. For a dashboard where partial data beats none, `allSettled` would arguably be better here."

try / catch / finally

```
try {
  setBooks(await fetchStoreBooks(params));
} catch (err) {
  console.error("Failed to load store books", err);
} finally {
  setLoading(false);        runs on BOTH paths — the point of finally
}
```

Why `finally` matters here: if `setLoading(false)` lived only in the `try`, a failed request would leave the skeleton spinning forever.

Destructuring

```
const { data } = await api.get("/store/books");                  object
const { search, author, category, sort } = req.query;            multiple
const { name, value, type, checked } = e.target;                 DOM event
const [searchParams, setSearchParams] = useSearchParams();        array (hooks)
const { id } = useParams();
const { items, subtotal, count } = cart;
```

Nearly every service function is one destructure and one return — `const { data } = await api.get(...); return data;` — which is what keeps components free of Axios response internals.

Spread and rest

```
{ id: d.id, ...d.data() }                      merge Firestore id with fields
{ ...form, [e.target.name]: e.target.value }   immutable form update
{ ...r, bookTitle: book?.title || r.bookTitle }  enrich without mutating
function requireRole(...roles) { ... }         rest parameters
validateBody({ ...doc.data(), ...req.body })   later keys win — partial update
```

Order matters: in `{ ...doc.data(), ...req.body }` the request body overrides stored values, which is precisely what makes a partial edit validate correctly.

Array methods

METHOD	REAL USAGE	PURPOSE
`map`	`snap.docs.map(d => ({ id: d.id, ...d.data() }))`	Transform every element
`filter`	`books.filter(b => b.category === category)`	Select a subset
`reduce`	`items.reduce((sum, i) => sum + i.lineTotal, 0)`	Fold to a single value (subtotal, average rating)
`sort`	`(b.createdAt?.toMillis?.() || 0) - (a...)`	Order in memory (**mutates** the array)
`find`	`items.find(i => i.bookId === bookId)`	First match or undefined
`some`	`cart.items.some(i => i.bookId === bookId)`	Boolean existence test
`includes`	`SOURCES.includes(source)`	Whitelist validation
`forEach`	`snap.docs.forEach(d => batch.delete(d.ref))`	Side effects, no return value
`Array.from`	`Array.from({ length: 8 }).map(...)`	Generate placeholder arrays

Optional chaining and nullish coalescing

```
err.response?.data?.message              a network error has no .response at all
b.createdAt?.toMillis?.() || 0           optional call — serverTimestamp may not be materialised
profile?.name?.split(" ")[0]             profile may not have loaded yet
points?.pointsToPurchase ?? 200          ?? falls back only on null/undefined
session.metadata?.userUid
```

`??` VS `||` — KNOW THE DIFFERENCE

`||` falls back on any falsy value (`0`, `""`, `false`); `??` only on `null` / `undefined`. Bookverse uses `points?.points ?? 0` deliberately: a genuine balance of **0** must display as 0, and `||` would work here by coincidence but expresses the wrong intent. Being able to explain the distinction with this exact example is a strong answer.

Closures

```
function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) return res.status(403).json({...});
    next();
  };
}
```

The clearest closure in the codebase. `requireRole("admin")` runs once at startup; the returned middleware runs on every request, minutes or days later, and still has access to `roles`. That captured variable *is* the closure.

Two more: `const cartRef = (uid) => db.collection(...).doc(uid)` closes over `db`; and `const key = (i) => i.source === book.source && i.bookId === book.bookId` in `WishlistContext.toggle` closes over the `book` parameter.

Modules — two systems in one repo

```
// Backend — CommonJS ("type": "commonjs")
const express = require("express");
module.exports = { verifyFirebaseToken, requireRole };

// Frontend — ES Modules ("type": "module")
import axios from "axios";
export default api;
export const taka = (amount) => ...;
```

Interview Q: "why two module systems?" → "The backend is plain Node with `"type": "commonjs"`, matching the Express ecosystem's default. The frontend is bundled by Vite, which is ESM-native and needs static imports for tree-shaking. They are separate packages with separate `package.json` files, so they can differ."

`URLSearchParams`

```
const next = new URLSearchParams(searchParams);   copy — never mutate the current one
if (value && value !== "All") next.set(key, value);
else next.delete(key);
setSearchParams(next);
```

Copying rather than mutating is essential: mutating the existing object would not trigger a router update, and React would not re-render.

Other essentials in use

- `Set` — `[...new Set(ids)]` for de-duplication; `POPUP_ENV_FAILURES.has(code)` for O(1) membership.
- `Object.fromEntries` / `Object.entries` — building the id→book lookup map, and turning the category tally into an array.
- **Template literals** — every URL: `` `/store/books/${id}` ``.
- `Number`, `Number.isNaN`, `Math.max`, `Math.round` — input coercion and clamping.
- `||=` — `(byUser[key] ||= []).push(b)` in the admin grouping.
- `toLocaleString` / `toLocaleDateString` — currency and dates.
- `e.preventDefault()` / `e.stopPropagation()` — form submission and the heart-inside-a-link problem.

21.3 Axios

Definition. A promise-based HTTP client. Chosen over `fetch` for three concrete reasons visible in this codebase: automatic JSON parsing, **interceptors**, and non-2xx responses rejecting instead of resolving.

```
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL || "http://localhost:5050/api" });

api.interceptors.request.use(async (config) => {
  if (auth) await auth.authStateReady();
  const user = auth?.currentUser;
  if (user) {
    const token = await user.getIdToken();
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

- `axios.create` — an isolated instance. The interceptor applies only to Bookverse calls, not to any third-party library that also uses Axios.
- `baseURL` — every call writes `/cart`, not the full origin, so switching between local and production is one environment variable.
- **The interceptor is** `async` — Axios awaits it before sending, which is what allows `authStateReady()` and `getIdToken()` to be awaited here.
- **It must** `return config` — the (possibly modified) config is what gets sent.

Interview Q: "why does a non-2xx throw?" → "Axios rejects on error statuses, so `try/catch` handles HTTP errors naturally and `err.response.data.message` carries the server's explanation. With `fetch` you must check `res.ok` manually — forgetting to is a very common bug."

There is **no response interceptor** in this project. A natural addition would be one that catches 401s globally and signs the user out.

21.4 Node.js and Express

The middleware pipeline

```
app.use(cors());                 ① every request
app.use(express.json());         ② parse body → req.body
app.use("/api/books", routes);   ③ path-scoped
app.use(errorHandler);           ④ 4 args = error handler, must be last
```

Interview Q: "what is middleware?" → "A function with access to `req`, `res` and `next` that runs before the route handler. It can enrich the request (`express.json` adds `req.body`, `verifyFirebaseToken` adds `req.user`), end it early (a 401), or pass control on. They run in registration order, which is why `express.json()` must precede the routers."

Router-level vs route-level guards

```
router.use(verifyFirebaseToken);                                  cart.js, checkout.js
router.use(verifyFirebaseToken, requireRole("admin"));            admin.js
router.post("/books", verifyFirebaseToken, requireRole("admin"), fn);  store.js — mixed access
```

Router-level where every route needs the same guard (so a future route cannot forget it); route-level where public and protected routes coexist in one file.

Parameters, query and body

```
router.get("/:source/:bookId", ...)   →  req.params.source, req.params.bookId
GET /api/books?search=gats&sort=az    →  req.query.search, req.query.sort
POST body { bookId, quantity }        →  req.body.bookId    (needs express.json())
Authorization: Bearer ...             →  req.headers.authorization
```

Everything in `req.query` and `req.params` is a string — hence `Number(minRating)` and `String(minRating) === String(r)`.

21.5 Authentication & Security

Authentication vs authorisation

	AUTHENTICATION — WHO ARE YOU?	AUTHORISATION — WHAT MAY YOU DO?
Implemented by	`verifyFirebaseToken`	`requireRole` + ownership checks
Source of truth	Google's token signature	MongoDB `role`, and document owner fields
Failure code	**401**	**403**

Firebase ID tokens

A JWT — three base64url segments (header.payload.signature) — signed with Google's private key. `verifyIdToken` checks the signature against Google's rotating public keys plus expiry (`exp`), issuer (`iss`) and audience (`aud`).

Interview Q: "the token is visible in devtools — isn't that a problem?" → "A JWT is signed, not encrypted, so it is readable by design. What matters is that it cannot be *forged* — any edit invalidates the signature. It expires in an hour, and it is scoped to this Firebase project via the audience claim."

Interview Q: "why `Bearer`?" → "It's the HTTP standard scheme from RFC 6750 for token authentication — 'whoever bears this token may act as its subject'. The server slices the first seven characters off to extract the token."

RBAC

Two roles, one field, enforced by one middleware, gating nine endpoints. See §18.

Environment variables and CORS

Covered in §2.5 (the public/secret split) and §5.9 (CORS is currently permissive, and why that is lower severity with bearer-token auth than it would be with cookies).

21.6 MongoDB & Mongoose

CONCEPT	BOOKVERSE USAGE
Schema / Model	3 schemas; `mongoose.model("User", userSchema)` compiles a schema into a queryable model
ObjectId	Primary keys; `Borrow.userId` and `Borrow.bookId` reference them
`ref` + `populate`	`.populate("bookId")`; returns `null` for deleted targets — hence snapshots
Validation	`required`, `enum`, `min` / `max`, `unique`, `trim`, `lowercase`, `default`
Queries	`find`, `findById`, `findOne`, `countDocuments`, `distinct`
Operators	`$regex`, `$options`, `$in`, `$set`, `$exists`
Atomic update	`findOneAndUpdate({ _id, available: true }, { $set: {...} })` — check and write in one operation
Bulk	`updateMany` in `preserveReferences`
Projection	`.select("title author coverImage")`
`.lean()`	Plain objects instead of Mongoose documents — faster for read-only paths
Sorting	`.sort({ createdAt: -1 })` — **`-1` descending, `1` ascending**
Indexes	`index: true` + `unique: true` on `firebaseUid` — used on every authenticated request
Transactions	**Not used.** MongoDB supports them, but the one cross-store operation needed a saga instead, since a Mongo transaction cannot cover Firestore.

21.7 Firestore

Fully covered in §19.4 and §20. The five things to be able to explain without notes:

1. `FieldValue.increment()` — atomic, no read required, concurrency-safe.
2. `serverTimestamp()` — the server's clock, so ordering cannot be forged.
3. `batch()` **vs** `runTransaction()` — a batch is blind atomic writes (up to 500); a transaction reads first, then writes, and retries on conflict. Bookverse uses a batch for mark-all-read and a transaction for spending points, and the difference is exactly whether the write depends on a value that was read.
4. **Admin SDK bypasses rules; client SDK does not.**
5. `onSnapshot` — the real-time listener that justifies Firestore's presence.

21.8 Stripe

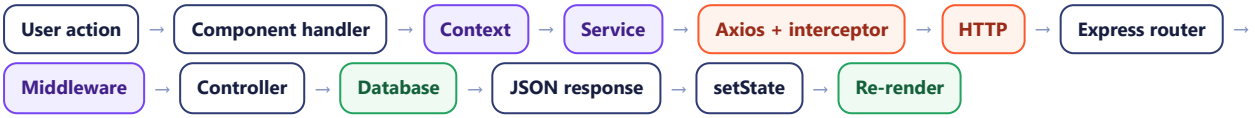
CONCEPT	HOW BOOKVERSE USES IT
Stripe Checkout (hosted)	Stripe hosts the payment page; Bookverse redirects to `session.url`
Checkout Session	`stripe.checkout.sessions.create({ mode: "payment", ... })`
`line_items` + `price_data`	Prices defined inline from the Firestore catalogue rather than pre-created Price objects
Smallest currency unit	`toStripeAmount = taka => Math.round(taka * 100)`
Secret key	Server-only; there is no publishable key and no browser SDK
`metadata`	Carries `userUid`, verified again on return
Session retrieval	`sessions.retrieve(id, { expand: ["line_items"] })`
Idempotency	Query `orders` by `stripeSessionId` before inserting
Webhooks	**Not implemented.** §16.9 explains the gap and the fix.

21.9 Web & API concepts

REST. Bookverse is broadly RESTful: resource-oriented URLs, HTTP verbs carrying meaning (`GET` read, `POST` create, `PUT` replace, `PATCH` partial, `DELETE` remove), JSON in and out, meaningful status codes, and a stateless server — every request carries its own token; no server session exists.

Be honest about where it bends: `POST /api/books/:id/borrow` and `POST /api/community/requests/:id/respond` are RPC-style actions, not resource manipulation. Strict REST would model a `/borrows` resource. Action endpoints are a pragmatic and extremely common choice — knowing that it is a deviation is the point.

The request/response lifecycle, end to end:



SECTION TWENTY-TWO

Code Reading — Why This Line Exists

The critical files, line by line: what each line does, what it holds, why it is needed, and what breaks without it.

22.1 `AuthContext.jsx` — the effect

```
1  useEffect(() => {
2    if (!auth) { setLoading(false); return; }
3    getRedirectResult(auth).catch((err) => {
4      console.error("Google redirect sign-in failed", err);
5      setRedirectError(err);
6    });
7    const unsubscribe = onAuthStateChanged(auth, async (fbUser) => {
8      setFirebaseUser(fbUser);
9      if (fbUser) {
10       try {
11         const { data } = await api.post("/auth/sync", { name: fbUser.displayName });
12         setProfile(data);
13         refreshPoints();
14       } catch (err) {
15         console.error("Failed to sync user profile", err);
16         setProfile(null);
17       }
18     } else {
19       setProfile(null);
20       setPoints(null);
21     }
22     setLoading(false);
23   });
24   return unsubscribe;
25 }, []);
```

LINE	WHAT IT DOES	WHAT BREAKS WITHOUT IT
2	Bails out when Firebase is unconfigured	The app hangs on `loading: true` forever, showing a spinner on every protected route
3–6	Settles a pending redirect sign-in, capturing its **error**	A failed Google redirect fails silently — the user returns to the login form with no explanation
7	Subscribes to auth changes. `fbUser` is a Firebase User or `null`	No sign-in would ever be noticed by React
8	Stores the session object	`ProtectedRoute` and both other contexts have nothing to react to
11	Exchanges the Firebase identity for the Mongo profile. `data` holds `{_id, firebaseUid, name, email, role, createdAt}`	No role, no name, no admin capability — and a Google user would never get a Bookverse record
13	Loads the points balance. **Not awaited** — deliberately, so the UI unblocks immediately	The navbar badge and buy button would show 0 until a manual refresh
16	Clears the profile on sync failure	A stale profile from a previous session could persist
19–20	Clears everything on sign-out	The next user would briefly see the previous user's name and points
22	Ends loading **on every path**, outside the `if` and outside the `try`	A failed sync would leave the app spinning forever
24	Returns the unsubscribe function as the cleanup	Listeners stack on remount; every auth change fires N times
25	`[]` — run exactly once	Any dependency would re-subscribe repeatedly

22.2 `api.js` — all twelve lines

```
1  import axios from "axios";
2  import { auth } from "../firebase";
3
4  const api = axios.create({
5    baseURL: import.meta.env.VITE_API_URL || "http://localhost:5050/api",
6  });
7
8  api.interceptors.request.use(async (config) => {
9    if (auth) await auth.authStateReady();
10   const user = auth?.currentUser;
11   if (user) {
12     const token = await user.getIdToken();
13     config.headers.Authorization = `Bearer ${token}`;
14   }
15   return config;
16 });
17
18 export default api;
```

LINE	WHAT IT DOES	WHAT BREAKS WITHOUT IT
4	Creates an **isolated** instance	Using the global `axios` would attach Bookverse tokens to any third-party request
5	`baseURL` from env, with a local fallback	Every call site would need the full URL; deployment would require a code change
8	Registers an **async** request interceptor	Every component would have to fetch and attach a token by hand
9	**Waits for the persisted session to restore.** Resolves once Firebase's initial auth state is known	**The refresh bug:** the first requests after a hard reload go out unauthenticated and 401 (§6.4)
10	Reads the current user; optional chaining for the unconfigured case	`TypeError` when Firebase is not initialised
11	Only attach a header when signed in	Public endpoints would receive `Bearer undefined`
12	Returns a valid token, refreshing transparently if expired. `token` is the raw JWT string	A cached token would expire after an hour and everything would 401
13	Sets the header in the standard scheme	`verifyFirebaseToken` finds nothing and returns 401
15	**Returns** `config`	Returning nothing means Axios has no request to send — everything breaks

22.3 `verifyFirebaseToken`

```
const authHeader = req.headers.authorization || "";                    ① never undefined
const token = authHeader.startsWith("Bearer ") ? authHeader.slice(7) : null;  ② strip 7 chars
if (!token) return res.status(401).json({ message: "Missing authorization token" });
const decoded = await admin.auth().verifyIdToken(token);               ③ the real check
req.firebaseUser = decoded;
const user = await User.findOne({ firebaseUid: decoded.uid });         ④ identity → account
if (!user) return res.status(401).json({ message: "User not found. Please complete signup." });
req.user = user;                                                        ⑤ hand to controllers
next();
```

1. `|| ""` means `.startsWith()` can never throw on a missing header.
2. `"Bearer "` is exactly 7 characters, so `slice(7)` yields the token. Checking the prefix rejects malformed schemes rather than passing garbage to Firebase.
3. The line everything rests on. `decoded` holds `{ uid, email, name?, iss, aud, exp, iat, ... }`. Any tampering breaks the signature; expiry, issuer and audience are all validated. Remove it and **anyone could send any uid and be believed**.
4. Converts a Google identity into a Bookverse account. Also a second gate — a valid Google token for someone who never completed signup is rejected.

5. Every downstream controller can now use `req.user._id`, `req.user.firebaseUid`, `req.user.role` and `req.user.name` as **proven** facts.

The whole body is wrapped in `try/catch` returning 401, because `verifyIdToken` *throws* on an invalid token rather than returning a falsy value.

22.4 `buildCart()` — the price-integrity function

```
const doc = await cartRef(uid).get();
const raw = doc.exists ? doc.data().items || [] : [];      ① double fallback
if (raw.length === 0) return { items: [], subtotal: 0, count: 0 };   ② early exit

const books = await Promise.all(                           ③ concurrent reads
  raw.map(async (i) => {
    const b = await db.collection(COLLECTIONS.storeBooks).doc(i.bookId).get();
    return b.exists ? { id: b.id, ...b.data() } : null;    ④ null for deleted
  })
);

raw.forEach((i, idx) => {
  const book = books[idx];
  if (!book) return;                                       ⑤ skip deleted lines
  items.push({ ..., price: book.price, quantity: i.quantity,
               lineTotal: book.price * i.quantity });      ⑥ money computed HERE
});
```

1. `doc.exists` handles a user with no cart; `|| []` handles a cart document with no `items` field.
2. Avoids an empty `Promise.all` and returns the canonical empty shape, so the client always receives the same three keys.
3. Three items cost one round-trip of latency, not three.
4. A product deleted from the Store after being added.
5. Without this, `book.price` would throw and **the entire cart page would fail to load**.
6. **The security line.** `price` comes from `book` (the catalogue), never from `i` (the cart). Swap them and clients could set their own prices.

22.5 The point-purchase saga

Covered in full in §11.5. The four lines to be able to recite:

```
Book.findOneAndUpdate({ _id, available: true }, { $set: { available: false } })  atomic claim
db.runTransaction(async (tx) => { const bal = (await tx.get(userRef))...; })     safe debit
await releaseClaim(book._id);                                                    compensate
await preserveReferences(book); await Book.findByIdAndDelete(book._id);          snapshot, then delete
```

22.6 The notification listener

Covered in §10.3. The three lines that matter most:

```
where("userUid", "==", firebaseUser.uid)   filter AND the thing that satisfies the rule
(err) => console.error(...)                error callback — else an unhandled rejection
return unsubscribe;                        cleanup — else a leaked live connection
```

22.7 The three `ProtectedRoute` guards

```
if (loading) return <Spinner />;                       ① or refresh logs you out
const from = `${location.pathname}${location.search}`;  ② or Stripe's session_id is lost
```

```
if (roles && !profile) return <Spinner />;            ③ or an admin is ejected
```

22.8 The Stripe confirmation guards

```
if (session.metadata?.userUid !== req.user.firebaseUid) return 403;  ① not your session
if (session.payment_status !== "paid")                 return 400;  ② not actually paid
if (!existing.empty) return res.json({ ...doc.data(), alreadyRecorded: true }); ③ no duplicates
```

Remove ① and anyone who guesses a session id claims someone else's order. Remove ② and visiting `/cart?session_id=anything` could record an unpaid order. Remove ③ and every page refresh creates another order.

22.9 `bootstrapAdmin`

```
const providedSecret = req.headers["x-admin-secret"];                    ① header, not URL
if (!process.env.ADMIN_SECRET || providedSecret !== process.env.ADMIN_SECRET)
  return res.status(403).json({ message: "Invalid admin secret." });     ② fails closed
if (req.user.role !== "admin") { req.user.role = "admin"; await req.user.save(); }  ③ idempotent
```

① keeps the secret out of logs and browser history. ② the `!process.env.ADMIN_SECRET` check means an unset variable denies everyone rather than accidentally matching `undefined`. ③ calling it twice is harmless. And the route itself still requires a valid token, so the secret alone is useless.

SECTION TWENTY-THREE

Complete Real-World Request Flows

Twenty-three end-to-end journeys, each naming the endpoint, the guard, the database and the file at every hop.

Flows 1–6 of the authentication set are given in full in §6.6; flows 7–13 of the community lifecycle in §9.8; the Stripe purchase in §16.7. This section is the consolidated quick-reference — use it for revision, and the detailed sections for depth.

#	FLOW	ENDPOINT · GUARD	PATH THROUGH THE SYSTEM
1	**New registration**	`POST /auth/sync` · inline token	Register.jsx → `createUserWithEmailAndPassword` → `updateProfile` → onAuthStateChanged → interceptor → syncUser → MONGO `User.create` (role `member`) → setProfile → refreshPoints → `/`
2	**Google login**	`POST /auth/sync`	Login.jsx → `signInWithPopup` (fallback `signInWithRedirect` via the first-party `/__/auth/*` proxy) → onAuthStateChanged → syncUser (name from `decoded.name`) → navigate `from`
3	**Protected API call**	any · `verifyFirebaseToken`	service → interceptor (`authStateReady` → `getIdToken`) → `Authorization: Bearer` → `verifyIdToken` → Mongo user lookup → `req.user` → controller scoped by `req.user.firebaseUid`
4	**Collection search/sort**	`GET /books` · public	Collection.jsx → `setSearchParams` → URL changes → effect deps change → `api.get` → query object built → MONGO `find(query).sort(sortOption)` → setBooks → grid re-renders
5	**Borrow official book**	`POST /books/:id/borrow` · token	BookDetails → ShippingModal → 4 guards (exists, available, shipping, not already borrowed) → MONGO `Borrow.create` + `bookSnapshot` → `available = false` → refetch book
6	**Buy with points**	`POST /points/purchase/:bookId` · token	validate → **atomic claim** MONGO → **transaction** FIRE debit + purchase doc → on failure `releaseClaim` → `preserveReferences` (both DBs) → delete Book → navigate `/my-library`
7	**Add a used book**	`POST /community/books` · token	AddUsedBookModal → validate → FIRE `usedBooks.add` with `ownerUid` from token → `users/{uid}` `increment(+50)` merge → toast → `loadAll()`
8	**Earn points**	(same request)	`FieldValue.increment(50)` — atomic, server-side; response echoes `pointsAwarded`; `refreshPoints()` updates the navbar badge
9	**Search community**	`GET /community/books` · public	Community.jsx → URL params → FIRE fetch all `orderBy createdAt desc` → **filter in Node memory** (no case-insensitive contains in Firestore) → JSON
10	**Request to borrow**	`POST /community/books/:id/request` · token	5 guards → FIRE `borrowRequests.add` (status `pending`, shipping) → `notify(ownerUid)`
11	**Owner is notified**	— (no request)	FIRE notification doc → matches the owner's open `onSnapshot` → **pushed** to the browser → `setItems` → unread badge. No polling, no Cloud Function.
12	**Owner approves**	`POST /community/requests/:id/respond` · owner	ownership + state-machine guards → request → `approved` → used book → `available: false` → notify requester → address revealed in the owner's UI
13	**Borrower returns**	`POST /community/requests/:id/return` · borrower	`requesterUid` check + status must be `approved` → `returned` → book `available: true` → notify owner
14	**Write a review**	`POST /reviews/:source/:bookId` · token	ReviewSection → validate source + rating 1–5 → **duplicate check** (3-field query) → snapshot book if official → FIRE `reviews.add` → `load()` refetches average + count
15	**Add to wishlist**	`POST /wishlist` · token	WishlistButton → **optimistic** insert with temp id → idempotency query → FIRE create → swap temp for real; on failure `load()` resyncs
16	**Add to cart**	`POST /cart` · token	StoreBookCard → `useCart().add` → book exists? → merge `items[]` → `buildCart()` re-reads prices → full cart returned → `setCart` → navbar badge
17	**Change quantity**	`PATCH /cart/:bookId` · token	Cart.jsx +/− → sends the **absolute** new quantity → 0 filters the line out → `buildCart()` → totals recomputed server-side
18	**Stripe checkout**	`POST /checkout/session` · token	**empty body** → `buildCart` → `line_items` with `toStripeAmount` → metadata `userUid` → `session.url` → `window.location.href`
19	**Payment confirmation**	`GET /checkout/confirm/:sessionId` · owner	Stripe redirect → `/cart?session_id=…` → `useRef` latch → retrieve session → **3 guards** → FIRE order created → cart emptied → URL stripped

#	FLOW	ENDPOINT · GUARD	PATH THROUGH THE SYSTEM
20	**Orders page**	`GET /checkout/orders` · token	scoped by `userUid` → sorted in memory → rendered with `taka()` and the dual-shape timestamp helper
21	**Admin bootstrap**	`POST /auth/bootstrap-admin` · token + secret	AdminSetup → `x-admin-secret` header → compared to `process.env.ADMIN_SECRET` → MONGO role → `admin` → `await refreshProfile()` → navigate `/admin`
22	**Admin adds a book**	`POST /books` · **admin**	AdminCollectionTab modal → `booksAdmin.create` → `verifyFirebaseToken` + `requireRole("admin")` → MONGO `Book.create` → `fetchAll()` refreshes table + stats
23	**Admin manages Store**	`POST/PUT/DELETE /store/books` · **admin**	AdminStoreTab → `storeBooksAdmin.*` → admin guard → `validateBody` (update merges old+new) → FIRE write

23.1 The universal shape

1. **User action** — a click or a keystroke in a page component
2. **Handler** sets local UI state (`submitting` , `busyId`) to prevent double submission
3. **Context or service** — domain vocabulary, never a raw URL in a component
4. **Axios interceptor** — `authStateReady()` , then `Bearer <token>`
5. **HTTP** to Express with `baseURL` prefixed
6. **Router** matches path + method, in registration order
7. **Middleware** — `verifyFirebaseToken` , then `requireRole` where applicable
8. **Controller** — validate, check ownership, apply the business rule
9. **Database** — Mongoose and/or the Firestore Admin SDK
10. **Response** — JSON with a meaningful status code
11. **State update** — `setX(data)` , replacing rather than patching wherever the server returns a full object
12. **Re-render** — every consumer of that state updates together

SECTION TWENTY-FOUR

Feature → File Map

The revision table. One row per feature, every layer named.

FEATURE	PAGE / COMPONENT	SERVICE / CONTEXT	ROUTE → CONTROLLER	DATA	SECURITY
Register	`pages/Register.jsx` `AuthLayout`, `GoogleIcon`	`AuthContext.signup`	`routes/auth.js` → `syncUser`	MONGO users	Inline `verifyIdToken`; uid from token
Login	`pages/Login.jsx`	`AuthContext.login` `utils/authError`	`routes/auth.js` → `syncUser`	MONGO users	Firebase verifies the credential
Google sign-in	Login / Register	`AuthContext.loginWithGoogle`	`/auth/sync`	MONGO users	OAuth via Firebase; first-party `/__/auth/*` proxy
Logout	`components/Navbar.jsx`	`AuthContext.logout`	—	—	`signOut` clears persistence; contexts cascade
Browse Collection	`pages/Collection.jsx` `BookCard`, `RatingStars`	`services/api` direct	`routes/books.js` (inline)	MONGO books	Public
Search / filter / sort	Collection · `useSearchParams`	—	`GET /api/books`	MONGO `$regex` + `.sort()`	Public; regex unescaped (§5.9)
Book details	`pages/BookDetails.jsx` `BookCover`, `ReviewSection`	`useAuth`, `pointsService`	`GET /api/books/:id`	MONGO books	Public to view
Borrow	BookDetails + `ShippingModal`	`services/api` direct	`POST /books/:id/borrow`	MONGO borrows	Token; 4 guards; **check-then-act race (§8.7)**
Return	`pages/MyLibrary.jsx`	`services/api` direct	`POST /books/:id/return`	MONGO borrows	Query scoped to `req.user._id`
Buy with points	BookDetails + ShippingModal	`pointsService.purchaseBook`	`points.js` → `purchaseOfficialBook`	MONGO + FIRE	Atomic claim + transaction + compensation
Points balance	Navbar, MyLibrary, AccountInfo	`AuthContext.points` `refreshPoints`	`GET /points/me`	FIRE users/{uid}	Owner-only rule; writes denied
Purchase history	MyLibrary → Purchased	`pointsService.fetchMyPurchases`	`GET /points/purchases`	FIRE purchases	Query scoped by `userUid`
Browse community	`pages/Community.jsx` `UsedBookCard`	`communityService.fetchUsedBooks`	`community.js` → `listUsedBooks`	FIRE usedBooks	Public read; **filtered in Node memory**
Contribute a book	`AddUsedBookModal`	`communityService.addUsedBook`	`POST /community/books` → `addUsedBook`	FIRE usedBooks + users	ownerUid from token; server-side `increment`
Withdraw a book	MyLibrary → Contributions	`removeUsedBook`	`DELETE /community/books/:id`	FIRE usedBooks	**Ownership check** (403)
Request to borrow	`CommunityBookDetails` + ShippingModal	`communityService.requestBorrow`	`POST /community/books/:id/request`	FIRE borrowRequests + notifications	5 guards; two-party read rule
Approve / decline	MyLibrary → Requests	`respondToRequest`	`POST /community/requests/:id/respond`	FIRE borrowRequests + usedBooks	Owner-only + state-machine guard
Return community book	MyLibrary → Requests	`returnUsedBook`	`POST /community/requests`	FIRE	**Borrower-only** + status must be approved

FEATURE	PAGE / COMPONENT	SERVICE / CONTEXT	ROUTE → CONTROLLER	DATA	SECURITY
			`/:id/return`		
Notifications	`components/NotificationBell.jsx`	`firebase.firestore` direct read `services/api` for writes	`routes/notifications.js`	FIRE notifications	Owner-only read rule; writes API-only; **no Cloud Functions**
Reviews	`components/ReviewSection.jsx`	`reviewService`	`reviews.js` → `reviewController`	FIRE reviews (+ MONGO for snapshots)	Public read; one review per user per book
My Reviews	`pages/MyReviews.jsx`	`fetchMyReviews`	`GET /reviews/mine`	both	Scoped by `userUid` ; `$in` batching
Wishlist	`WishlistButton` , `pages/Wishlist.jsx`	`WishlistContext` `wishlistService`	`wishlist.js` → `wishlistController`	FIRE wishlist	Owner-only; idempotent add; batch delete
Store catalogue	`pages/Store.jsx` , `StoreBookCard`	`storeService`	`store.js` → `storeController`	FIRE storeBooks	Public read; **filtered in Node memory**
Cart	`pages/Cart.jsx`	`CartContext` `cartService`	`cart.js` → `cartController`	FIRE carts	`router.use(verifyFirebaseToken)` ; **prices re-read every call**
Stripe checkout	Cart → Stripe-hosted page	`cartService.startCheckout`	`checkout.js` → `createCheckoutSession`	FIRE carts + storeBooks	Server-side pricing; `metadata.userUid`
Payment confirm	Cart (receipt state)	`confirmCheckout`	`GET /checkout/confirm/:sessionId`	FIRE orders + carts	Ownership + `payment_status` + idempotency
Orders	`pages/Orders.jsx`	`fetchOrders`	`GET /checkout/orders`	FIRE orders	Scoped by `userUid`
Admin bootstrap	`pages/AdminSetup.jsx`	`adminService.bootstrapAdmin` `refreshProfile`	`POST /auth/bootstrap-admin`	MONGO users	`x-admin-secret` header + valid token
Admin Collection	`admin/AdminCollectionTab.jsx`	`adminService.booksAdmin`	`POST/PUT/DELETE /books`	MONGO books	`requireRole("admin")`
Admin Store	`admin/AdminStoreTab.jsx`	`adminService.storeBooksAdmin`	`POST/PUT/DELETE /store/books`	FIRE storeBooks	`requireRole("admin")`
Admin Members	`admin/AdminMembersTab.jsx`	`adminService.fetchMembers`	`admin.js` → `listMembers`	MONGO users + borrows	`router.use(verify, requireRole("admin"))`
Statistics	AdminCollectionTab	`services/api` direct	`routes/stats.js`	both	**Public endpoint**; only the UI is gated
Route protection	`components/ProtectedRoute.jsx`	`useAuth`	—	—	**UX only** — real enforcement is server-side

SECTION TWENTY-FIVE

Potential Interview Questions

Eighty questions grounded in the actual implementation — 30 basic, 30 intermediate, 20 advanced. Each with the answer, why it is correct, and the code that proves it.

Part A — 30 Basic Questions

1 What is Bookverse and what problem does it solve?

A full-stack library platform with three shelves: an official Collection you borrow or buy with points, a Community shelf where members lend each other used books, and a Store selling new books for real money. A contribution-point economy ties them together — sharing books earns points, points buy ownership.

CODE — three shelves map to three data areas: `Book` / `Borrow` in MongoDB, `usedBooks` / `borrowRequests` in Firestore, and `storeBooks` / `carts` / `orders` in Firestore.

2 What is the tech stack?

React 19 + Vite 8 + Tailwind 3 + Framer Motion + React Router 7 + Axios on the frontend. Node + Express 4 on the backend. MongoDB via Mongoose 8 and Cloud Firestore for data. Firebase Authentication for identity, Firebase Admin SDK server-side, Stripe for payments. Deployed on Vercel.

WHY CORRECT — every item appears in the two `package.json` files. There is deliberately no Stripe browser SDK and no Cloud Functions.

3 What does MERN stand for, and is Bookverse a MERN app?

MongoDB, Express, React, Node. Yes — with the addition of Firestore alongside MongoDB and Firebase Auth replacing hand-rolled JWT authentication.

CODE — commit `c1c4113` migrated from JWT/bcrypt to Firebase Auth, dropping `bcryptjs` and `jsonwebtoken`.

4 What is the difference between the frontend and backend here?

The frontend is a React SPA served as static files; it renders UI and holds no secrets. The backend is an Express JSON API holding all business rules, all authorisation and all credentials. They are separately deployed and communicate only over HTTP.

CODE — `backend/vercel.json` routes everything to a serverless function; `frontend/vercel.json` rewrites all paths to `index.html`.

5 What is a React component?

A function that returns JSX describing UI. Bookverse has 25 page components and 18 shared ones, all function components using hooks.

CODE — `BookCard.jsx` takes `{ book, index }` and returns a card.

6 Props vs state?

Props are inputs passed from a parent and are read-only inside the component. State is owned and updated by the component itself.

CODE — `ReviewSection` receives `source` and `bookId` as props, but owns `rating`, `text` and `submitting` as state.

7 What does `useState` do?

Adds state to a function component, returning the current value and a setter. Calling the setter schedules a re-render.

CODE — `const [loading, setLoading] = useState(true)` starts `true` so the first render shows a skeleton, not an empty state.

8 What does `useEffect` do?

Runs side effects after render — fetching, subscribing, DOM listeners. The dependency array controls when it re-runs, and the returned function cleans up.

CODE — `useEffect(() => { fetchBooks(); }, [search, author, category, sort])` in Collection.jsx re-fetches when any filter changes.

9 What is the Context API and why use it here?

A way to share values down the tree without passing props through every level. Bookverse uses three contexts: Auth, Cart, Wishlist.

WHY CORRECT — `firebaseUser` is needed by the navbar, route guards and deep card components; prop-drilling it would touch every intermediate component.

10 Why not Redux?

Only three pieces of global state and no complex derivation. Context is built in and sufficient. Redux would earn its place with heavy cross-slice logic or time-travel debugging needs.

WHY CORRECT — the largest context, `AuthContext`, is 148 lines including all auth actions.

11 What is React Router used for?

Client-side routing — 25 routes mapping paths to page components without full page reloads, plus route params, programmatic navigation and query-string state.

CODE — `App.jsx` declares the route table; `useParams`, `useNavigate`, `useSearchParams` and `useLocation` are all used.

12 What is a protected route?

A route that redirects unauthenticated (or unauthorised) users. `ProtectedRoute` wraps six routes; passing `roles={["admin"]}` additionally requires the admin role.

CODE — `components/ProtectedRoute.jsx`. Crucially it is UX only; real enforcement is server-side.

13 What is Express?

A minimal Node web framework for routing and middleware. Bookverse mounts 13 routers covering 46 endpoints.

CODE — `backend/app.js`, thirteen `app.use("/api/...", router)` lines.

14 What is middleware?

A function receiving `(req, res, next)` that runs before the handler. It can enrich the request, end it, or pass control on.

CODE — `verifyFirebaseToken` attaches `req.user`; `express.json()` attaches `req.body`.

15 What is a REST API?

Resource-oriented URLs with HTTP verbs carrying meaning, JSON payloads, meaningful status codes, and a stateless server.

CODE — `GET/POST/PUT/DELETE /api/store/books`. Note some endpoints are RPC-style actions (`/borrow`, `/respond`) — a pragmatic deviation.

16 Explain 200, 201, 400, 401, 403, 404, 500.

200 OK; 201 Created; 400 bad input; 401 not authenticated; 403 authenticated but not permitted; 404 not found; 500 server error.

CODE — Bookverse uses all of them, plus 503 when `STRIPE_SECRET_KEY` is missing.

17 401 vs 403?

401 = "I don't know who you are." 403 = "I know who you are and you still may not do this."

CODE — a bad token gives 401 from `verifyFirebaseToken`; a member hitting an admin route gives 403 from `requireRole`.

18 What is MongoDB, and what does Bookverse store there?

A document database. Bookverse stores three collections: `users` (identity + role), `books` (the official Collection), `borrows` (loan records).

CODE — `backend/models/` contains exactly those three schemas.

19 What is Mongoose?

An ODM for MongoDB providing schemas, validation, typed models and query helpers.

CODE — `role: { type: String, enum: ["member","admin"], default: "member" }` enforces valid roles at the data layer.

20 What is an ObjectId?

MongoDB's 12-byte primary key, encoding a timestamp, machine id, process id and counter — so it is roughly sortable by creation time.

CODE — `Borrow.userId` and `Borrow.bookId` are `ObjectId` references.

21 What is Firestore and what is stored there?

A NoSQL document database with real-time listeners and declarative security rules. Bookverse stores ten collections: points, community books, borrow requests, notifications, reviews, purchases, wishlist, store books, carts and orders.

CODE — the `COLLECTIONS` map in `backend/lib/firestore.js`.

22 What is Firebase Authentication?

Google's managed identity service. It handles password storage, Google OAuth, session persistence and token issuance. Bookverse's server never sees a password.

CODE — `createUserWithEmailAndPassword`, `signInWithEmailAndPassword`, `signInWithPopup` in `AuthContext.jsx`.

23 What is Axios and why not `fetch`?

A promise-based HTTP client. Chosen for automatic JSON parsing, interceptors, and because non-2xx responses reject rather than resolve.

CODE — the request interceptor in `api.js` is the decisive reason; `fetch` has no equivalent.

24 What is Stripe used for?

Store payments via Stripe-hosted Checkout. The server creates a session with its own prices and redirects the browser to Stripe.

CODE — `checkoutController.createCheckoutSession`; there is no card form in this codebase.

25 How does a user become an admin?

By visiting `/admin/setup` while signed in and entering the admin key, which is compared server-side against `ADMIN_SECRET`. Their MongoDB role is set to `admin`.

CODE — `authController.bootstrapAdmin`, reading `req.headers["x-admin-secret"]`.

26 How many points does a contribution earn, and what does a book cost?

50 per contribution; 200 to own an official book. Four shares buy one book.

CODE — `POINTS_PER_CONTRIBUTION = 50`, `POINTS_TO_PURCHASE = 200` in `lib/firestore.js`, served to the client via `/points/me`.

27 What currency does the Store use?

Bangladeshi Taka. Prices are stored as plain numbers with `currency: "BDT"`, formatted by the `taka()` helper, and converted to poisha for Stripe.

CODE — `toStripeAmount = (taka) => Math.round(Number(taka) * 100)`.

28 What is Tailwind CSS?

A utility-first CSS framework. Bookverse extends it with a custom palette (navy, coral, indigo, cream) and two fonts, Fraunces and Manrope.

CODE — `tailwind.config.js` `theme.extend`.

29 What is Vite?

The build tool and dev server — ES-module based, with fast HMR. It also supplies `import.meta.env` for configuration.

CODE — every `VITE_*` variable is read that way; only `VITE_`-prefixed values are exposed to the bundle.

30 Where is the app deployed?

Vercel — the frontend as static files, the backend as a serverless function.

CODE — `backend/api/index.js` exports the app without calling `listen()`; `server.js` exists for local development.

Part B — 30 Intermediate Questions

31 How does Firebase authentication reach Express?

Firebase issues an ID token (a JWT signed by Google). The Axios request interceptor waits for `authStateReady()`, calls `getIdToken()` and sets `Authorization: Bearer <token>`. Express strips the prefix and calls `admin.auth().verifyIdToken(token)`, which validates the signature, expiry, issuer and audience. It then looks up the MongoDB user by `firebaseUid` and attaches it as `req.user`.

CODE — `services/api.js` + `middleware/auth.js`. This is the single most likely question to be asked; §22.2–22.3 has it line by line.

32 Why did you use Firestore *and* MongoDB?

MongoDB holds the core domain — users, books, loans — where I want a strict schema, relational references and expressive queries. Firestore holds the social and transactional data because it offers two things MongoDB can't: real-time listeners for the notification bell, and per-document security rules enforced by the database. Honestly, the boundary also reflects chronology — the project started MongoDB-only and adopted Firestore with the Firebase Auth migration.

CODE — the comment in `wishlistController.js` names the convention: *"per the project's 'new features use Firestore' rule."*

33 Why are Firestore client writes disabled?

Because rules can express ownership but not business rules. They cannot express "points may only increase by exactly 50, and only when a used book was created in the same operation." Since a server is needed for those cases anyway, routing all writes through it avoids maintaining two authorisation systems that could disagree.

CODE — every collection has `allow write: if false`, and the Admin SDK bypasses rules.

34 How is the Stripe price protected from frontend manipulation?

The cart stores only `{ bookId, quantity }`. On checkout, `buildCart()` re-reads every price from the Firestore catalogue and those values become `unit_amount`. The checkout request body is literally empty — there is no price parameter anywhere in the API to tamper with.

CODE — `createCheckoutSession` calls `buildCart(req.user.firebaseUid)`; the comment says *"the client never gets to say what anything costs."*

35 How does the cart avoid trusting stale prices?

It never stores a price. Every cart operation calls `buildCart()`, which re-reads the catalogue and recomputes line totals and the subtotal. A cart left open for a week reflects today's prices, and the same function is used by checkout — so the displayed total and the charged total cannot diverge.

CODE — `cartController.buildCart`, imported by `checkoutController`.

36 How does the owner of a community book receive a borrow notification?

Inside `requestBorrow`, after creating the request, the server calls `notify(book.ownerUid, {...})`, which writes a document to `notifications` with `userUid` set to the owner. The owner's browser holds an open `onSnapshot` listener on `notifications where userUid == myUid`, so Firestore pushes the new document down that connection and the bell badge updates within a second. **No polling and no Cloud Function are involved.**

CODE — `communityController.notify` + `NotificationBell.jsx`. Correcting the Cloud Functions assumption is a strong moment.

37 Where does the community-book search happen?

In Node.js memory on the server. Firestore fetches all used books ordered by date, then `Array.filter` applies the text filters — because Firestore has no case-insensitive substring operator. It is filtered on the server, so the client still receives only matches.

CODE — the comment in `listUsedBooks`: *"Firestore has no case-insensitive contains, and the community shelf is small."*

38 And where does the Collection search happen?

Inside MongoDB. The parameters are compiled into a query object using `$regex` with the `i` option, and `.sort()` is applied by the database engine. Nothing is filtered in JavaScript.

WHY IT MATTERS — being able to contrast this with Q37 shows you understand the engines rather than having copied a pattern.

39 How do you prevent duplicate reviews?

Before inserting, a Firestore query with three equality filters — `source`, `bookId`, `userUid` — checks for an existing review and returns 400 if found.

HONEST ADDENDUM — it is read-then-write, not transactional. A deterministic document id like `` `${source}_${bookId}_${userUid}` `` with `create()` would make the database enforce it.

40 How does the system prevent unauthorised return of another user's book?

Two different mechanisms. For official books, the query itself is the authorisation — `Borrow.findOne({ bookId, userId: req.user._id, status: "borrowed" })` simply cannot match someone else's loan, so it 404s. For community books there is an explicit check: `if (request.requesterUid !== req.user.firebaseUid) return 403`, plus the status must be `approved`.

CODE — `routes/books.js` return handler and `communityController.returnUsedBook`.

41 How does the admin system prevent members from modifying official books?

Every write endpoint carries `verifyFirebaseToken, requireRole("admin")`. The role comes from MongoDB, resolved from a cryptographically verified token. Hiding the admin link in React is cosmetic; the middleware is what actually enforces it.

DEMONSTRATION — forge `profile.role = "admin"` in DevTools and the dashboard renders, but every privileged request returns 403.

42 Why store the role in MongoDB rather than a Firebase custom claim?

Immediate revocation. A custom claim lives inside the token, so a demoted admin keeps their privileges until the token refreshes — up to an hour. A database lookup reflects the change on the very next request. The usual advantage of claims — letting Firestore rules check the role — is irrelevant here because all client writes are denied anyway.

COST — one extra Mongo lookup per authenticated request, which is why `firebaseUid` is indexed.

43 What does `authStateReady()` do and why is it necessary?

It resolves once Firebase has finished restoring the persisted session. Without it, on a hard page refresh a mounting component's first requests fire before restoration completes, `auth.currentUser` is null, no token is attached, and they all 401 — despite a perfectly valid session. The symptom is "works when I navigate, 401s when I refresh."

CODE — `if (auth) await auth.authStateReady();` in `api.js`, with a comment explaining exactly this.

44 Why does `ProtectedRoute` show a spinner while loading?

Because on a refresh `firebaseUser` is briefly null for a signed-in user. Redirecting during that window would log people out on every refresh.

CODE — `if (loading) return <Spinner />` is the first statement in the component.

45 Why does `ProtectedRoute` preserve the query string?

Because Stripe returns the shopper to `/cart?session_id=…`. If the guard captured only the pathname, a user whose session hadn't restored would be bounced to login and lose the session id — leaving the payment taken but the order never recorded.

CODE — `const from = \`${location.pathname}${location.search}\``, with a comment naming the Stripe case. Fixed in commit `72a88fb`.

46 How are duplicate orders prevented?

Three layers, only one of which is authoritative: the server queries `orders` by `stripeSessionId` before inserting and returns the existing order with `alreadyRecorded: true` if found. The client also uses a `useRef` latch and strips `session_id` from the URL — both are polish.

CODE — `confirmCheckout`, comment: "*Reloading the success page must not create a second order.*"

47 Why `useRef` rather than `useState` for the checkout latch?

State updates are asynchronous and trigger re-renders. Under StrictMode the effect runs twice before any re-render, so a state flag would still read `false` the second time. `ref.current = true` takes effect immediately and causes no render.

CODE — `const confirmed = useRef(false)` in `Cart.jsx`.

48 Why is the URL used as state on listing pages?

Filtered views become shareable and bookmarkable, the back button undoes a filter, refresh preserves the view, and there is one source of truth so inputs and results can never disagree. Changing the URL is what triggers the fetch, via the effect's dependency array.

TRADE-OFF — no debounce, so typing fires a request per keystroke. Fine at this scale; I'd add a 300 ms debounce before real traffic.

49 Why does the cart API return the whole cart on every mutation?

So the client never computes money. Each context method is one line — `setCart(await api.addToCart(...))` — and the displayed total is always exactly what the server would charge.

CODE — all five cart endpoints end with `res.json(await buildCart(uid))`.

50 What is `FieldValue.increment()` and why use it?

An atomic server-side arithmetic operation. Firestore performs the addition itself, so two concurrent contributions both count. A read-modify-write in Node would lose one of them.

CODE — `points: FieldValue.increment(POINTS_PER_CONTRIBUTION)`.

51 What does `{ merge: true }` do?

Makes `.set()` behave as an upsert that preserves existing fields. A first-time contributor has no `users` document, so merge creates it; an existing user keeps their other fields.

CODE — used for points and for carts.

52 Batch vs transaction in Firestore?

A batch is a set of blind writes committed atomically (up to 500) — used for mark-all-read and wishlist deletion. A transaction reads first, then writes based on what it read, retrying on conflict — used for spending points, where the balance must be re-read safely.

RULE OF THUMB — if the write depends on a value you read, you need a transaction.

53 Why does `populate()` sometimes return null, and how is that handled?

Because `populate` is a second query, not a database join — a deleted target yields null. Bookverse deliberately deletes books bought with points, so borrow records carry a `bookSnapshot` taken at loan time. The fallback reshapes the snapshot to look exactly like a populated book, so the UI needs no special case.

CODE — `routes/borrow.js` `withFallback`, plus `bookGone` so the UI drops the dead link.

54 Why is a book deleted when bought with points rather than flagged?

Because it never comes back. A borrowed book is still the library's, so it stays marked unavailable. A sold copy leaves permanently, so keeping it would mean it still appeared in listings and admin tables. The buyer's purchase record is the only trace.

CODE — the comment in `models/Book.js`. This replaced an earlier `purchased` flag — see §28.3.

55 What does `preserveReferences()` do?

Runs before the delete and handles three kinds of dangling reference differently: borrow records and reviews get a snapshot of the title/author/cover (historical records must survive), while wishlist entries are deleted outright (a forward-looking intention that is now impossible).

CODE — `pointsController.preserveReferences`, touching both databases.

56 Why does the review system need a `source` field?

Because a MongoDB ObjectId and a Firestore auto-id come from different id spaces and could collide. Every review carries the composite key `(source, bookId)`, which is why the endpoint is `/api/reviews/:source/:bookId`. The wishlist does the same.

CODE — `const SOURCES = ["official", "community"]`, validated as a whitelist on both endpoints.

57 How is the average rating computed?

On demand with `reduce` over the fetched reviews, rounded to two decimals with `Number(average.toFixed(2))`. It is not stored on the book.

TRADE-OFF — always correct and never needs maintenance, but recomputed per page view. At scale you'd keep a running sum and count updated in a transaction.

58 Why is the wishlist add endpoint idempotent?

So a double-click or a retried request can't create duplicates. If an entry already exists it returns it with 200 instead of creating another with 201.

CODE — `if (!existing.empty) return res.json({...})`. Deletion is keyed on `(source, bookId)` so the client needn't track document ids.

59 Why is the wishlist optimistic but the cart is not?

A wrong heart for 200 ms is harmless; a wrong price or quantity is not. Optimism is applied only where being briefly wrong costs nothing.

CODE — `WishlistContext.toggle` updates local state first and calls `load()` to resync on failure.

60 Why must `refreshProfile()` be awaited after the admin bootstrap?

The server updated MongoDB but React's `profile` still says `member`. Navigating to `/admin` without refreshing would fail `ProtectedRoute`'s role check against stale state and bounce the new admin home.

CODE — `await bootstrapAdmin(...); await refreshProfile(); navigate("/admin")`.

Part C — 20 Advanced / Debugging / Security Questions

61 You have a book in MongoDB and points in Firestore. How do you make a purchase atomic?

You can't — no transaction spans two databases. I used a **saga with a compensating transaction**: claim the copy with one atomic conditional update (`findOneAndUpdate({ _id, available: true })`), then debit points inside a Firestore transaction, and if that fails call `releaseClaim()` to put the copy back. Only after a successful payment is the book snapshotted and deleted.

CODE — `pointsController.purchaseOfficialBook`. The comment states the constraint explicitly. This is the strongest architectural story in the project.

62 Why `findOneAndUpdate` rather than checking `available` and then saving?

Check-then-act is two operations with a gap. Two concurrent buyers could both pass the check and both proceed. `findOneAndUpdate({ _id, available: true }, { $set: { available: false } })` is a single atomic operation — MongoDB serialises them, the second one's filter no longer matches, and it receives `null`.

BONUS HONESTY — the *borrow* path still uses check-then-act and has this exact race. I know the fix because the purchase path already demonstrates it.

63 What is the biggest security risk in this app and how is it mitigated?

Home addresses and phone numbers in `borrowRequests`. Mitigated by the only two-party read rule in the file — `request.auth.uid == resource.data.ownerUid || == resource.data.requesterUid` — plus the fact that all writes are denied, so no one can insert themselves into a request.

CAVEAT I'D VOLUNTEER — the UI only reveals the address after approval, but the API returns it for pending requests too. Proper enforcement would strip it server-side until approved.

64 Walk me through what happens if an attacker sets `profile.role = "admin"` in DevTools.

`ProtectedRoute` passes and the dashboard renders. `/api/stats` is public so numbers appear. But `/admin/members` returns 403, and every create/edit/delete returns 403 — because `requireRole` reads the role from MongoDB using an identity proven by a Google-signed token. They get an empty shell.

THE FRAMING — client guards are UX, server middleware is security, and I know which one is load-bearing.

65 What happens if a user pays on Stripe and then closes the tab?

Stripe has the money and Bookverse has no order — the cart isn't even emptied. Orders are recorded only when the browser returns and calls `confirm`. The fix is a Stripe webhook on `checkout.session.completed`, verified with `stripe.webhooks.constructEvent` against the raw body. The existing `stripeSessionId` idempotency check already makes the webhook and the browser confirmation safe to both run — whichever arrives first wins.

VOLUNTEER THIS. It is the most important production gap, and the architecture is already webhook-ready.

66 Google sign-in leaves users on the login page. How would you debug it?

Check whether Firebase actually has a session (`auth.currentUser` in the console) versus whether the app noticed. If Firebase has no session after returning from Google, the redirect handshake failed. The cause in our case was third-party storage partitioning: `signInWithRedirect` completes through an iframe on the Firebase auth domain, which Chrome 115+, Firefox 109+ and Safari 16.1+ block. The fix was to reverse-proxy `/__/auth/*` to the Firebase auth domain and point `authDomain` at our own site, making the handler first-party.

CODE — `frontend/vercel.json` rewrite; commits `96b2f49` and `1695aa7`. Full story in §28.1.

67 Requests work when navigating but 401 after a refresh. Diagnose it.

A race between Firebase's asynchronous session restoration and the component's first fetch. On refresh, `auth.currentUser` is still null when the interceptor runs, so no token is attached. The fix is `await auth.authStateReady()` at the top of the interceptor.

WHY THE SYMPTOM IS DIAGNOSTIC — navigating works because auth resolved long ago; only a cold load exposes the gap.

68 A user's contributions page is always empty. What would you check?

Route ordering. `/books/mine` must be registered *before* `/books/:id`, or Express matches the dynamic route first with `id === "mine"`, looks up a document literally named "mine", and 404s.

CODE — both `community.js` and `reviews.js` carry comments explaining the ordering, because the code otherwise looks arbitrary.

69 A reader's history shows "Untitled" with a broken cover. Why?

The book was bought with points and deleted, so `populate("bookId")` returned null. Fixed by `bookSnapshot` — a copy of title/author/cover stored on the borrow record at loan time, reshaped at read time to look like a populated book so the UI needs no special case.

CODE — `models/Borrow.js` comment says exactly this; the same bug class hit reviews ("Unknown book") and was fixed the same way.

70 How would you scale the Store search past a few thousand books?

Today it fetches everything and filters in Node — fine for 24 books, hopeless at 10,000. Options: denormalise a lowercased search field and use Firestore range queries for prefix search; add `firestore.indexes.json` with composite indexes for the category × rating × sort combinations; or move search to Algolia or Elasticsearch. I'd also add pagination, which the app currently has none of.

WHY THE CURRENT DESIGN IS DEFENSIBLE — the constraint is real (Firestore can't combine a range filter with arbitrary ordering without composite indexes) and the trade-off was made knowingly.

71 Is `{ $regex: search }` a security problem?

It is not injection — Mongoose passes it as a value, not as code. But it is a potential **ReDoS** vector: a crafted pattern with catastrophic backtracking could pin the CPU. The fix is to escape regex metacharacters before building the query, or use a text index instead.

HONESTY — naming the correct threat model (ReDoS, not injection) is more impressive than either ignoring it or overstating it.

72 Why is exposing the Firebase API key in the bundle not a vulnerability?

A Firebase web API key is an identifier, not a credential — it tells Google which project a request belongs to and grants no data access. Access is decided by Firebase Authentication and by `firestore.rules`. The dangerous credential is `FIREBASE_PRIVATE_KEY`, which is server-only and deliberately not `VITE_`-prefixed.

CODE — `frontend/.env.example` says *"These are public by design"*; `backend/.env.example` says *"SERVER-SIDE ONLY. Never expose these through VITE_* variables."*

73 Critique the admin bootstrap endpoint.

Strengths: it still requires a valid token, so the secret alone is useless; it converts a shared secret into durable, attributable state so every later action is tied to a named account; the secret never leaves the server; it fails closed if `ADMIN_SECRET` is unset; and it's idempotent. Weaknesses: no rate limiting, so it's brute-forceable; a plain `!==` rather than a constant-time compare; and it never disables itself after the first promotion.

WHY THIS ANSWER WORKS — it shows you designed it deliberately *and* can critique it.

74 Is CORS being wide open a problem?

It should be scoped to `CLIENT_URL`. But the severity is lower than it looks: authentication is a bearer token attached by JavaScript, not a cookie, so the browser never sends credentials automatically — which means CSRF doesn't apply. A malicious site can call the API, but without a valid token it gets 401.

CODE — `app.use(cors())` with no options. The fix is one line.

75 How would you add real-time updates to the community shelf?

The infrastructure already exists — `firestore.rules` allows public reads on `usedBooks`, and the client SDK is already initialised. I'd replace the one-shot fetch in `Community.jsx` with an `onSnapshot` listener, exactly like `NotificationBell`. Availability would then flip live when someone's request is approved. The catch is that search would have to move fully client-side, since the server currently does the filtering.

WHY IT'S A GOOD ANSWER — it identifies the enabler, the pattern to copy, and the trade-off.

76 Two users click "buy with points" on the same book simultaneously. What happens?

Both reach `findOneAndUpdate({ _id, available: true })`. MongoDB serialises them; the first flips `available` to false and gets the document; the second's filter no longer matches and it gets `null`, so it returns 400 "This book was just taken by someone else." Only the winner proceeds to spend points.

AND IF THE WINNER HAS INSUFFICIENT POINTS? The transaction returns `{ ok: false }`, `releaseClaim()` restores `available: true`, and the book is buyable again.

77 Where would you add tests first, and what kind?

There are currently none. I'd start with integration tests on the highest-risk paths: the point-purchase saga (including the concurrent-buyer and insufficient-funds compensation cases), the Stripe confirmation guards, and the ownership checks on community requests. Then unit tests for `buildCart`, since it is the price-integrity boundary. UI tests last — the business rules are where the real risk lives.

WHY THIS ORDERING — test where a bug costs money or leaks data, not where it costs a misaligned button.

78 What are the weakest parts of this architecture?

Five, in order: (1) no Stripe webhook, so a closed tab loses an order; (2) the duplicate `users` collection across two databases; (3) the check-then-act race in the borrow path; (4) two independent rating systems that are never reconciled — a review never changes `Book.rating`; (5) no pagination anywhere, so every list endpoint returns everything.

WHY VOLUNTEER THIS — an engineer who can rank their own weaknesses is far more credible than one who claims none.

79 If you rebuilt Bookverse, what would you change?

I'd consolidate `storeBooks` into MongoDB, because the Store needs exactly the rich filtering MongoDB does well and gains nothing from real-time — that alone would fix the in-memory filtering. I'd move `points` onto the Mongo user document to kill the duplicate collection. I'd keep notifications, borrow requests and reviews in Firestore, since real-time and per-document rules genuinely earn their place there. And I'd extract `bookController` and `borrowController` so the routing layer is uniform.

WHY IT WORKS — it keeps what's justified and changes what's historical, which shows judgement rather than fashion.

80 Explain the whole architecture in thirty seconds.

"React SPA talking to an Express API over HTTPS. Firebase Authentication owns identity — the browser gets an ID token, an Axios interceptor attaches it as a bearer header, and Express verifies it with the Admin SDK, then resolves a MongoDB user that carries the role. MongoDB holds users, books and loans; Firestore holds points, community books, requests, notifications, reviews, the store, carts and orders. The browser has one direct, read-only Firestore connection for live notifications; every write goes through Express using the Admin SDK, because `firestore.rules` denies all client writes. Stripe handles payments with prices computed server-side from our own catalogue. The rule underneath all of it: the client is never trusted with a decision, an identity, or a price."

PRACTISE THIS ONE OUT LOUD. It is the answer that frames every follow-up question.

SECTION TWENTY-SIX

Accuracy Register — What Is and Is Not Implemented

A single place to verify every claim in this document, and an explicit list of things that do not exist in the codebase.

WHY THIS SECTION EXISTS

The worst outcome in an interview is confidently describing a feature your code does not have. This register is the defence: everything in the left column is verifiable in the repository, and everything in the "not implemented" list should be described as future work, never as shipped.

26.1 Confirmed present

CLAIM	VERIFIED IN
46 REST endpoints across 13 routers	`backend/routes/*.js`, mounted in `app.js`
Two databases in active use	`config/db.js` (Mongo) and `lib/firestore.js` (Firestore)
3 Mongo models, 10 Firestore collections	`models/` and the `COLLECTIONS` map
Firebase Auth with email/password + Google	`AuthContext.jsx`
Popup sign-in with redirect fallback	`loginWithGoogle` + `POPUP_ENV_FAILURES`
First-party auth-handler proxy	`frontend/vercel.json` rewrite of `/__/auth/:path*`
Token verification + Mongo user resolution	`middleware/auth.js`
RBAC with two roles	`requireRole`, gating 9 endpoints
Admin bootstrap via shared secret	`authController.bootstrapAdmin`
Client-read-only Firestore rules	`firestore.rules` — `allow write: if false` ×10
Real-time notifications via `onSnapshot`	`NotificationBell.jsx`
Points economy: 50 earn / 200 spend	`lib/firestore.js` constants
Cross-database purchase saga	`pointsController.purchaseOfficialBook`
Firestore transaction on points	`db.runTransaction` in the same function
Atomic conditional claim	`Book.findOneAndUpdate({ _id, available: true })`
Snapshot preservation before delete	`preserveReferences()`
Server-side cart pricing	`cartController.buildCart`
Stripe hosted Checkout	`checkoutController` + `lib/stripe.js`
Order idempotency by session id	`confirmCheckout` existence query
One review per user per book	`reviewController.addReview` duplicate query
Idempotent wishlist add	`wishlistController.addToWishlist`
Shipping captured in 3 flows	`ShippingModal` + 3 server-side validations
Serverless-ready backend split	`app.js` / `server.js` / `api/index.js`

26.2 Explicitly NOT implemented

PLANNED OR ASSUMED, BUT ABSENT FROM THE CODEBASE

NOT PRESENT	HOW TO DESCRIBE IT HONESTLY
Firebase Cloud Functions	No `functions/`, no `firebase.json`. Notifications are written inline by Express, and delivered by a client listener. **Never claim a Function exists.**
Stripe webhooks	Not implemented. The architecture is webhook-ready thanks to the existing idempotency check — describe it as the top production gap. (§16.9)
Pagination	No page/limit/skip anywhere. Every list endpoint returns all matches.
Related / recommended books	No endpoint, no component.
Editing or deleting a review	Reviews are write-once.
Shipping on Store orders	Stripe collects an email only; no address is gathered.
Order status progression	The field exists but is only ever `"paid"`.
Admin control of community books	Deliberately absent — owners control their own contributions.
Promoting another user via the UI	No endpoint. Requires the secret or direct DB access.
Password reset / profile editing	No UI, though Firebase supports reset.
Image upload	Covers are user-supplied URLs; no Firebase Storage flow.
Tests	No test framework, no test files. (§25 Q77 covers where to start.)
Rate limiting	None on any endpoint.
Firestore composite indexes	No `firestore.indexes.json` — queries are kept simple to avoid needing them.
Stock quantity in the Store	No field; "In stock" is hard-coded and the cart has no upper bound.
Debounced search	Typing fires a request per keystroke.
Response interceptor / global 401 handling	Only a request interceptor exists.

26.3 Known inconsistencies within the codebase

INCONSISTENCY	EXPLANATION
4 routers keep logic inline	`books.js`, `borrow.js`, `stats.js`, `notifications.js` predate the controllers folder, which arrived with the Firebase migration.
Two error-handling styles	Newer controllers use `next(err)`; older routers catch locally and leak `error: err.message`.
No `bookService.js`	Five components import `api` directly for `/books` and `/borrowed` — the same historical reason.
Category lists differ	The Store computes categories from data; Collection and Community use a hard-coded array of eight.
Two rating systems	`Book.rating` (Mongo, seeded) is never reconciled with the Firestore review average.
Two `users` collections	Mongo holds role; Firestore holds points. Joined only by UID. (§19.6)
Borrow uses check-then-act	The purchase path is race-safe; the borrow path is not. (§8.7)

SECTION TWENTY-SEVEN

How to Use This Document

A revision plan, the ten things worth memorising, and the traps to avoid.

27.1 A four-pass revision plan

PASS	READ	GOAL
1 — Orientation	§1, §2, §19	Be able to draw the architecture and name what lives in each database, from memory.
2 — Mechanism	§3–§6, §22	Trace a request end to end without looking. Know `api.js` and `verifyFirebaseToken` line by line.
3 — Features	§7–§18	For each feature, name the page, the service, the endpoint, the controller, the database and the guard.
4 — Defence	§5, §20, §25, §28	Answer security and debugging questions, and volunteer your own weaknesses before being asked.

The evening before: read §24 (the feature/file map), §23 (the flow table), and the Top 10 problems in §28. Say the thirty-second architecture answer (§25 Q80) out loud until it is fluent.

27.2 The ten things worth memorising

1. **The auth chain** — Firebase issues → interceptor attaches → `verifyIdToken` verifies → Mongo lookup → `req.user`.
2. **Why `authStateReady()` exists** — the refresh race that silently 401s.
3. **Why Firestore client writes are all denied** — rules can express ownership, not business rules.
4. **How prices are protected** — the cart stores only ids and quantities; `buildCart` re-reads the catalogue.
5. **The three Stripe guards** — ownership, payment status, idempotency.
6. **The purchase saga** — atomic claim → transaction → compensate → snapshot → delete.
7. **How notifications arrive live** — server writes a document; the client's `onSnapshot` is pushed to. No Cloud Functions.
8. **Client guards are UX; server middleware is security** — with the DevTools demonstration.
9. **Why `populate` returns null** — and how `bookSnapshot` fixes it.
10. **Why both databases** — history plus genuine capability differences, stated honestly.

27.3 Traps to avoid

DO NOT SAY	SAY INSTEAD
"A Cloud Function sends the notification."	"Express writes the notification document; the client's Firestore listener receives it."
"Stripe webhooks confirm the order."	"Confirmation happens on the browser's return. A webhook is the production gap I'd close first."
"The role is in the token."	"The role is in MongoDB, looked up per request — so revocation is immediate."
"Protected routes secure the admin panel."	"They hide it. `requireRole` secures it."
"Search is done in the database."	"For the Collection, yes — MongoDB. For the Store and Community, in Node memory, because Firestore has no case-insensitive contains."
"The rating updates when you review."	"There are two independent rating systems, and they're not reconciled — a known simplification."
"It's fully tested."	"There are no tests. I'd start with the purchase saga and the Stripe guards."

27.4 If you only remember one sentence

THE THESIS OF THE WHOLE SYSTEM

Firebase says who you are; MongoDB says what role you have; Express decides what you may do; the databases store the result; and the browser is never trusted with any of those decisions — not with an identity, not with a permission, and not with a price.

SECTION TWENTY-EIGHT

Problems, Bugs, Debugging & Solutions

The real defect history of this project, reconstructed from commit messages, diffs and the explanatory comments left in the code.

EVIDENCE STANDARD FOR THIS SECTION

Every problem below is supported by a **commit message, a diff, or a comment written into the source specifically to explain it**. Nothing here is inferred from "this looks like it might have been a bug." Where the original broken code is no longer recoverable from history, the entry is labelled "*Previously encountered — exact original code not available.*"

28.1 Google sign-in stranded users on the login page

Evidence: commits `96b2f49` and `1695aa7`, plus the surviving comment block in `AuthContext.jsx`. This bug took **two commits and a reversal** to fix properly, which is what makes it the most instructive story in the project.

Original problem	Google sign-in never completed.
Observed behaviour	The user clicked "Continue with Google", authenticated successfully at Google, was returned to the site — and was **still logged out**, sitting on the login form with no error message. Signed in at Google, signed out of Bookverse.
Feature affected	Google authentication on both Login and Register.
Root cause	**Third-party storage partitioning.** `signInWithRedirect` completes its handshake through a cross-origin iframe hosted on the Firebase auth domain (`*.firebaseapp.com`). Chrome 115+, Firefox 109+ and Safari 16.1+ all block storage access for that iframe because it is third-party relative to the site. The session token was written into storage the site could never read back.
Why it appeared at all	`signInWithPopup` was failing first for an environmental reason (popup blocker or storage partitioning), and the code fell back to `signInWithRedirect` . As the commit message puts it: the fallback "*could therefore never rescue a failed popup; it only turned a visible error into a silent one.*"
Files responsible	`frontend/src/context/AuthContext.jsx` , and — as it turned out — the **absence** of a rewrite in `frontend/vercel.json` .

Attempt 1 — remove the fallback (commit `96b2f49`)

BEFORE

```
const POPUP_ENV_FAILURES = new Set([...]);
async function loginWithGoogle() {
  try { return (await signInWithPopup(auth, provider)).user; }
  catch (err) {
    if (!POPUP_ENV_FAILURES.has(err?.code)) throw err;
    await signInWithRedirect(auth, provider);   ← silently fails
    return null;
  }
}
```

AFTER ATTEMPT 1 — POPUP ONLY

```
// Popup only, deliberately. signInWithRedirect completes its handshake through
// a cross-origin iframe on the Firebase auth domain, and Chrome 115+,
// Firefox 109+ and Safari 16.1+ all block that third-party storage...
async function loginWithGoogle() {
  const provider = new GoogleAuthProvider();
  const cred = await signInWithPopup(auth, provider);
  return cred.user;
}
```

Plus richer error messages in `authError.js`, so a blocked popup told the user to allow popups instead of dumping them back on the form.

This **stopped the silent failure** — a real improvement — but it left users in blocked-popup environments with no way to sign in with Google at all. The fix traded a silent bug for a hard limitation.

Attempt 2 — make the handler first-party (commit `1695aa7`) ✓ the real fix

DEBUGGING INSIGHT

The problem was never `signInWithRedirect` itself — it was **where the auth handler was served from**. If the handler runs on our own origin, its storage is first-party, and no browser blocks it. Firebase documents this exactly (*redirect-best-practices*), and the commit cites it.

AFTER — THE ACTUAL SOLUTION, IN TWO PARTS

```
// PART 1 — frontend/vercel.json: reverse-proxy the auth handler
{
  "rewrites": [
    { "source": "/__/auth/:path*",
      "destination": "https://bookverse-project-b6fc9.firebaseapp.com/__/auth/:path*" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}

// PART 2 — point authDomain at OUR site, not at *.firebaseapp.com
VITE_FIREBASE_AUTH_DOMAIN=<the Bookverse site's own domain>
```

With the handler now first-party, the redirect leg became reliable, so the popup fallback was **restored** — giving both paths:

```
async function loginWithGoogle() {
  const provider = new GoogleAuthProvider();
  try { return (await signInWithPopup(auth, provider)).user; }
  catch (err) {
    if (!POPUP_ENV_FAILURES.has(err?.code)) throw err;
    await signInWithRedirect(auth, provider);
    return null; page navigates to Google; nothing to return here
  }
}
```

THE DEPLOYMENT STEP THAT IS EASY TO FORGET

The commit message records it: "*Requires `https://…/__/auth/handler` to be an authorized redirect URI on the project's Google OAuth client.*" The code change alone is not sufficient — the OAuth client must accept the new origin.

Why the fixed implementation works — complete flow

1. User clicks "Continue with Google" → `signInWithPopup`.
2. Popup blocked → error code is in `POPUP_ENV_FAILURES` → `signInWithRedirect`. Returns `null`, so `handleGoogle` skips navigation.
3. Browser navigates to Google, user authenticates, Google redirects to `https://<our-site>/__/auth/handler`.
4. **The critical hop:** Vercel's rewrite proxies that path to the Firebase auth domain, but the browser only ever sees **our origin**. The handler's storage is therefore **first-party** and is not partitioned.
5. The session persists successfully. `onAuthStateChanged` fires with the restored user; `getRedirectResult` settles (and would surface any error into `redirectError`).
6. The effect in Login/Register — `if (!authLoading && firebaseUser) navigate(from, { replace: true })` — routes the user onward. The original click handler is long gone, which is exactly why this effect exists.

INTERVIEW LESSON

Browser privacy changes can break OAuth flows that worked for years. The concepts to name: third-party storage partitioning, first-party vs third-party context, and reverse-proxying an identity provider's handler onto your own origin. Also worth saying: the first fix removed the symptom, the second fixed the cause — and knowing the difference is the point.

28.2 Official Collection endpoints were completely unauthenticated

Evidence: commit `f9667c0`.

Problem	The commit states it plainly: *"The official Collection represents the library's own holdings, but its create/update/delete endpoints were unauthenticated — anyone who knew the URL could edit it."*
Behaviour	A single `curl -X DELETE /api/books/<id>` from anyone, signed in or not, would delete a book from the library.
Root cause	No authorisation layer existed. The `User` model had no `role` field at all, so there was nothing to check against.
Files	`routes/books.js`, `models/User.js`, `middleware/auth.js`

BEFORE

```
router.post("/", async (req, res) => { ... });          no guard
router.put("/:id", async (req, res) => { ... });        no guard
router.delete("/:id", async (req, res) => { ... });     no guard
```

AFTER

```
router.post("/",       verifyFirebaseToken, requireRole("admin"), handler);
router.put("/:id",     verifyFirebaseToken, requireRole("admin"), handler);
router.delete("/:id",  verifyFirebaseToken, requireRole("admin"), handler);

// models/User.js
role: { type: String, enum: ["member", "admin"], default: "member" }
```

Plus `requireRole`, the `bootstrap-admin` endpoint, the role-guarded `/admin` route, and `/admin/setup`.

Why it works: two independent checks in sequence — authentication proves identity, authorisation proves permission. **Lesson:** a write endpoint with no guard is a vulnerability, not a to-do. Also note the design choice recorded in the message: the secret promotes *an account*, so every later action is attributable rather than "whoever has the secret."

28.3 Books bought with points stayed on the shelf forever

Evidence: commits `ea962a9` then `2521929`. Like §28.1, this took two attempts — and the second replaced the first.

Problem	From the commit: *"Buying with points wrote a purchase record and deducted points but never touched the book, so a sold copy stayed on the shelf forever — still listed, still borrowable, and buyable again by anyone."*
Behaviour	Uday spends 200 points on *1984*. The book remains in the Collection, marked available. Rafi borrows it. Someone else buys it again. The library sells one physical copy repeatedly.
Second bug in the same area	*"Purchase also never checked availability, so a book someone else was reading could be sold out from under them."*
Root cause	The purchase flow treated points and the book as unrelated. It debited the balance and wrote a ledger entry, but never changed the book's state — an incomplete transaction boundary.
File	`backend/controllers/pointsController.js`

Attempt 1 — a `purchased` flag (`ea962a9`)

Added `purchased: Boolean` to the Book model and filtered purchased books out of listings, borrowing and stats. It also introduced the **claim-then-pay-then-release** pattern that survives today, with the reasoning recorded verbatim: "*Points live in Firestore and books in MongoDB, so there's no transaction spanning both. Claim the copy first with one atomic conditional update, then spend the points, and release the claim if payment fails.*"

A SUBTLE BUG FOUND INSIDE THE FIX

The commit records a second discovery: "*The claim matches `purchased: {$ne: true}` rather than `false`, because books seeded before this field existed have no such key and an equality match skips them.*"

This is an excellent interview anecdote. In MongoDB, `{ purchased: false }` matches only documents where the field *exists and equals false*. Documents predating the field have no key at all and are silently excluded — so every seeded book would have been unbuyable, with no error. `$ne: true` matches missing fields correctly. **Lesson: adding a field to an existing collection creates two populations of document, and equality queries only see one of them.**

Attempt 2 — delete outright (`2521929`) ✓ current behaviour

AFTER

From the commit: "*A borrowed book comes back to the library, so it stays in the collection marked unavailable. A book bought with points never comes back, so keep no record of it in the collection at all… This replaces the earlier 'purchased' flag, which left sold copies sitting in the database and visible to admin.*"

```
await preserveReferences(book);
await Book.findByIdAndDelete(book._id);
```

Deleting created a **new** problem — dangling references — which the same commit solved: "*Deleting the document orphans everything pointing at it, so snapshot first… otherwise a reader's history would render 'Untitled' and their review 'Unknown book'.*" That is §28.4.

28.4 "Untitled" and "Unknown book" — the dangling-reference bug

Evidence: commit `2521929`, plus explanatory comments now living in `models/Borrow.js`, `routes/borrow.js`, `reviewController.js` and `adminController.js`.

Problem	Deleting a purchased book orphaned every record referencing it.
Behaviour	A reader's borrow history showed **"Untitled"** with a broken cover image. My Reviews showed **"Unknown book"**. The admin member table showed "Untitled". Wishlist entries linked to a 404.
Root cause	`.populate("bookId")` resolves a reference with a **second query**, not a database join. When the target no longer exists it yields `null`, and `doc.bookId?.title` falls through to the fallback string.
Diagnosis	The symptom is specific and diagnostic: it appears *only* for books that were purchased, and *only* after the purchase. That points directly at the delete.
Files	`models/Borrow.js`, `routes/borrow.js`, `controllers/pointsController.js`, `controllers/reviewController.js`, `controllers/adminController.js`, `pages/MyReviews.jsx`, `pages/MyLibrary.jsx`

AFTER — THREE DIFFERENT FIXES FOR THREE DIFFERENT NEEDS

```
// ① Borrow records — snapshot at loan time, and again before delete
bookSnapshot: { title: String, author: String, coverImage: String }

await Borrow.updateMany(
  { bookId: book._id, "bookSnapshot.title": { $exists: false } },
  { $set: { bookSnapshot: snapshot } }
);

// ② Reviews — batch-write the snapshot onto every review of this book
reviews.docs.forEach((d) => reviewBatch.update(d.ref, {
  bookTitle: snapshot.title, bookAuthor: snapshot.author, bookCover: snapshot.coverImage }));
```

```
// ③ Wishlist — DELETE, because the book can never be obtained again
wishes.docs.forEach((d) => wishBatch.delete(d.ref));
```

And the read side reshapes the snapshot so the UI needs no special case:

```
if (!doc.bookId && doc.bookSnapshot) {
  doc.bookId = { _id: null, title: doc.bookSnapshot.title,
                 author: doc.bookSnapshot.author, coverImage: doc.bookSnapshot.coverImage };
  doc.bookGone = true;    the only thing the UI branches on
}
```

INTERVIEW LESSON

A document database has no foreign keys and no `ON DELETE CASCADE` — referential integrity is the application's job. The mature part of this fix is that it treats three reference types differently: **historical records** (borrows, reviews) are snapshotted because they must remain truthful about the past; a **forward-looking intention** (wishlist) is deleted because it has become impossible. Recognising that "handle dangling references" is not one decision but three is what makes this a strong answer.

28.5 Payment taken but order never recorded

Evidence: commit `72a88fb`.

Problem	From the commit: *"ProtectedRoute now preserves the query string when bouncing to login. It kept only the pathname, so a shopper whose session hadn't restored yet would come back from Stripe, get redirected to sign in, and lose `?session_id=` — leaving the payment taken but never confirmed."*
Behaviour	Customer pays successfully. Stripe returns them to `/cart?session_id=cs_test_…`. The guard sees `firebaseUser` as null (session still restoring), redirects to `/login` storing only `"/cart"`. After login they land on an ordinary cart page. **Stripe has the money; Bookverse has no order.**
Root cause	Two independent factors combining: (1) `ProtectedRoute` captured `location.pathname` and discarded `location.search`; (2) the auth-restoration race meant the guard could fire before the session was known.
Files	`components/ProtectedRoute.jsx`, `pages/Cart.jsx`, and the deleted `pages/CheckoutSuccess.jsx`

BEFORE

```
return <Navigate to="/login" state={{ from: location.pathname }} replace />;
// "/cart?session_id=cs_test_a1B2c3" → "/cart"   ← the id is gone
```

AFTER

```
// Keep the query string, not just the path — returning from Stripe carries
// ?session_id=..., and dropping it would lose the order confirmation.
const from = `${location.pathname}${location.search}`;
return <Navigate to="/login" state={{ from }} replace />;
```

The same commit also restructured the return journey: the standalone `/checkout/success` page was deleted and the cart itself became the receipt — *"it is where they left off, so the cart turning into a receipt reads as the same journey finishing rather than a detour"* — and the session id is now stripped from the URL after confirmation so a refresh cannot re-run it.

INTERVIEW LESSON

A redirect that discards part of the URL is a data-loss bug, and in a payment flow it is a money-loss bug. Two individually reasonable pieces of code — a route guard and a payment return — combined into a failure neither would produce alone. Worth pairing with the honest follow-up: even with this fixed, a customer who closes the tab still loses the order, which is why a webhook is the real answer (§16.9).

28.6 The refresh race — silent 401s on every hard load

Evidence: introduced in commit `c1c4113`; the reasoning is preserved as a comment in `services/api.js`. *Previously encountered — exact original broken code not available*, because the guard was present from the file's first commit. The comment documents the failure mode it was written to prevent.

Problem	On a hard page load, requests fire before Firebase has restored the persisted session.
Behaviour	Navigating to `/my-library` works. Pressing F5 on it returns `401` for every request and renders an empty page — despite a valid session.
Root cause	Session restoration is asynchronous. Reading `auth.currentUser` synchronously in the interceptor returns `null` during that window, so no `Authorization` header is attached and the request goes out anonymous.
File	`frontend/src/services/api.js`

THE GUARD

```
api.interceptors.request.use(async (config) => {
  // On a hard page load, Firebase hasn't yet restored the persisted session
  // at the moment a mounting component's first request fires — reading
  // auth.currentUser synchronously here would silently send the request
  // unauthenticated. authStateReady() waits for that initial restore.
  if (auth) await auth.authStateReady();
  ...
});
```

The sibling of this bug lives in `ProtectedRoute`: the same race would redirect a signed-in user to `/login` on every refresh, which is why `if (loading) return <Spinner />` is the component's first statement.

INTERVIEW LESSON

"Works when navigating, fails on refresh" almost always means an async initialisation race. The general principle: never read asynchronously-restored state synchronously — wait for the library's readiness signal.

28.7 The display-name race during signup

Evidence: the comment in `authController.syncUser`, written specifically to explain the `else if` branch.

Problem	A user registering as "Uday Barua" could end up permanently named "uday".
Root cause	Two calls to `/auth/sync` race during signup. `onAuthStateChanged` fires the instant the account is created — **before** `updateProfile()` has set `displayName` — so it posts `{ name: null }`, and the server falls back to `decoded.email.split("@")[0]`. Meanwhile `signup()` posts the real name. Whichever arrived first won.
File	`backend/controllers/authController.js`

THE FIX

```
} else if (name && name !== user.name) {
  // The signup form's explicit call and the auth-state-change listener's
  // call can race (the listener fires before updateProfile() finishes
  // setting displayName). Treat an explicitly provided name as authoritative
  // whenever it arrives, instead of only setting it at creation time.
  user.name = name;
  await user.save();
}
```

Why it works: it makes the operation **order-independent**. Rather than trying to sequence the two calls, an explicitly-supplied name is accepted whenever it arrives. **Lesson:** when you cannot control ordering, design the operation so ordering does not matter.

28.8 Route ordering — `/mine` captured as an id

Evidence: defensive comments in `routes/community.js` and `routes/reviews.js`. *Previously encountered — exact original code not available;* the comments exist precisely because the correct ordering looks arbitrary without them.

Express matches in registration order and stops at the first match. With `/books/:id` registered first, a request to `/books/mine` matches it with `id === "mine"`, the handler looks up a Firestore document literally named "mine", and returns 404 — so the contributions tab would be permanently empty **with no error anywhere**.

```
// "mine" is declared before "/books/:id" so it isn't swallowed as an id.
router.get("/books/mine", verifyFirebaseToken, listMyUsedBooks);
router.get("/books/:id",  getUsedBook);
```

Lesson: static path segments must always be registered before dynamic ones at the same depth. The bug is silent, which is what makes it dangerous.

28.9 The backend could not run on Vercel

Evidence: commit `2313592`.

Problem	The Express app called `app.listen()` at module scope, which is invalid in a serverless environment where the platform owns the listener.
Fix	*"Split Express app definition from the listen call so it can be reused as a Vercel serverless function (`api/index.js`) while `server.js` still works for local dev."*

```
// app.js — builds, never listens  module.exports = app;
// server.js — local            app.listen(PORT, ...);
// api/index.js — serverless     module.exports = app;
```

Lesson: separate **construction** from **invocation** and one codebase serves both a long-running server and a serverless function. A related consequence worth mentioning: serverless functions are stateless and cold-start, which is why the Mongoose connection is opened at module scope and reused across invocations.

28.10 The navbar was unreadable on light pages

Evidence: commit `a1a44ea`.

"The unscrolled navbar was fully transparent with cream-colored text, which only worked over pages with a dark hero image. Book Details, Community Book Details, Admin Setup and the auth pages have no dark hero, so the text blended into the page."

The fix gave the unscrolled navbar a persistent `navy-950/70` fill. What makes this worth including is the **method** recorded in the message: *"Verified against the actual page background: 7.88:1 contrast (WCAG AAA) versus 3.22:1 at a weaker 45% tried first."*

Lesson: a component that assumes a particular backdrop breaks the moment it is reused elsewhere. And accessibility is **measurable** — quoting a computed contrast ratio rather than "it looks fine now" is exactly the right instinct. A one-line diff with real rigour behind it.

28.11 An unnecessary dependency in the payment path

Evidence: commit `aef233a`.

"Checkout is Stripe-hosted: the server creates the session and the browser is redirected to `session.url`*, which needs no publishable key and no client SDK."* `@stripe/stripe-js` and the publishable-key environment variable were removed.

Not a runtime bug, but a real defect of a different kind: dead weight in the bundle and an unused credential in configuration. **Lesson:** removing an unnecessary dependency from a payment path reduces bundle size *and* attack surface. Being able to say "we ship no Stripe client code at all, so we never touch card data" is a stronger security statement than any amount of added protection.

28.12 A confusing field in the shipping form

Evidence: commit `5a0e3b4`.

"An ID card/passport number was confusing to test and unclear where a real user would get one for a demo. Phone number is the more realistic field for a delivery flow and needs no explanation."

A usability defect with a privacy dividend: a courier needs a phone number, never a government ID. **Lesson:** collect the minimum data the process actually requires — the least sensitive field that does the job is usually also the more usable one.

28.13 Top 10 Bookverse Problems I Should Be Able to Explain

#	PROBLEM	ROOT CAUSE	SOLUTION	CONCEPT
1	Google sign-in stranded users	Third-party storage partitioning blocks the cross-origin auth iframe	Reverse-proxy `/__/auth/*` and point `authDomain` at our own origin	OAuth redirect flows; first-party vs third-party storage
2	Payment taken, order lost	Route guard dropped `location.search`, losing `?session_id=`	Preserve pathname **and** query string when redirecting	Redirect state preservation; payment idempotency
3	Sold books stayed on the shelf	Purchase debited points but never changed the book's state	Claim atomically, then delete outright after payment	Transaction boundaries across services
4	"Untitled" / "Unknown book"	`populate()` returns null for deleted targets	Snapshot onto historical records; delete forward-looking ones	Referential integrity without foreign keys
5	401 on every refresh	Requests fired before Firebase restored the session	`await auth.authStateReady()` in the interceptor	Async initialisation races
6	Collection writes unauthenticated	No role field, no authorisation layer	Add `role` + `requireRole("admin")`	Authentication vs authorisation; RBAC
7	Two buyers, one copy	Check-then-act across two operations	`findOneAndUpdate({ _id, available: true })`	Atomic conditional updates; race conditions
8	Seeded books unbuyable	`{ purchased: false }` skips documents lacking the field	`{ purchased: { $ne: true } }`	Schema evolution in a schemaless store
9	Wrong display name on signup	Two `/auth/sync` calls racing; the null-name one could win	Accept an explicit name whenever it arrives	Order-independent operations
10	`/books/mine` returned 404	`/books/:id` registered first captured "mine" as an id	Register static segments before dynamic ones	Route matching order

Interview-ready answers

1 Google sign-in

"Google sign-in looked like it worked but left users logged out. The redirect leg completes through an iframe on the Firebase auth domain, and modern browsers partition third-party storage — so the session was written somewhere the site could never read. I first removed the redirect fallback to stop the silent failure, then fixed the cause: I reverse-proxied `/__/auth/*` to Firebase through our own domain and pointed `authDomain` at our site, making the handler first-party. Then I restored the fallback, because it finally worked."

2 Lost payment

"A customer paid, Stripe returned them to `/cart?session_id=…`, but if their session hadn't restored yet the route guard bounced them to login — storing only the pathname. The session id was lost and the order was never recorded. Fix was one line: capture `pathname + search`. The deeper fix is a Stripe webhook, so confirmation doesn't depend on the browser coming back at all."

3 Sold books

"Buying with points deducted the balance and wrote a receipt but never touched the book, so a sold copy stayed listed, borrowable and re-sellable. I first added a `purchased` flag, then replaced it with an outright delete — a borrowed book comes back, a sold one doesn't, so it shouldn't exist in the collection at all."

4 Dangling references

"Deleting purchased books made borrow history render 'Untitled' and reviews 'Unknown book', because `populate` yields null for a missing target. MongoDB has no cascade, so I handled it in the application — and deliberately differently per reference type. Borrows and reviews are historical records, so they got a snapshot of the title, author and cover. Wishlist entries are intentions that had become impossible, so they were deleted."

5 Refresh 401s

"Everything worked until you pressed F5, then every request 401'd. Firebase restores the persisted session asynchronously, and my Axios interceptor was reading `auth.currentUser` synchronously — null during that window, so requests went out with no token. One line fixed it: `await auth.authStateReady()`. The same race in the route guard is why it renders a spinner while loading instead of redirecting."

6 Unauthenticated writes

"The Collection's create, update and delete endpoints had no guard at all — anyone with the URL could delete a book. I added a `role` field with an enum, wrote a `requireRole` middleware, and gated all three. Bootstrapping the first admin uses a server-held secret once, which converts the secret into an account role — so every action afterwards is attributable to a named user rather than to whoever has the secret."

7 Concurrent buyers

"Two people buying the last copy simultaneously could both pass an `if (book.available)` check, because check-then-act is two operations with a gap. I made it one: `findOneAndUpdate({ _id, available: true }, { $set: { available: false } })`. MongoDB serialises them, so the second one's filter no longer matches and it gets null. I'll add that the borrow path still has this race — I know exactly how to fix it, because the purchase path already demonstrates it."

8 Schema evolution

"I added a `purchased` boolean and filtered with `{ purchased: false }` — and every seeded book became unbuyable, silently. In MongoDB an equality match only finds documents where the field *exists*; older documents had no such key. `{ $ne: true }` matches missing fields correctly. The lesson is that adding a field to an existing collection creates two populations of document, and equality queries only see one."

9 Name race

"New users could end up named after their email prefix. During signup two `/auth/sync` calls race — the auth-state listener fires before `updateProfile` sets `displayName`, so it posts a null name. Rather than trying to sequence them, I made the operation order-independent: an explicitly provided name is treated as authoritative whenever it arrives."

10 **Route ordering**

"The 'my contributions' tab was always empty with no error. `/books/:id` was registered before `/books/mine`, so Express matched the dynamic route with `id = "mine"` and looked up a document by that name. Static segments must be registered before dynamic ones at the same depth — and I left a comment on both routers that do this, because the ordering looks arbitrary otherwise."

28.14 The most technically challenging problems

1. The cross-database purchase (§11.5, §28.3)

Why it was hard: the book is in MongoDB and the points are in Firestore, so **no transaction can span both**. There is no framework answer — it required designing a distributed-transaction pattern by hand: an atomic claim, a transactional payment, and compensating rollback on two distinct failure paths (insufficient funds, and a thrown exception). Then the successful path had to clean up references across *both* databases before deleting. It also went through two designs before landing, and a subtle MongoDB query bug was discovered inside the first one. This is genuinely senior-level work, and it is the strongest thing in the project to talk about.

2. Google sign-in and browser storage partitioning (§28.1)

Why it was hard: the failure was **silent** — no error, no exception, no failed request. Everything looked correct. The cause was not in the code at all but in browser privacy behaviour interacting with where a third party serves an iframe. Diagnosing it required understanding OAuth redirect mechanics, third-party storage partitioning, and the difference between a symptom fix and a cause fix — the first attempt removed the fallback entirely before the second made it work. The solution spans application code, deployment configuration and an external OAuth client setting.

3. Referential integrity without foreign keys (§28.4)

Why it was hard: deleting one document silently corrupted the display of four unrelated features across two databases. There is no `ON DELETE CASCADE` to lean on. The hard part was not writing the code but the **judgement**: recognising that borrows, reviews and wishlists needed three *different* treatments, and that the snapshot should be reshaped to mimic a populated document so no UI component needed a special case.

4. The payment-return race (§28.5)

Why it was hard: it emerged from the interaction of three correct-looking things — a route guard, an async auth restore, and an external redirect — and it only reproduced under a specific timing condition, on a flow involving real money. Fixing the immediate bug was one line; recognising that the architecture still loses orders when a tab closes, and that a webhook is the actual answer, is the more valuable half.

5. Getting the whole authentication migration right (§28.6, §28.7)

Why it was hard: commit `c1c4113` replaced JWT/bcrypt with Firebase across 32 files, and asynchronous identity introduced an entire class of races that hand-rolled JWTs never had: the session restores asynchronously (breaking refresh), the auth-state listener fires before profile updates complete (breaking display names), and route guards can evaluate before the profile arrives (ejecting admins from their own dashboard). Three separate guards — `authStateReady()`, the authoritative-name branch, and `if (roles && !profile)` — exist because of three different manifestations of the same underlying shift from synchronous to asynchronous identity.

REFERENCE

Glossary of Technical Terms

Every term used in this document, defined and tied to where it appears in Bookverse.

TERM	DEFINITION	IN BOOKVERSE
Admin SDK	Firebase's server-side library, authenticating as a service account with full project access	Verifies tokens and performs every Firestore write; bypasses security rules
Atomic operation	An operation that completes entirely or not at all, with no observable intermediate state	`findOneAndUpdate`, `FieldValue.increment`
Authentication	Proving who someone is	`verifyFirebaseToken` → 401 on failure
Authorisation	Deciding what a known someone may do	`requireRole` + ownership checks → 403
Batch write	Multiple writes committed atomically as one operation	`db.batch()` — mark-all-read, wishlist delete, review snapshots
Bearer token	An HTTP auth scheme (RFC 6750): whoever holds the token may act as its subject	`Authorization: Bearer <ID token>`
CommonJS	Node's original module system — `require` / `module.exports`	The entire backend
Compensating transaction	An action that undoes an earlier step when a later step fails	`releaseClaim()` in the purchase saga
Composite key	A key made of two or more fields	`(source, bookId)` for reviews and wishlist items
Context (React)	A mechanism for sharing values down the tree without prop drilling	Auth, Cart, Wishlist
Controlled component	A form input whose value is driven by React state	Every form in the project
CORS	Browser policy governing cross-origin requests	`app.use(cors())` — currently unrestricted
Closure	A function retaining access to variables from its defining scope	`requireRole(...roles)` returning middleware
Debounce	Delaying an action until input stops	**Not implemented** — a known gap on search inputs
Defence in depth	Multiple independent controls guarding one asset	Store prices: write rules deny tampering *and* checkout re-reads prices
Denormalisation	Duplicating data to avoid joins on read	`ownerName`, `bookTitle`, wishlist display fields
ESM	ECMAScript Modules — `import` / `export`	The entire frontend
Fail closed	Denying by default when no rule explicitly permits	The `match /{document=**}` catch-all in `firestore.rules`
Higher-order function	A function taking or returning another function	`requireRole`
HMR	Hot Module Replacement — swapping modules without a full reload	Vite dev server
ID token	A signed JWT asserting an authenticated identity	Issued by Firebase, verified by `verifyIdToken`
Idempotent	Repeating the operation has the same effect as doing it once	Wishlist add; order confirmation; admin bootstrap
Interceptor	A hook running before every request or after every response	The Axios request interceptor attaching the token
JWT	JSON Web Token — header.payload.signature, base64url-encoded and signed	The Firebase ID token; signed not encrypted, so readable but unforgeable
Middleware	A function running between the request and the handler	`cors`, `express.json`, `verifyFirebaseToken`, `requireRole`, `errorHandler`
N+1 problem	Fetching a list then querying once per item	Avoided in `listMyReviews` (`$in`) and `listMembers` (two queries + in-memory join)
ODM	Object–Document Mapper — schemas and models over a document database	Mongoose
ObjectId	MongoDB's 12-byte primary key, encoding a timestamp	`Borrow.userId`, `Borrow.bookId`
Optimistic update	Updating the UI before the server confirms, reverting on failure	`WishlistContext.toggle`
Optional chaining	`?.` — short-circuits to `undefined` instead of throwing	`err.response?.data?.message`
Populate	Mongoose replacing a reference with the referenced document — a second query, not a join	`.populate("bookId")`; returns `null` for deleted targets
Projection	Requesting only specific fields	`.select("title author coverImage")`

TERM	DEFINITION	IN BOOKVERSE
Prop drilling	Passing props through components that don't use them	Avoided by the three contexts
Race condition	Behaviour depending on unpredictable operation ordering	The refresh 401s, the display-name race, the borrow check-then-act
RBAC	Role-Based Access Control	`role: "member" | "admin"` + `requireRole`
ReDoS	Denial of service via a regex with catastrophic backtracking	A theoretical risk in the unescaped `$regex` search
Referential integrity	Ensuring references point at existing records	Application-enforced via `preserveReferences()`
REST	Resource-oriented HTTP API style with meaningful verbs and status codes	46 endpoints; a few RPC-style action routes
Saga	A sequence of local transactions with compensating actions on failure	`purchaseOfficialBook`
Security rules	Declarative access control evaluated by Firestore itself	`firestore.rules` — read-scoped, all writes denied
Serverless function	Code invoked per request by a platform that owns the lifecycle	`backend/api/index.js` on Vercel
Server timestamp	A time value written by the database, not the client	`FieldValue.serverTimestamp()` on every `createdAt`
Session persistence	Keeping a user signed in across reloads	Firebase `browserLocalPersistence` (the default); Bookverse stores no token itself
Snapshot (data)	A copy of fields taken at a point in time so a record survives deletion of its source	`bookSnapshot`; review `bookTitle` / `bookAuthor` / `bookCover`
Snapshot (Firestore)	The result object of a query or listener	`snap.docs`, `snap.empty`, `doc.exists`
SPA	Single-Page Application — one HTML shell, client-side routing	The React frontend; `vercel.json` rewrites all paths to `index.html`
StrictMode	A React development mode that double-invokes effects to surface impurity	Why `Cart.jsx` uses a `useRef` latch
Third-party storage partitioning	Browsers isolating storage for cross-origin iframes	The cause of the Google sign-in bug (§28.1)
Transaction (Firestore)	Read-then-write executed atomically, retried on conflict	`db.runTransaction` when spending points
Upsert	Update if present, insert if not	`.set(..., { merge: true })`
Webhook	A server-to-server callback from an external service	**Not implemented** — the top production gap (§16.9)

END OF DOCUMENT

Everything in these 28 sections describes the Bookverse codebase as it exists in `D:\Library Project` on branch `main`. Where a capability is absent it is labelled as absent; where a design has a known weakness it is named rather than hidden. That honesty is not a caveat on the document — in an interview, it is the most valuable thing in it.